

Unterlagen für den Informatikunterricht in der Klasse 2fP

Jacques Mock Schindler

9. Juni 2026

Inhaltsverzeichnis

1	Obligatorisches Fach Informatik (2fP)	10
1.1	Programm	10
1.2	Hilfsmittel für den Unterricht	10
1.3	Beurteilung	11
I	Start	12
II	Anwendungen	13
2	Landesvermessung	14
2.1	Anwendungsübung	15
2.1.1	Arbeitsschritte	15
2.1.2	Berechnungen mit Hilfe von Python	16
III	Cybersicherheit	22
3	Cyberisiken – von der Lebenswelt der Jugendlichen zum gesellschaftlichen Problem	23
3.1	Cyberisiken im Alltag von Gymnasiastinnen und Gymnasiasten	23
3.2	Von der persönlichen Bedrohung zum gesellschaftlichen Risiko	24
3.3	Wer steckt dahinter? Ein Blick auf die Täterseite	25
	Literatur und Quellen	26
4	Cyber-Selbstverteidigung: Was der eigene Rechner verrät	29
4.1	Worum es geht	29
4.1.1	Lernziele	29
4.2	Rechtlicher und ethischer Rahmen – bitte zuerst lesen	29
4.3	Vorbereitung: Software installieren	30
4.3.1	Windows 11	30
4.4	Teil 1 – Welche Dienste lauschen auf meinem Rechner?	30
4.4.1	Beobachtungsfragen	31
4.5	Teil 2 – Portscan auf den eigenen Rechner	31
4.5.1	Beobachtungsfragen	32
4.6	Teil 3 – Wireshark beobachtet den Scan	32
4.6.1	Schritt 1: Wireshark starten und Interface wählen	32
4.6.2	Schritt 2: (Optional) Capture-Filter setzen	32

4.6.3	Schritt 3: Aufzeichnung starten	32
4.6.4	Schritt 4: Im zweiten Fenster den Scan ausführen	32
4.6.5	Schritt 5: Aufzeichnung stoppen und analysieren	33
4.6.6	Beobachtungsfragen	33
4.7	Teil 4 – Was tut der Rechner, wenn ich gar nichts tue?	33
4.7.1	Schritt 1: Vorbereitung	33
4.7.2	Schritt 2: Wireshark neu starten	33
4.7.3	Schritt 3: Zwei Minuten warten	34
4.7.4	Schritt 4: Mit Display-Filtern erkunden	34
4.7.5	Beobachtungsfragen	34
4.8	Teil 5 – Eine «harmlose» Website besuchen	34
4.8.1	Schritt 1: Vorbereitung	34
4.8.2	Schritt 2: Eine Website öffnen	34
4.8.3	Schritt 3: Drittparteien sichtbar machen	35
4.8.4	Beobachtungsfragen	35
4.8.5	Optional: Vergleich	35
4.9	Synthese: Was nehmen wir mit?	35
4.10	Optional / Weiterführendes	36
4.11	Anhang: Häufige Stolpersteine	36

IV Datenbanken II 37

5 Notebook-Übung: SQL-Aggregatfunktionen 38

5.1	Analyse einer Verkaufsdatenbank	38
5.2	Datenbank-Schema	39
5.3	Datenbank im Notebook	39
5.4	Abfrageübungen	40
5.5	Weiterführendes Training	41
5.6	Reflexion	41

6 Notebook-Übung: Tabellen verknüpfen mit SQL JOINS 43

6.0.1	Kontext	43
6.0.2	Das Wichtigste auf einen Blick	43
6.0.3	Setup	44
6.0.4	Aufgabe 1: INNER JOIN – Wer hat die Prüfung geschrieben?	47
6.0.5	Aufgabe 2: LEFT JOIN – Die vollständige Übersicht	47
6.0.6	Aufgabe 3: Online-Shop – JOIN + GROUP BY	49
6.0.7	Aufgabe 4: Sportverein – Wer hat (nicht) am Turnier teilgenommen?	50
6.0.8	Aufgabe 5: Bibliothek – Welche Bücher wurden nie ausgeliehen?	51
6.0.9	Bonus: JOIN über drei Tabellen	53
6.1	Reflexion	54

V	Nachhaltige IT	55
7	Kernbotschaft:	56
7.1	Einführung: Was ist der ökologische Fußabdruck der IT?	56
7.2	Transport	56
7.3	Bergbau	56
7.4	Konstruktion	57
7.5	Umweltkosten des Computing	57
7.5.1	Energieverbrauch	57
7.5.2	Grüne Rechenzentren	57
7.6	Nachhaltige Software	58
7.7	Coltan: Ein kritisches Mineral mit hohem Preis	58
7.7.1	Konflikte und Menschenrechte im Bergbausektor der DRK	58
7.7.2	Wirtschaftliche und ökologische Auswirkungen	59
7.7.3	Herausforderungen der globalen Lieferkette	59
8	Green IT – Datenvisualisierungsprojekt	61
8.1	Lernziele	61
8.2	Aufgabe 1: Power Usage Effectiveness (PUE)	61
8.2.1	Gute Visualisierungspraxis	62
8.3	Aufgabe 2: Elektroschrott-Analyse	62
8.4	Quellenverzeichnis	63
9	Open-Source-Software	65
9.1	Nutzung von Open-Source-Software	65
9.2	Beispiele für Open-Source-Software	66
9.3	Open-Source-Lizenzen	66
9.4	Geschäftsmodelle auf Basis von Open-Source-Software	66
9.5	Ausblick	67
10	Redaktionsübung	68
10.1	Orientierung	68
10.2	Absicht	68
10.3	Aufgabe	68
10.4	Spezifische Anforderungen	68
10.4.1	Inhalt	68
10.4.2	Formale Anforderungen	69
10.4.3	Quellen	69
10.4.4	Bewertung	69
10.4.5	Allgemeine Informationen	70
10.5	Kontakt	71
VI	Kryptologie	72
11	File Handling in Python	73

12 Caesar Chiffre	75
12.1 Python Implementation	75
12.1.1 Simple (naive) Implementation	75
12.1.2 Verbesserte Implementation	76
12.2 Caesar Chiffre brechen	77
12.2.1 Brute Force Attack	78
12.2.2 Häufigkeitsanalyse	78
13 Polyalphabetische Chiffren	80
13.1 Vigenère Chiffre	80
13.2 Vigenère Chiffre	85
14 Vigenère Chiffre Implementierung in Python	91
15 Die Vigenère-Verschlüsselung brechen	95
15.1 Die Kasiski-Methode – Zusammenfassung	95
15.1.1 Grundprinzip	95
15.1.2 Schritt-für-Schritt-Anleitung	95
15.1.3 Beispiel	96
15.1.4 Warum es funktioniert	97
16 Public-Key-Kryptographie	98
16.1 Verschlüsselung mit Hilfe eines Graphen	98
16.2 Definition Dominierende Menge	100
17 RSA	102
17.1 Lernziele	102
17.2 Die Mathematik hinter RSA	102
17.2.1 Tipps	102
17.2.2 Signieren	103
VII Computernetzwerke	104
18 Computernetzwerke	105
18.1 Das OSI- bzw. TCP/IP-Modell	105
18.2 Analogie zur Kapselung in der objektorientierten Programmierung (OOP)	107
19 IP Adressen und DNS	108
19.1 IP Adressen	108
19.2 DNS Lookup	108
19.2.1 Funktionsweise	108
19.2.2 Typen von DNS-Einträgen	109
19.2.3 Bedeutung	109
20 Network Address Translation (NAT)	110
20.1 Was ist NAT?	110

20.2	Funktionsweise von NAT	110
20.2.1	Grundprinzip	110
20.2.2	NAT-Tabelle	110
20.3	Private IP-Adressbereiche	111
20.4	Arten von NAT	111
20.4.1	1. Static NAT (One-to-One NAT)	111
20.4.2	2. Dynamic NAT	111
20.4.3	3. Port Address Translation (PAT)	111
20.5	Vorteile und Nachteile	112
20.5.1	Vorteile	112
20.5.2	Nachteile	112
20.6	Praktisches Beispiel	112
20.7	Bedeutung für die Zukunft	113
21	Top Level Domains und ihre Vergabe	114
21.1	Was sind Top Level Domains?	114
21.2	Arten von Top Level Domains	114
21.2.1	1. Generic Top Level Domains (gTLDs)	114
21.2.2	2. Country Code Top Level Domains (ccTLDs)	114
21.2.3	3. New Generic Top Level Domains (ngTLDs)	115
21.3	Die Vergabestelle: ICANN und IANA	115
21.3.1	Hierarchie der Vergabe	115
21.4	Der Vergabeprozess für neue gTLDs	115
21.4.1	Phase 1: Bewerbung	115
21.4.2	Phase 2: Evaluation	115
21.4.3	Phase 3: Delegation	116
21.5	Kosten und Gebühren	116
21.5.1	Für Registry-Betreiber	116
21.5.2	Für Endkunden (Beispiele)	116
21.6	Besondere Regelungen	116
21.6.1	Restricted TLDs	116
21.6.2	Internationalized Domain Names (IDN)	116
21.7	Statistische Betrachtung	117
21.8	Bedeutung und Ausblick	117
22	Beobachten von Internetverbindungen	118
22.1	Aufzeichnen von Netzwerkpaketen	118
22.2	Beobachten der DNS-Anfragen	120
22.2.1	Filtern der Aufzeichnung	120
22.2.2	Analyse der gefilterten Pakete	121
22.3	Beobachtung des Verbindungsaufbaus	123
23	Python Sockets: Ein Einfaches Client-Server Beispiel	127
23.1	Struktur der Verbindung	127
23.2	Server Code	127
23.3	Client Code	130

23.4	Verbindung im LAN	132
23.5	Fazit	132
VIII	Datenbanken	134
24	Einführung in Datenbanken	135
24.1	Einführung	135
24.2	Charakteristika von Datenbanken	135
25	Abfrage von Daten	139
25.1	Grundstruktur einer SQL Abfrage	139
25.1.1	Einfache Abfrage	139
25.1.2	Abfrage mit Bedingung	140
25.1.3	Sortierung der Ausgabe	140
25.2	Abfrage aus mehreren Tabellen	141
25.3	Ausblick	142
26	Lernziele für die Prüfung vom 21. Mai 2025	143
IX	Datenstrukturen und Codierung	144
27	Datenstrukturen Stack und Queue	145
28	Queues in Python	148
29	Binary Search Tree	150
29.1	Klasse BSTNode	150
29.2	Klasse BST	151
30	Das Binärsystem	155
30.1	Umrechnung von Dezimalzahlen in Binärzahlen	155
30.1.1	Beispiel: Umrechnung von 42 ins Binärsystem	155
30.1.2	Überprüfung:	156
30.2	Umrechnung von Binärzahlen in Dezimalzahlen	156
30.2.1	Methode:	156
30.2.2	Beispiel: Umrechnung von 10110_2 ins Dezimalsystem	156
30.2.3	Formel:	156
30.2.4	Praktischer Trick:	157
30.3	Binärcodierung von Strings	157
30.3.1	Unicode: Ein universeller Zeichencode	157
30.3.2	UTF-8: Die häufigste Unicode-Codierung	157
30.3.3	UTF-8 Codierungsschema:	157
30.3.4	Beispiel: Codierung deutscher Umlaute	158
30.3.5	Weiteres Beispiel: Das Euro-Zeichen €	158
30.3.6	Beispiel: 'Informatik ist interessant.'	158

30.3.7 Vorteile der Unicode-Codierung:	159
31 Python Funktionen zum Umrechnen zwischen den Zahlensystemen	160
31.1 Umrechnung von Binär- in Dezimalzahlen	160
31.2 Umrechnung von Dezimal- in Binärzahlen	160
31.3 Binäre Repräsentation von Strings	160
31.4 Umwandlung der binären Repräsentation in einen String	161
32 Python Funktionen zum Umrechnen zwischen den Zahlensystemen	162
32.1 Umrechnung von Binär- in Dezimalzahlen	162
32.2 Umrechnung von Dezimal- in Binärzahlen	162
32.3 Binäre Repräsentation von Strings	163
32.4 Umwandlung der binären Repräsentation in einen String	163
33 Base64-Codierung	164
33.1 Prinzip der Base64-Codierung	164
33.2 Base64-Zeichensatz	165
33.3 Codierungsprozess	165
33.4 Beispiele:	165
33.4.1 Beispiel 1: Codierung von "KBW"	165
33.4.2 Beispiel 2: Codierung von "Hallo"	165
33.5 Anwendungen der Base64-Codierung	166
34 Lernziele für die Prüfung vom 26. März 2025	167
X Künstliche Intelligenz	168
35 The Social Dilemma	169
36 Was ist eine Künstliche Intelligenz?	170
37 Maschinelles Lernen: Punkt- und Linienklassifikatoren	171
38 Maschinelles Lernen: Gewinnstrategie	172
XI Anleitungen	173
39 Installation von Python	174
39.1 Python herunterladen und installieren	174
39.2 Installation überprüfen	174
39.3 Erste Schritte mit Python	175
40 Anleitung zum Arbeiten mit Python Virtual Environments	176
40.1 Arbeiten mit einer bereits bestehenden venv	176
40.2 Einrichten einer neuen venv	177

41 Jupyter Notebook	178
41.1 Installationsanleitung	178
41.2 Jupyter Notebook starten	179
41.3 Ihr erstes Notebook erstellen	179
41.4 Tipps zur Verwendung	179
41.5 Bearbeiten von Jupyter Notebooks in VS Code	179
42 Plain Text Writing	181
42.1 Text- und Binärdateien	181
42.2 Plain Text Formate für die Textredaktion	181
43 Konvertierung von Markdown in PDF	183
43.1 Vorbereitung der Konvertierung	183
43.2 Spezifische Formatierung der Ausgabe	184
44 Referenzen in Markdown	186
44.1 Textformat (.yaml)	186
44.2 Bibliographische Angaben	188
44.3 Zitierstil	188
44.4 Einfügen von Referenzen in Markdown	189
44.5 Erstellen des PDF mit Referenzen	189
45 Literaturverwaltung mit Zotero	190
45.1 Zotero installieren und einrichten	190
45.2 Einträge in Zotero erstellen und organisieren	190
45.3 Erstellen einer Bibliographie Datei für den Markdown – Pandoc-Workflow	191
XII Alte Programme	192
46 Programm im Frühlingssemester 2025	193

1 Obligatorisches Fach Informatik (2fP)

In diesem Repository sollen die Unterlagen für das obligatorische Fach Informatik zusammengetragen werden.

1.1 Programm

Das Programm entspricht dem aktuellen Stand der Planung. Es kann zu Änderungen kommen. Das bisherige Skript findet sich [hier](#).

Datum	Thema
20.02.2026	Einführung: Digitale Nachhaltigkeit & Green IT
27.02.2026	Open Source Software: Philosophie, Lizenzen & Praxis
06.03.2026	Digitale Nachhaltigkeit: Fallstudien
13.03.2026	LEISTUNGSNACHWEIS 1 Textabgabe
20.03.2026	SQL Vertiefung: Aggregatfunktionen & GROUP BY
27.03.2026	SQL Vertiefung: Subqueries
10.04.2026	SQL DML: INSERT, UPDATE, DELETE
17.04.2026	Datenbankdesign: Normalisierung (1NF-3NF)
08.05.2026	Datenbank-Miniprojekt
22.05.2026	Repetition: SQL & Datenbanken
29.05.2026	LEISTUNGSNACHWEIS 2 Schriftliche Prüfung
05.06.2026	Informationssicherheit
12.06.2026	Datenschutz: DSGVO & DSG Schweiz
19.06.2026	KI-Vertiefung: Ethik & aktuelle Entwicklungen
26.06.2026	Projekt: Datenethik-Fallstudie
03.07.2026	Projektpräsentationen
10.07.2026	Semesterabschluss & Ausblick

1.2 Hilfsmittel für den Unterricht

Der Unterricht findet über weite Strecken am Computer statt. Die Anleitungen für die Installation von Python bzw. der Jupyter Notebooks sind in der Seitenliste aufrufbar.

1.3 Beurteilung

Die Note wird aus dem Durchschnitt der beiden schriftlichen Prüfungen sowie der Benotung der mündlichen Beteiligung berechnet. Der Durchschnitt der Prüfungen zählt zu 90%, die mündliche Beteiligung zu 10%.

Ab dem zweiten Semester wird konstruktive Kritik an den Unterlagen, welche durch ein mir zugewiesenes Issue (der Zugang findet sich unter dem GitHub-Logo  in der Kopfzeile) eingebracht wird, mit Maximal einem Notenpunkt auf die jeweils thematisch passende Prüfung angerechnet.

Falls jemand eine persönliche Besprechung wünscht, kann hier einen Termin für eine Sprechstunde reservieren (Rent-A-Mock).

Teil I

Start

Teil II

Anwendungen

2 Landesvermessung

Die Landesvermessung basiert traditionellerweise im Wesentlichen auf der Anwendung von trigonometrischen Funktionen. Man spricht daher auch von Triangulation.

i Trigonometrische Grundlagen

In einem allgemeinen Dreieck (vgl. Abbildung 2.1) gelten die folgenden beiden Sätze:

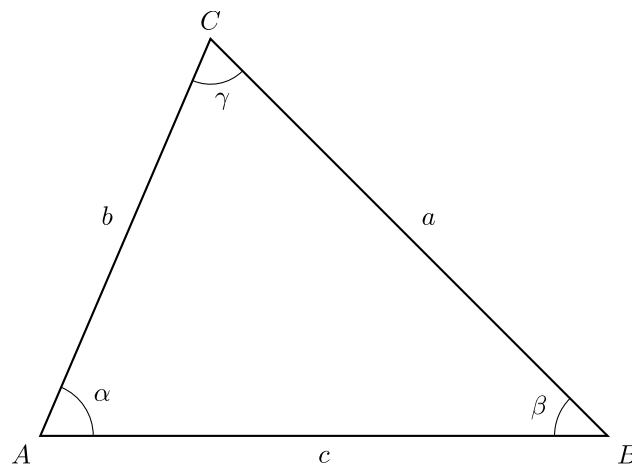


Abbildung 2.1: Allgemeines Dreieck.

Sinussatz

$$\frac{a}{\sin \alpha} = \frac{b}{\sin \beta} = \frac{c}{\sin \gamma}$$

Kosinussatz

$$a^2 = b^2 + c^2 - 2 \cdot b \cdot c \cdot \cos \alpha$$

$$b^2 = c^2 + a^2 - 2 \cdot c \cdot a \cdot \cos \beta$$

$$c^2 = a^2 + b^2 - 2 \cdot a \cdot b \cdot \cos \gamma$$

2.1 Anwendungsübung

Im Folgenden soll das anhand der Vermessung einer Strecke in unzugänglichem Gelände illustriert und angewendet werden.

Gegeben sei die folgende Ausgangslage (vgl. Abbildung 2.2):

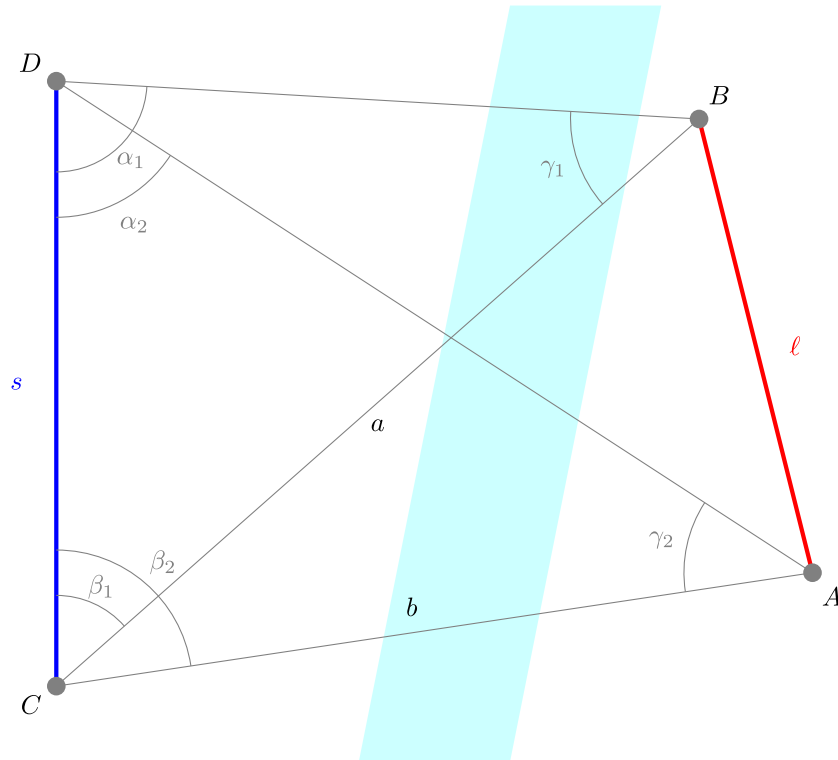


Abbildung 2.2: Vermessungsskizze.¹

Gesucht ist die Strecke $\overline{AB}$ (ℓ). Gemessen werden kann die Strecke $\overline{CD}$ (s) sowie die Winkel α_1 , α_2 , β_1 und β_2 . Dabei liegen α_1 und α_2 im Punkt D , β_1 und β_2 im Punkt C .

2.1.1 Arbeitsschritte

Die Idee ist, die Basisstrecke $s = \overline{CD}$ über zwei Hilfsdreiecke mit der gesuchten Strecke zu verknüpfen. Aus dem Dreieck CDB erhält man die Strecke $\overline{CB}$, aus dem Dreieck CDA die Strecke $\overline{CA}$. Beide Strecken teilen sich den Punkt C ; der von ihnen eingeschlossene Winkel ist $\beta_2 - \beta_1$. Damit lässt sich die Zielstrecke im Dreieck CAB über den Kosinussatz bestimmen.

Mit den gemessenen Größen lassen sich der Reihe nach die folgenden fehlenden Größen berechnen:

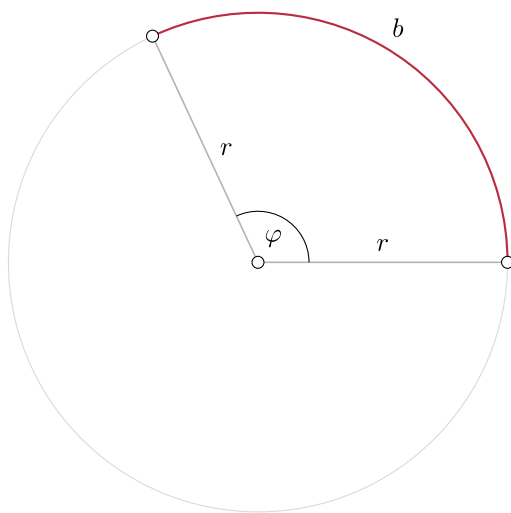
¹Ernst-Erich Doberkat und Michael Jung, *Ebene Trigonometrie & Analytische Geometrie: Grundlagen und Anwendungen für Geodäsie, Karographie und verwandte Disziplinen* (Springer Berlin Heidelberg, 2024), 56, zugegriffen 4. Juni 2026, https://doi.org/10.1007/978-3-662-68843-4_1.

1. $\gamma_1 = 180^\circ - (\alpha_1 + \beta_1)$
2. Mit Hilfe des Sinussatzes (Strecke gegenüber α_1 , d.h. $\overline{CB}$): $a = \frac{s \cdot \sin \alpha_1}{\sin \gamma_1}$
3. $\gamma_2 = 180^\circ - (\alpha_2 + \beta_2)$
4. Mit Hilfe des Sinussatzes (Strecke gegenüber α_2 , d.h. $\overline{CA}$): $b = \frac{s \cdot \sin \alpha_2}{\sin \gamma_2}$
5. Mit Hilfe des Kosinussatzes im Dreieck CAB : $\ell = \sqrt{a^2 + b^2 - 2ab \cos(\beta_2 - \beta_1)}$

2.1.2 Berechnungen mit Hilfe von Python

Es ist deutlich einfacher, alle Messwerte einer Python-Funktion zu übergeben, als alle Schritte manuell mit Hilfe eines Taschenrechners einzeln zu berechnen. Dabei ist allerdings zu beachten, dass Python Winkelberechnungen in Bogenmass und nicht in Grad vornimmt.

i Winkel in Bogenmass



Das Bogenmass $\widehat{\varphi}$ eines Winkels ist das Verhältnis der Länge des vom Winkel φ ausgeschnittenen Kreisbogens zum Radius des Kreises. Es wird auch mit $\text{arc } \varphi$ bezeichnet.

$$\widehat{\varphi} = \frac{b}{r} = \text{arc } \varphi = \frac{\pi}{180^\circ} \varphi \iff \varphi = \frac{180^\circ}{\pi} \widehat{\varphi}$$

Auf dem Einheitskreis ($r = 1$) gilt: $\widehat{\varphi} = b$. Die Einheit des Bogenmasses ist der Radiant (rad). Ein Winkel von 1 rad entspricht einem Kreisbogen, der dieselbe Länge wie der Radius hat.^a

^aWerner Durandi u. a., *Formeln, Tabellen, Begriffe: Mathematik - Physik - Chemie*, 8., aktualisierte Auflage, hg. von Verein Schweizerischer Mathematik- und Physiklehrkräfte und Werner Durandi (Orell Füssli Verlag, 2022), 91.

Glücklicherweise stellt Python im Modul `math` die Funktion `math.radians()` zur Verfügung, mit welcher Grad in Bogenmass umgerechnet werden können. Hierzu muss das Modul vorab mit `import math` importiert werden. Das Modul `math` stellt darüber hinaus auch alle erforderlichen trigonometrischen Funktionen (`math.sin()` und `math.cos()`) zur Verfügung.

Um die Python-Funktion so schlank wie möglich und gut lesbar zu halten, wird als erstes eine Hilfsfunktion implementiert, welche Grad in Bogenmass umrechnet. Weil es aber auch Kompass mit einer Eichung in A‰ und Gon gibt, soll die Funktion alternativ auch Werte in diesen Einheiten umrechnen.

```

1 import math

1 def angle_converter(angle: float, mil: bool = False, gon: bool = False)
  ↪ -> float:
2     """Konvertiert einen Winkel in das Bogenmass (Radiant).
3
4     Unterstützt die Umrechnung von Altgrad (Standard),
  ↪ Artillerie-Promille
5     oder Gon. Es darf jeweils nur ein Modus aktiv sein.
6
7     Args:
8         angle (float): Der zu konvertierende Winkelwert.
9         mil (bool, optional): Wenn True, wird der Winkel als
  ↪ Artillerie-Promille
10        (A‰) interpretiert. Standard ist False.
11        gon (bool, optional): Wenn True, wird der Winkel als Gon
  ↪ (Neugrad)
12        interpretiert. Standard ist False.
13
14    Returns:
15        float: Der umgerechnete Winkel im Bogenmass.
16
17    Raises:
18        ValueError: Wenn sowohl 'mil' als auch 'gon' auf True gesetzt
  ↪ sind.
19    """
20    if mil and gon:
21        raise ValueError(
22            "Es kann nur 'mil=True' (Artillerie) ODER 'gon=True'
  ↪ gewählt werden."
23        )
24
25    if mil:
26        return angle * (math.pi / 3200)
27    elif gon:
28        return angle * (math.pi / 200)
29    else:
30        return math.radians(angle)

```

Wie die für die Berechnung erforderlichen Winkel gemessen werden, soll am Beispiel von α_1 in

Abbildung 2.2 gezeigt werden. Dazu wird mit dem Kompass von der Position D aus der Azimut nach B und nach C gemessen. Die Differenz der beiden Messungen entspricht α_1 .

Um diese Berechnung in einer Python-Funktion zu implementieren, muss sie als Algorithmus beschrieben werden. Dabei gilt es den Fall zu berücksichtigen, dass die beiden Messungen die Nordrichtung einschliessen. Um auch diese Situation abzudecken, bedient man sich der *modularen Subtraktion*.

i Modulare Subtraktion

In der modularen Arithmetik geht es darum, das uns bekannte Rechnen mit Zahlen aus einer unendlichen Menge auf endliche Zahlenmengen zu übertragen.^a So kann sichergestellt werden, dass das Resultat der Winkelberechnung innerhalb der Windrose bleibt.

Allgemein für 360° $(a - b) \pmod{360}$

1. Differenz berechnen: $d = a - b$
2. Modulo bilden: $r = d \pmod{360}$

Der in der klassischen Darstellung übliche dritte Schritt – «falls das Ergebnis negativ ist, einmal 360 addieren» – entfällt in Python: Der Operator % liefert bei positivem Divisor stets ein Ergebnis im Bereich $[0, 360)$ und normalisiert damit automatisch.

Zahlenbeispiel $40^\circ - 110^\circ$

1. $40 - 110 = -70$
2. $-70 < 0$: Wahr
3. $-70 + 360 = 290$
4. $40 - 110 \equiv 290 \pmod{360}$

^aSebastian Iwanowski und Rainer Lang, *Diskrete Mathematik mit Grundlagen: Lehrbuch für Studierende von MINT-Fächern* (Springer Fachmedien Wiesbaden, 2021), 166, zugegriffen 24. Mai 2026, <https://doi.org/10.1007/978-3-658-32760-6>.

Implementiert in einer Python-Funktion sieht das folgendermassen aus:

```
1 def angle_diff(angle1: float, angle2: float) -> float:
2     """Berechnet die modulare Differenz (angle1 - angle2) modulo 360
3         ↪ Grad.
4
5     Berücksichtigt automatisch den Nord-Übergang, indem negative
6     ↪ Differenzen
7     in den positiven Kreislauf [0, 360) überführt werden.
8
9     Args:
10         angle1 (float): Der Ausgangswinkel (z. B. erste Peilung).
11         angle2 (float): Der abzuziehende Winkel (z. B. zweite Peilung).
12
13     Returns:
```

```

12         float: Die modulare Winkeldifferenz im Bereich von 0 bis <360
    ↪ Grad.
13         """
14         return (angle1 - angle2) % 360

```

Nun braucht es noch eine Hilfsfunktion für den Sinussatz.

```

1 def sine_rule(length: float, opposite_angle: float, ref_angle: float)
    ↪ -> float:
2     """Berechnet eine Dreieckseite mit Hilfe des Sinussatzes.
3
4     Bestimmt die Seite, die dem Winkel ``opposite_angle``
    ↪ gegenüberliegt,
5     ausgehend von einer bekannten Seite ``length`` und deren gegenüber-
6     liegendem Winkel ``ref_angle``:
7
8         gesuchte Seite = length * sin(opposite_angle) / sin(ref_angle)
9
10    Alle Winkel müssen im Bogenmass (Radiant) übergeben werden.
11
12    Args:
13        length (float): Bekannte Seite (hier die Basisstrecke s).
14        opposite_angle (float): Winkel gegenüber der gesuchten Seite
15                               (im Bogenmass).
16        ref_angle (float): Winkel gegenüber der bekannten Seite
    ↪ ``length``
17                               (im Bogenmass).
18
19    Returns:
20        float: Die Länge der gesuchten Seite.
21        """
22    return (length * math.sin(opposite_angle)) / math.sin(ref_angle)

```

Damit liegend die erforderlichen Hilfsfunktionen vor und die eigentliche Berechnung entlang der Formel $\ell = \sqrt{a^2 + b^2 - 2ab \cos(\beta_2 - \beta_1)}$ kann als Python-Funktion implementiert werden.

```

1 def triangulation(s: float,
2                 alpha1_a: float, alpha1_b: float,
3                 alpha2_a: float, alpha2_b: float,
4                 beta1_a: float, beta1_b: float,
5                 beta2_a: float, beta2_b: float,
6                 mil: bool = False,
7                 gon: bool = False) -> float:
8     """Berechnet eine unzugängliche Strecke mittels Triangulation aus
    ↪ Kompasspeilungen.

```

```

9
10     Bestimmt die Länge der unbekannten Zielstrecke AB (l) aus einer
↪     bekannten
11     Basislänge CD (s) und den zugehörigen Azimut-Peilungen. Aus je zwei
12     Peilungen wird über die modulare Differenz ein innerer
↪     Dreieckswinkel
13     gebildet. Über den Sinussatz werden daraus die Strecken von C nach
↪     B bzw.
14     von C nach A bestimmt; der Kosinussatz im Dreieck C-A-B liefert
↪     schliesslich
15     die Zielstrecke, wobei der von beiden Strecken in C eingeschlossene
↪     Winkel
16     der Differenz beta2 - beta1 entspricht.
17
18     Alle vier Teilwinkel werden aus je zwei Kompasspeilungen (a und b)
19     rekonstruiert.
20
21     Args:
22         s (float): Die gemessene, zugängliche Basisstrecke (CD).
23         alpha1_a (float): Erste Kompasspeilung für den Teilwinkel Alpha
↪     1 (bei D).
24         alpha1_b (float): Zweite Kompasspeilung für den Teilwinkel
↪     Alpha 1 (bei D).
25         alpha2_a (float): Erste Kompasspeilung für den Teilwinkel Alpha
↪     2 (bei D).
26         alpha2_b (float): Zweite Kompasspeilung für den Teilwinkel
↪     Alpha 2 (bei D).
27         beta1_a (float): Erste Kompasspeilung für den Teilwinkel Beta 1
↪     (bei C).
28         beta1_b (float): Zweite Kompasspeilung für den Teilwinkel Beta
↪     1 (bei C).
29         beta2_a (float): Erste Kompasspeilung für den Teilwinkel Beta 2
↪     (bei C).
30         beta2_b (float): Zweite Kompasspeilung für den Teilwinkel Beta
↪     2 (bei C).
31         mil (bool, optional): Wenn True, werden alle Peilungswerte als
32         Artillerie-Promille (A2) interpretiert. Standard ist False.
33         gon (bool, optional): Wenn True, werden alle Peilungswerte als
34         Gon (Neugrad) interpretiert. Standard ist False.
35
36     Returns:
37         float: Die berechnete Länge der unzugänglichen Zielstrecke (l).
38
39     Raises:
40         ValueError: Wenn über die Hilfsfunktionen sowohl 'mil' als auch
↪     'gon'

```



```

41         auf True gesetzt sind.
42     """
43     # Aus je zwei Peilungen den inneren Teilwinkel als modulare
44     ↪ Differenz bilden.
45     input_angles = [alpha1_a, alpha1_b, alpha2_a, alpha2_b,
46                     beta1_a, beta1_b, beta2_a, beta2_b]
47     angles = []
48     for i in range(0, len(input_angles), 2):
49         diff = angle_diff(input_angles[i], input_angles[i + 1])
50         angles.append(diff)
51
52     # In Bogenmass umrechnen.
53     alpha1, alpha2, beta1, beta2 = (
54         angle_converter(angle, mil, gon) for angle in angles
55     )
56
57     # Hilfsdreiecke an der Basis s: gamma liegt jeweils gegenüber s.
58     gamma1 = math.radians(180) - (alpha1 + beta1)
59     gamma2 = math.radians(180) - (alpha2 + beta2)
60
61     # Sinussatz liefert die Strecke gegenüber alpha (von C aus): CB
62     ↪ bzw. CA.
63     strecke_cb = sine_rule(s, alpha1, gamma1)
64     strecke_ca = sine_rule(s, alpha2, gamma2)
65
66     # Kosinussatz im Dreieck C-A-B; eingeschlossener Winkel in C ist
67     ↪ beta2 - beta1.
68     l = math.sqrt(
69         strecke_cb**2 + strecke_ca**2
70         - 2 * strecke_cb * strecke_ca * math.cos(beta2 - beta1)
71     )
72     return l

```

Teil III

Cybersicherheit

3 Cyberrisiken – von der Lebenswelt der Jugendlichen zum gesellschaftlichen Problem

Unterlagentext für den Unterricht an der Gymnasialen Oberstufe

3.1 Cyberrisiken im Alltag von Gymnasiastinnen und Gymnasiasten

Das Smartphone liegt auf dem Pult, Benachrichtigungen popen auf, und nebenbei läuft noch ein Tab mit dem nächsten YouTube-Video. Die digitale Welt ist für Schülerinnen und Schüler an Gymnasien selbstverständlicher Lebensraum – und genau deshalb ist sie auch ein Raum mit realen Risiken.¹

Phishing und Social Engineering begegnen Jugendlichen häufiger, als sie denken. Eine gefälschte E-Mail vom «Spotify-Support», eine SMS mit einem Link zu einem angeblichen Paketdienst, oder eine Instagram-Nachricht eines unbekannten Accounts mit einem «exklusiven Gewinn» – hinter all diesen Kontaktversuchen steckt dasselbe Ziel: die Empfängerin oder den Empfänger dazu zu bringen, Zugangsdaten preiszugeben oder Schadsoftware zu installieren.²

Der entscheidende Punkt: Social Engineering funktioniert nicht durch technische Überlegenheit, sondern durch gezielte psychologische Manipulation. Angreifer nutzen dabei eine Handvoll bewährter Mechanismen. *Dringlichkeit* erzeugt Stress und verhindert ruhiges Nachdenken («Ihr Konto wird in 24 Stunden gesperrt»). *Autorität* senkt die Hemmschwelle zu reagieren («Schreiben von der IT-Abteilung der Schule»). *Verknappung* weckt Angst, etwas zu verpassen («Nur noch 2 Plätze verfügbar»). *Vertrautheit* täuscht eine bestehende Beziehung vor – etwa indem ein Angreifer den Namen einer echten Kollegin oder eines echten Kollegen verwendet, dessen Profil er zuvor ausgespäht hat.³ Wer diese Muster kennt, kann innehalten und eine vermeintlich dringende Nachricht mit kühlem Kopf beurteilen, bevor er handelt.

¹Bundesamt für Sozialversicherungen (BSV), *Jugend Und Medien: Chancen Und Risiken Der Digitalen Welt* (Bundesamt für Sozialversicherungen / Nationales Programm zur Förderung von Medienkompetenz, 2023).

²Bundesamt für Cybersicherheit (BACS), *Halbjahresbericht 2024/I: Lage Der Informationssicherheit in Der Schweiz* (Bundesamt für Cybersicherheit, 2024); Schweizerische Kriminalprävention (SKP), *Cyberkriminalität: Prävention Und Schutz* (Schweizerische Kriminalprävention, 2023).

³European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024* (ENISA, 2024).

Identitätsdiebstahl ist eine unmittelbare Konsequenz erfolgreicher Phishing-Angriffe. Wer die Zugangsdaten eines fremden Accounts übernimmt, kann im Namen der betroffenen Person kommunizieren, einkaufen oder in sozialen Netzwerken Schaden anrichten. Für Jugendliche besonders heikel: Ein kompromittierter Account kann peinliche oder rufschädigende Inhalte verbreiten, die lange nachwirken.⁴

Cybermobbing und Deepfakes – Cybermobbing ist technisch gesehen kein klassischer «Cyberangriff», aber eines der wirkungsmächtigsten digitalen Risiken in der Altersgruppe der 15- bis 20-Jährigen.⁵ Die Möglichkeit, anonym zu agieren, Inhalte sekundenschnell zu verbreiten und Betroffene rund um die Uhr zu erreichen, verschärft die psychischen Folgen erheblich gegenüber traditionellem Mobbing.

Eine neue, besonders gravierende Dimension kommt durch **Deepfakes** hinzu. Mit frei verfügbaren KI-Tools lassen sich heute täuschend echte Bild-, Audio- und Videoaufnahmen erstellen, die eine reale Person in Situationen zeigen oder sprechen lassen, die nie stattgefunden haben.⁶ Im schulischen Umfeld bedeutet das: Mitschülerinnen oder Mitschüler können ohne technisches Vorwissen kompromittierende Aufnahmen von anderen erstellen und verbreiten. Die Folgen für die Betroffenen – Rufschädigung, sozialer Rückzug, psychische Belastung – sind real, auch wenn die Aufnahmen gefälscht sind. In rechtlicher Hinsicht steht in der Schweiz in erster Linie das Instrumentarium des Persönlichkeitsschutzes nach den Art. 28 ff. ZGB zur Verfügung. Dieses stammt aber noch aus einer Zeit der analogen Kommunikation. Ausserdem scheitert die Geltendmachung der dort garantierten Rechte meist an technischen Hürden.

Unsichere Passwörter und Mehrfachverwendung sind in dieser Altersgruppe weit verbreitet.⁷ Wer dasselbe Passwort für Netflix, den Schul-Account und das E-Banking verwendet, riskiert, dass ein einziger Datenleck eine Kettenreaktion auslöst. Datenlecks sind keine Ausnahme: Grosse Plattformen werden regelmässig kompromittiert, und die gestohlenen Zugangsdaten landen innerhalb von Stunden im Darknet.⁸

Daten als Währung – viele Jugendliche nutzen Apps und Dienste *gratis*, sie lesen deren Datenschutzbestimmungen nicht und erlauben den Zugriff auf Standortdaten, Nutzungsprofile und persönliche Vorlieben. Diese Daten werden dann an Dritte weitergegeben.⁹ Werbetreibende, aber auch Datenhändler, bezahlen für diese Information – das Produkt ist hier nicht die App, sondern die Daten der Nutzerin oder des Nutzers.

3.2 Von der persönlichen Bedrohung zum gesellschaftlichen Risiko

Die beschriebenen Risiken betreffen einzelne Personen – aber sie sind Ausdruck eines viel grösseren Phänomens. Cyberrisiken sind längst zu einer gesamtgesellschaftlichen Herausforderung geworden,

⁴Schweizerische Kriminalprävention (SKP), *Cyberkriminalität*.

⁵Pro Juventute, *Cybermobbing Unter Jugendlichen in Der Schweiz: Aktuelle Zahlen Und Trends* (Pro Juventute, 2022).

⁶European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*.

⁷Bundesamt für Sozialversicherungen (BSV), *Jugend Und Medien*.

⁸Bundesamt für Cybersicherheit (BACS), *Halbjahresbericht 2024/I*.

⁹Bundesamt für Sozialversicherungen (BSV), *Jugend Und Medien*.

die Wirtschaft, Staat und kritische Infrastrukturen gleichermaßen bedroht.¹⁰

Wo im privaten Bereich ein Phishing-Angriff ein Bankkonto leert, zielt **Ransomware** auf Unternehmen und Behörden: Angreifer verschlüsseln sämtliche Daten eines Systems und fordern Lösegeld für die Freigabe.¹¹ Spitäler, Gemeinden und Industriebetriebe waren in der Schweiz bereits Opfer solcher Angriffe – mit teils gravierenden Folgen für die Betriebsfähigkeit.

Kritische Infrastrukturen wie Stromversorgung, Wasserversorgung, Verkehrsleitsysteme oder Finanzmarktinfrastrukturen sind zunehmend vernetzt und damit angreifbar.¹² Ein erfolgreicher Angriff auf ein Umspannwerk oder ein Klärwerk ist kein Science-Fiction-Szenario mehr, sondern ein ernst zu nehmendes Risiko, dem in den staatlichen Sicherheitsstrategien Rechnung getragen werden muss.

Desinformation und Manipulation sind die digitale Entsprechung zum Social Engineering im privaten Bereich: Koordinierte Kampagnen nutzen soziale Netzwerke, um die öffentliche Meinung zu beeinflussen, Vertrauen in Institutionen zu untergraben oder politische Prozesse zu destabilisieren.¹³ Deepfakes spielen auch hier eine wachsende Rolle – gefälschte Aussagen von Politikerinnen und Politikern können innerhalb von Minuten Millionen von Menschen erreichen.¹⁴

Schliesslich verursacht die **zunehmende Vernetzung** («Internet of Things») neue Risiken: Industrieanlagen, Fahrzeuge, medizinische Geräte – je mehr Geräte online sind, desto grösser die potenzielle Angriffsfläche.¹⁵

3.3 Wer steckt dahinter? Ein Blick auf die Täterseite

Cyberangriffe werden oft mit dem Bild des einsamen Hackers im dunklen Keller assoziiert. Die Realität ist erheblich vielschichtiger. Täterinnen und Täter lassen sich grob in vier Kategorien einteilen, die sich in Motivation, Ressourcen und Professionalität stark unterscheiden – und die je nach Kontext ganz unterschiedliche Risikogruppen ins Visier nehmen.¹⁶

Gelegenheitstäter und Jugendliche stehen am einen Ende des Spektrums. Technische Hilfsmittel – Phishing-Kits, Deepfake-Apps, Hacking-Tutorials – sind online frei verfügbar.¹⁷ Cybermobbing oder das Erstellen und Verbreiten von Deepfakes erfordert heute kein technisches Fachwissen mehr. Ihre primären Opfer sind Gleichaltrige aus dem sozialen Nahbereich: Mitschülerinnen und Mitschüler, Bekannte, Ex-Partnerinnen und Ex-Partner.¹⁸ Oft fehlt es an einem Unrechtsbewusstsein

¹⁰European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*; Center for Security Studies (CSS), ETH Zürich, *Cyber Defence: Challenges for Switzerland* (ETH Zürich, Center for Security Studies, 2024).

¹¹Bundesamt für Cybersicherheit (BACS), *Halbjahresbericht 2024/II: Schwerpunktthema Ransomware* (Bundesamt für Cybersicherheit, 2024).

¹²Center for Security Studies (CSS), ETH Zürich, *Cyber Defence*.

¹³Europol, *Internet Organised Crime Threat Assessment (IOCTA) 2023* (Europol, 2023).

¹⁴European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*.

¹⁵European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*.

¹⁶Europol, *Internet Organised Crime Threat Assessment (IOCTA) 2023*; European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*.

¹⁷Schweizerische Kriminalprävention (SKP), *Cyberkriminalität*.

¹⁸Pro Juventute, *Cybermobbing Unter Jugendlichen in Der Schweiz*.

– was als Streich oder Spass beginnt, kann nach Art. 143, 143bis, 144bis und 179quater StGB strafbar sein und bei den Betroffenen erheblichen und dauerhaften psychischen Schaden anrichten.

Cyberkriminelle handeln mit klarer wirtschaftlicher Motivation. Organisierte Gruppen verkaufen gestohlene Zugangsdaten, betreiben Ransomware-as-a-Service – das heisst, sie vermieten ihre Angriffswerkzeuge an andere Kriminelle – oder ergaunern über Phishing-Kampagnen in industriellem Massstab Millionenbeträge.¹⁹ Jugendliche sind für diese Tätergruppe insbesondere als leichtgläubige Opfer von Phishing interessant: Sie verfügen über Zahlungsmittel, sind weniger misstrauisch als Erwachsene und haben oft unsichere Zugangsdaten.²⁰ Gleichzeitig stehen Unternehmen und Behörden im Fokus von Ransomware-Angriffen – hier ist der finanzielle Ertrag eines erfolgreichen Angriffs um ein Vielfaches höher.²¹

Hacktivisten verfolgen politische oder ideologische Ziele. Sie legen Websites von Organisationen lahm, die sie ablehnen, oder veröffentlichen gestohlene Daten, um Aufmerksamkeit auf ein Anliegen zu lenken. Schulen und Jugendliche sind für diese Gruppe in der Regel kein primäres Ziel; im Fokus stehen staatliche Institutionen, Unternehmen oder Medienorganisationen.²² Die Grenze zwischen politischem Protest und krimineller Handlung ist dabei fließend und juristisch oft umstritten.

Staatliche Akteure – Nachrichtendienste und spezialisierte Militäreinheiten – betreiben Cyberspionage, sabotieren ausländische Infrastrukturen oder beeinflussen durch Desinformationskampagnen politische Prozesse in anderen Ländern.²³ Sie sind die einzige Tätergruppe, die gezielt kritische Infrastrukturen angreift: Stromnetze, Wasserversorgung, Finanzsysteme oder militärische Kommunikation. Solche Operationen sind hochprofessionell, langfristig angelegt und mit erheblichen staatlichen Ressourcen ausgestattet.²⁴ Für Einzelpersonen und Jugendliche sind sie in der Regel keine unmittelbare Bedrohung – wohl aber für die Gesellschaft als Ganzes, auf deren funktionierende Infrastruktur alle angewiesen sind.

Was alle Kategorien verbindet: Der Cyberraum senkt die Einstiegshürde für Angriffe erheblich. Anonymität, globale Reichweite und die Tatsache, dass Angriffe oft grenzüberschreitend stattfinden und damit schwer verfolgbar sind, machen ihn attraktiv – für Jugendliche mit schlechten Absichten ebenso wie für staatlich finanzierte Geheimdienste.²⁵ Cyberrisiken sind damit nicht nur ein technisches Problem, sondern eine **rechtliche, wirtschaftliche und politische Herausforderung** ersten Ranges. Wer die Täter, ihre Methoden und ihre Ziele versteht, versteht einen wesentlichen Teil der Risiken unserer vernetzten Gegenwart.

Literatur und Quellen

«Amazon Linux 2023». In *Amazon Web Services, Inc.* <https://aws.amazon.com/linux/amazon-linux-2023/>, 2023.

¹⁹Europol, *Internet Organised Crime Threat Assessment (IOCTA) 2023*.

²⁰Bundesamt für Sozialversicherungen (BSV), *Jugend Und Medien*.

²¹Bundesamt für Cybersicherheit (BACS), *Halbjahresbericht 2024/II*.

²²Europol, *Internet Organised Crime Threat Assessment (IOCTA) 2023*.

²³Center for Security Studies (CSS), ETH Zürich, *Cyber Defence*.

²⁴Center for Security Studies (CSS), ETH Zürich, *Cyber Defence*; European Union Agency for Cybersecurity (ENISA), *ENISA Threat Landscape 2024*.

²⁵Bundesamt für Cybersicherheit (BACS), *Halbjahresbericht 2024/I*.

- Bundesamt für Cybersicherheit (BACS). *Halbjahresbericht 2024/I: Lage Der Informationssicherheit in Der Schweiz*. Bundesamt für Cybersicherheit, 2024.
- Bundesamt für Cybersicherheit (BACS). *Halbjahresbericht 2024/II: Schwerpunktthema Ransomware*. Bundesamt für Cybersicherheit, 2024.
- Bundesamt für Sozialversicherungen (BSV). *Jugend Und Medien: Chancen Und Risiken Der Digitalen Welt*. Bundesamt für Sozialversicherungen / Nationales Programm zur Förderung von Medienkompetenz, 2023.
- Center for Security Studies (CSS), ETH Zürich. *Cyber Defence: Challenges for Switzerland*. ETH Zürich, Center for Security Studies, 2024.
- Doberkat, Ernst-Erich, und Michael Jung. *Ebene Trigonometrie & Analytische Geometrie: Grundlagen und Anwendungen für Geodäsie, Karographie und verwandte Disziplinen*. Springer Berlin Heidelberg, 2024. Zugriffen 4. Juni 2026. https://doi.org/10.1007/978-3-662-68843-4_1.
- Durandi, Werner, Baoswan Dzung Wong, Markus Kriener, u. a. *Formeln, Tabellen, Begriffe: Mathematik - Physik - Chemie*. 8., aktualisierte Auflage. Herausgegeben von Verein Schweizerischer Mathematik- und Physiklehrkräfte und Werner Durandi. Orell Füssli Verlag, 2022.
- European Union Agency for Cybersecurity (ENISA). *ENISA Threat Landscape 2024*. ENISA, 2024.
- Europol. *Internet Organised Crime Threat Assessment (IOCTA) 2023*. Europol, 2023.
- Fix, Blair. «Why and How I Write Scientific Documents in Plain Text». In *Economics from the Top Down*. 2020.
- Freiermuth, Karin. *Einführung in die Kryptologie: Lehrbuch für Unterricht und Selbststudium*. 2., überarb. Aufl. Einführung in die Kryptologie. Vieweg, 2014.
- Healy, Kieran. *The Plain Person's Guide to Plain Text Social Science*. 2020.
- Iwanowski, Sebastian, und Rainer Lang. *Diskrete Mathematik mit Grundlagen: Lehrbuch für Studierende von MINT-Fächern*. Springer Fachmedien Wiesbaden, 2021. Zugriffen 24. Mai 2026. <https://doi.org/10.1007/978-3-658-32760-6>.
- «Leichter Hypervisor – AWS Nitro System – AWS». In *Amazon Web Services, Inc.* <https://aws.amazon.com/de/ec2/nitro/>, 2026.
- Pro Juventute. *Cybermobbing Unter Jugendlichen in Der Schweiz: Aktuelle Zahlen Und Trends*. Pro Juventute, 2022.
- Schweizerische Kriminalprävention (SKP). *Cyberkriminalität: Prävention Und Schutz*. Schweizerische Kriminalprävention, 2023.

suhuruli. *Einführung in den Azure Linux-Containerhost für AKS*. <https://learn.microsoft.com/de-de/azure/azure-linux/intro-azure-linux>, o. J.

«TIOBE Index». In *TIOBE*. 2026.

4 Cyber-Selbstverteidigung: Was der eigene Rechner verrät

Praktische Anleitung mit nmap und Wireshark – für Windows 11 und Linux

4.1 Worum es geht

In diesem Selbstversuch beobachten Sie Ihren eigenen Rechner aus zwei Perspektiven:

1. **Aktiv**, indem Sie ihn mit nmap von « aussen » scannen und sehen, welche Dienste auf dem Gerät lauschen.
2. **Passiv**, indem Sie mit Wireshark den Netzwerkverkehr mitschneiden und sichtbar machen, was der Rechner und einzelne Programme *ohne Ihre Aufforderung* mit der Welt austauschen.

Ziel ist nicht, Hacker:in zu werden, sondern ein realistisches Bild der eigenen Angriffsfläche und der eigenen Datenspur zu gewinnen. Daraus folgen die konkreten Massnahmen der «Cyber-Selbstverteidigung» – Firewall, Updates, Tracker-Schutz, Verschlüsselung – fast von selbst.

4.1.1 Lernziele

Nach diesem Versuch können Sie

- den Unterschied zwischen aktivem Scannen und passivem Mitschneiden erklären;
- die offenen Ports Ihres eigenen Geräts identifizieren und die zugehörigen Dienste benennen;
- einen Portscan im Netzwerkverkehr wiedererkennen (SYN / SYN-ACK / RST);
- mit Display-Filtern in Wireshark gezielt DNS-, TLS- und Tracker-Verkehr sichtbar machen;
- begründen, warum heutige Schutzmassnahmen weit mehr umfassen als «alle Ports zu».

4.2 Rechtlicher und ethischer Rahmen — bitte zuerst lesen

Portscans und Netzwerkmitschnitte sind in der Schweiz nur auf **eigenen** Systemen und im **eigenen** Netzwerk unproblematisch. Bereits ein Scan auf einen fremden Rechner kann den Tatbestand des unbefugten Eindringens in ein Datenverarbeitungssystem (Art. 143bis StGB) erfüllen.

Regeln für diesen Selbstversuch

- Scans und Mitschnitte ausschliesslich auf dem **eigenen Gerät** (127.0.0.1 / localhost).
- Im Schulnetz nur nach ausdrücklicher Erlaubnis und nur auf das eigene Notebook.
- Mitschnitte können fremde Daten enthalten (z. B. Geräte von Mitschüler:innen im selben WLAN). Mitschnittdateien deshalb **nicht weitergeben** und nach dem Versuch löschen.

4.3 Vorbereitung: Software installieren

Sie brauchen zwei Werkzeuge: **Wireshark** (passiver Netzwerk-Sniffer) und **Nmap** (aktiver Portscanner).

4.3.1 Windows 11

1. Wireshark von <https://www.wireshark.org/download.html> herunterladen (64-bit Windows Installer).
2. Installer ausführen. Während der Installation wird **Npcap** mitinstalliert. Wichtig:
 - Im Npcap-Dialog die Option «**Support loopback traffic ('Npcap Loopback Adapter')**» aktivieren.
 - Falls Wireshark bei Ihnen bereits installiert ist, aber kein «Adapter for loopback traffic capture» erscheint: Npcap separat von <https://npcap.com/> neu installieren und die Loopback-Option setzen.
3. Nmap von <https://nmap.org/download.html> herunterladen («Latest stable release self-installer»). Zenmap (GUI) kann mitinstalliert werden, ist für unseren Zweck aber nicht nötig.
 - Falls Nmap fragt, ob es ein eigenes Npcap installieren soll: das bereits von Wireshark installierte Npcap behalten.
4. Schnelltest in der **PowerShell** (Windows-Taste, dann «PowerShell» eingeben):

```
1 nmap --version
```

sollte die Versionsnummer ausgeben.

4.4 Teil 1 — Welche Dienste lauschen auf meinem Rechner?

Bevor wir scannen, schauen wir mit Bordmitteln, welche Programme aktuell auf eingehende Verbindungen warten.

PowerShell als normaler Benutzer öffnen und dann die folgenden Befehle eingeben:

```

1 Get-NetTCPConnection -State Listen |
2   Select-Object LocalAddress, LocalPort, OwningProcess |
3   Sort-Object LocalPort |
4   Format-Table

```

Den Prozessnamen zu einer PID ermitteln:

```

1 Get-Process -Id <PID>

```

Klassisch funktioniert auch:

```

1 netstat -ano | findstr LISTENING

```

4.4.1 Beobachtungsfragen

- Welche Ports sind bei Ihnen offen? Welche Dienste vermuten Sie dahinter?
- Recherchieren Sie unbekannte Ports (5353, 631, 139, 445, 5355, 5040 ...). Brauchen Sie den jeweiligen Dienst?
- Auf welcher Adresse lauscht der Dienst? 127.0.0.1 (nur lokal), 0.0.0.0 (alle Netzwerkkarten) oder eine konkrete IP?

4.5 Teil 2 — Portscan auf den eigenen Rechner

Wir prüfen jetzt aus «Aussenperspektive», was tatsächlich antwortet.

Auf beiden Betriebssystemen, Terminal bzw. PowerShell:

```

1 nmap -sT -p 1-1024 127.0.0.1

```

- **-sT** macht einen **TCP-Connect-Scan**: Nmap baut für jeden Port eine richtige TCP-Verbindung auf. Vorteil: funktioniert ohne Administrationsrechte.
- **-p 1-1024** beschränkt auf die ersten 1024 Ports (die «well-known ports»). Mit **-p-** werden alle 65 535 gescannt – dauert länger.

Variante mit Versionserkennung (langsamer):

```

1 nmap -sT -sV -p 1-1024 127.0.0.1

```

-sV versucht, beim offenen Port die laufende Software inkl. Version zu erraten.

4.5.1 Beobachtungsfragen

- Stimmt das Ergebnis mit Ihrer Liste aus Teil 1 überein? Wo nicht – warum?
- Was bedeuten `open`, `closed`, `filtered`?
- Was würde sich vermutlich ändern, wenn Sie `localhost` durch die **IP im WLAN** ersetzen würden? (Hinweis: Firewall des Betriebssystems.)

4.6 Teil 3 — Wireshark beobachtet den Scan

Jetzt schauen wir uns denselben Scan aus der Netzwerk-Perspektive an. Wireshark zeichnet auf, *was tatsächlich auf der Leitung passiert*.

4.6.1 Schritt 1: Wireshark starten und Interface wählen

- **Linux:** Im Wireshark-Startfenster die Schnittstelle `lo` (Loopback) doppelklicken.
- **Windows 11:** Den Eintrag «**Adapter for loopback traffic capture**» doppelklicken. Falls dieser nicht erscheint, fehlt die Loopback-Option in Npcap (siehe Installation).

4.6.2 Schritt 2: (Optional) Capture-Filter setzen

Im Startfenster im Feld neben dem Interface den Capture-Filter eingeben:

```
tcp
```

Damit wird ARP-, mDNS- und IPv6-Hintergrundrauschen schon beim Aufzeichnen weggeschnitten.

4.6.3 Schritt 3: Aufzeichnung starten

Auf das blaue Haifischflossen-Symbol oben links klicken. Wireshark zeigt jetzt live alle Pakete.

4.6.4 Schritt 4: Im zweiten Fenster den Scan ausführen

Terminal/PowerShell öffnen und erneut:

```
1 nmap -sT -p 1-1024 127.0.0.1
```

4.6.5 Schritt 5: Aufzeichnung stoppen und analysieren

Auf das rote Quadrat klicken. Sie sehen jetzt mehrere tausend Pakete. Mit **Display-Filtern** machen wir Struktur sichtbar:

Display-Filter	Was Sie sehen
<code>tcp.flags.syn == 1 and</code> <code>tcp.flags.ack == 0</code>	Die SYN-Pakete des Scans: einer pro geprüftem Port
<code>tcp.flags.syn == 1 and</code> <code>tcp.flags.ack == 1</code>	SYN-ACK von offenen Ports
<code>tcp.flags.reset == 1</code>	RST von geschlossenen Ports
<code>tcp.port == 80</code>	Nur Verkehr für einen konkreten Port

Den Filter ins Feld direkt unter der Symbolleiste eingeben und Enter drücken.

4.6.6 Beobachtungsfragen

- Wie viele SYN-Pakete schickt nmap pro Sekunde? Was würde das für einen Scan auf alle 65 535 Ports bedeuten?
- Welche der von Ihnen in Teil 2 als open identifizierten Ports sehen Sie jetzt mit SYN-ACK?
- Was passiert nach dem Drei-Wege-Handshake bei -sT an einem offenen Port? (Tipp: schauen Sie auf die FIN- bzw. RST-Pakete am Ende.)

4.7 Teil 4 — Was tut der Rechner, wenn ich gar nichts tue?

Jetzt drehen wir den Aufbau: keine Aktion am Rechner, aber Wireshark hört auf der **echten Netzwerkkarte**.

4.7.1 Schritt 1: Vorbereitung

- Schliessen Sie **alle** Browserfenster.
- Lassen Sie aber Mail-Client, Cloud-Sync (OneDrive, Nextcloud, ...), Messenger u.ä. laufen, falls vorhanden – die wollen wir ja gerade beobachten.

4.7.2 Schritt 2: Wireshark neu starten

Diesmal Interface:

- **Linux:** wlan0, wlp... oder eth0 (die echte Karte, *nicht* lo).
- **Windows 11:** «Wi-Fi» oder «Ethernet».

Kein Capture-Filter setzen.

4.7.3 Schritt 3: Zwei Minuten warten

Zwei Minuten lang den Rechner in Ruhe lassen. Dann Capture stoppen.

4.7.4 Schritt 4: Mit Display-Filtern erkunden

Display-Filter	Was Sie sehen
<code>dns</code>	Alle DNS-Anfragen: welche Hostnamen werden aufgelöst?
<code>tls.handshake.type == 1</code>	TLS-«Client Hello»: mit welchen Servern wird verschlüsselt gesprochen?
<code>mdns</code>	Geräte im lokalen Netz, die sich gegenseitig ankündigen
<code>dhcp</code>	Adressvergabe im lokalen Netz
<code>!arp && !ipv6.dst == ff02::/16</code>	Alles ausser dem grössten Rauschen

Besonders aufschlussreich: **rechte Maustaste auf eine DNS-Zeile → Follow → UDP Stream**, oder **rechte Maustaste auf ein TLS-Hello → Expand → Server Name Indication**.

4.7.5 Beobachtungsfragen

- Welche Hostnamen erscheinen, obwohl Sie nichts machen? Welche Applikationen verraten sich dadurch?
- Finden Sie Update-Checks, Cloud-Sync, Telemetrie? Notieren Sie drei konkrete Beispiele.
- Welche Hostnamen erkennen Sie *nicht*? Wie könnten Sie sie zuordnen?

4.8 Teil 5 — Eine «harmlose» Website besuchen

Jetzt sehen wir uns an, was der Browser tut, wenn Sie eine einzige Newsseite öffnen.

4.8.1 Schritt 1: Vorbereitung

- Wireshark läuft weiterhin auf der echten Netzwerkkarte.
- Browser noch **geschlossen**.
- Aufzeichnung starten.

4.8.2 Schritt 2: Eine Website öffnen

Browser starten und **eine** Webseite besuchen, zum Beispiel eine Schweizer Nachrichtenseite. Warten, bis die Seite vollständig geladen ist, dann Aufzeichnung stoppen.

4.8.3 Schritt 3: Drittparteien sichtbar machen

Display-Filter:

```
tls.handshake.extensions_server_name
```

Wireshark zeigt jetzt nur noch TLS-Handshakes, bei denen sichtbar ist, mit welchem Hostnamen verhandelt wird. Diese Information (SNI, *Server Name Indication*) wird **vor** der Verschlüsselung übertragen und ist deshalb für uns lesbar – obwohl der Inhalt der Verbindung selbst verschlüsselt ist.

Zusätzlich praktisch: **Statistics** → **Endpoints** → **IPv4** zeigt eine Liste aller kontaktierten Gegenstellen mit Paketzahlen.

4.8.4 Beobachtungsfragen

- Wie viele unterschiedliche Hostnamen wurden für *eine* Seite kontaktiert?
- Welche davon sind die Newsseite selbst, welche sind Drittparteien?
- Welche Drittparteien sind erkennbar Werbe- oder Tracking-Anbieter (doubleclick.net, googletagmanager.com, facebook.net, criteo.com, adnxs.com ...)?
- Was sagt das über das Geschäftsmodell der Seite?

4.8.5 Optionaler Vergleich

Installieren Sie im Browser **uBlock Origin**, leeren Sie den Cache, wiederholen Sie Schritt 2 mit derselben Seite und vergleichen Sie die Anzahl Drittparteien.

4.9 Synthese: Was nehmen wir mit?

1. **Aktive Sicht (Portscan).** Der Rechner hat eine Angriffsfläche aus *lauschenden Diensten*. Schutz: nicht benötigte Dienste deaktivieren, Firewall einschalten, Updates fahren.
2. **Passive Sicht (Mitschnitt im Leerlauf).** Der Rechner spricht ohne mein Zutun mit Dutzenden Servern. Schutz: Telemetrie minimieren, Cloud-Dienste bewusst wählen, evtl. Pi-hole oder NextDNS einsetzen.
3. **Passive Sicht (Webseiten-Aufruf).** Beim Surfen entstehen Datenspuren über ganze Werbe-Ökosysteme hinweg. Schutz: Content-Blocker, datenschutzfreundlicher Browser, Tracking-Schutz, ggf. eigener DNS-Resolver.
4. **Was Wireshark *nicht* zeigt.** Phishing-Mails, Social Engineering, kompromittierte Anmeldedaten, Supply-Chain-Angriffe auf installierte Software. Genau dort liegt heute das grössere Risiko – deshalb ergänzen wir technische Massnahmen um **Verhalten** (Passwortmanager, 2FA / Passkeys, Skepsis bei Mails), **Resilienz** (Backups) und **Hygiene** (Updates, Patch-Management).

4.10 Optional / Weiterführendes

- **DNS over HTTPS aktivieren** (im Browser oder system-weit) und Capture wiederholen: Die dns-Filterzeile bleibt leer, der DNS-Verkehr läuft jetzt in TLS verpackt zu einem konfigurierten Resolver.
- **Eigenes Heimnetz vermessen** (nur dort!) mit `nmap -sn 192.168.1.0/24` zur Host-Erkennung. Wie viele Geräte sind angeschlossen? Erkennen Sie alle?
- **Mit `nmap -O` ein Betriebssystem-Fingerprinting** auf den eigenen Rechner versuchen. Wie genau ist die Vermutung?
- **Display-Filter-Galerie in Wireshark** (Statistics → Protocol Hierarchy) ansehen: zeigt, welche Protokollanteile Ihr Verkehr in welcher Zeit hatte.

4.11 Anhang: Häufige Stolpersteine

Problem	Ursache / Lösung
Wireshark zeigt unter Linux keine Interfaces	User nicht in Gruppe <code>wireshark</code> , oder Sitzung nicht neu gestartet
Unter Windows fehlt «Adapter for loopback traffic capture»	Npcap ohne Loopback-Option installiert; Npcap mit Option neu installieren
nmap zeigt unter Windows weniger Ports als unter Linux	Windows Defender Firewall filtert auch lokal; <code>127.0.0.1</code> ist davon meist nicht betroffen, externe IPs schon
Mitschnitt enthält praktisch nichts	Falsches Interface (z. B. virtuelle Schnittstelle statt WLAN) gewählt
Wireshark wird auf grossen Capture-Dateien extrem langsam	Mit Capture-Filter (nicht Display-Filter!) schon beim Aufzeichnen einschränken

Teil IV

Datenbanken II

5 Notebook-Übung: SQL-Aggregatfunktionen

i Wie man dieses Notebook lokal ausführt

1. Erstelle einen Ordner
2. Erstelle eine virtuelle Python-Umgebung
`python -m venv .venv`
3. Aktiviere die virtuelle Umgebung
`.venv\Scripts\activate`
4. Installiere Jupyter und Pandas in dieser Umgebung
`pip install jupyter pandas`
5. Lade diese Datei in deinen neu erstellten Ordner herunter
6. Starte den lokalen Jupyter-Server
`jupyter lab`

5.1 Analyse einer Verkaufsdatenbank

Kontext: Du analysierst eine vereinfachte Verkaufsdatenbank für einen Online-Shop. Die Datenbank enthält zwei Tabellen: `products` (Produktkatalog) und `orders` (Kundenbestellungen).

Ziel: Anwendung der SQL-Aggregatfunktionen `COUNT`, `SUM`, `AVG`, `MIN` und `MAX` in Kombination mit `GROUP BY` und `HAVING` in einer SQLite-Datenbank direkt in diesem Notebook.

Dauer: ca. 15–20 Minuten

Voraussetzungen: Grundkenntnisse in Python und pandas (DataFrame);
SQL-Grundlagen: `SELECT`, `FROM`, `WHERE`

Bootcamp Um deine SQL-Kenntnisse aufzufrischen, kannst du folgende Übungen machen:

1. [Basics](#)
 2. [SELECT](#)
-

5.2 Datenbank-Schema

Tabelle: products

product_id	product_name	category	price
1	Laptop	Electronics	1200.00
2	Mouse	Electronics	25.00
3	Desk	Furniture	450.00
4	Chair	Furniture	320.00
5	Headset	Electronics	80.00

Tabelle: orders

order_id	product_id	quantity	customer
1	1	2	Anna
2	2	5	Ben
3	3	1	Clara
4	1	1	Daniel
5	4	3	Anna
6	5	2	Ben
7	2	8	Clara
8	3	2	Evi
9	5	1	Anna

5.3 Datenbank im Notebook

Die folgende Zelle stellt im Jupyter Notebook eine abfragbare Datenbank zur Verfügung.

```
1 # --- Setup (nicht verändern) ---
2 import sqlite3
3 import pandas as pd
4
5 # Hilfsfunktion: Führt eine SQL-Abfrage aus und gibt ein DataFrame
  ↳ zurück.
6 def sql(query):
7     cursor.execute(query)
8     cols = [d[0] for d in cursor.description]
9     return pd.DataFrame(cursor.fetchall(), columns=cols)
10
```

```

11 # Erstelle eine SQLite-Datenbank im Arbeitsspeicher (RAM)
12 conn = sqlite3.connect(':memory:')
13 cursor = conn.cursor()
14
15 cursor.executescript("""
16 CREATE TABLE products (
17     product_id    INTEGER PRIMARY KEY,
18     product_name  TEXT,
19     category      TEXT,
20     price         REAL
21 );
22 CREATE TABLE orders (
23     order_id      INTEGER PRIMARY KEY,
24     product_id    INTEGER,
25     quantity      INTEGER,
26     customer      TEXT,
27     FOREIGN KEY (product_id)
28         REFERENCES products(product_id)
29 );
30 INSERT INTO products VALUES
31     (1, 'Laptop',    'Electronics', 1200.00),
32     (2, 'Mouse',     'Electronics',  25.00),
33     (3, 'Desk',      'Furniture',   450.00),
34     (4, 'Chair',     'Furniture',   320.00),
35     (5, 'Headset',   'Electronics',  80.00);
36 INSERT INTO orders VALUES
37     (1, 1, 2, 'Anna'),
38     (2, 2, 5, 'Ben'),
39     (3, 3, 1, 'Clara'),
40     (4, 1, 1, 'Daniel'),
41     (5, 4, 3, 'Anna'),
42     (6, 5, 2, 'Ben'),
43     (7, 2, 8, 'Clara'),
44     (8, 3, 2, 'Evi'),
45     (9, 5, 1, 'Anna');
46 """)
47 conn.commit()
48 print('Datenbank bereit. Viel Erfolg!')

```

5.4 Abfrageübungen

```

1 # --- Übungen ---
2 # Nutze sql(""" ... """) um deine Abfragen auszuführen.

```

```

3 # Ersetze den Platzhalter-Kommentar mit deinem SQL-Code.
4
5 # AUFGABE 1: Wie viele Bestellungen gibt es insgesamt?
6 # Nutze COUNT(*) auf der Tabelle 'orders'.
7 result1 = sql("""
8 -- Deine Abfrage hier
9 """)
10 print('Aufgabe 1:\n', result1)

```

```

1 # AUFGABE 2: Was ist der günstigste und der teuerste
2 # Produktpreis? Nutze MIN und MAX.
3 result2 = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 2:\n', result2)

```

```

1 # AUFGABE 3: Wie viele Bestellungen und welche Gesamtmenge
2 # wurden pro Kategorie aufgegeben?
3 # Verbinde (JOIN) 'orders' und 'products', gruppieren nach 'category'.
4 result3 = sql("""
5 -- Deine Abfrage hier
6 """)
7 print('Aufgabe 3:\n', result3)

```

```

1 # AUFGABE 4: Zeige nur die Kategorien an, in denen die
2 # bestellte Gesamtmenge (quantity) größer als 5 ist.
3 # Nutze HAVING (nicht WHERE!).
4 result4 = sql("""
5 -- Deine Abfrage hier
6 """)
7 print('Aufgabe 4:\n', result4)

```

5.5 Weiterführendes Training

Für weiteres Training zu Aggregatfunktionen siehe [SUM and COUNT](https://sqlzoo.net) auf sqlzoo.net.

5.6 Reflexion

Beantworte die folgenden Fragen in dieser Zelle:

1. Warum erzeugt `WHERE category = 'Electronics'` vor dem `GROUP BY` ein anderes Ergebnis als `HAVING category = 'Electronics'` nach dem `GROUP BY`?

Ihre Antwort:

2. Was ist der Unterschied zwischen `COUNT(*)` und `COUNT(product_id)`? Wann könnte dies einen Unterschied machen?

Ihre Antwort:

3. In welchen realen beruflichen Situationen würdest du Abfragen wie diese verwenden? Nenne zwei konkrete Beispiele.

Ihre Antwort:

6 Notebook-Übung: Tabellen verknüpfen mit SQL JOINS

i Wie man dieses Notebook lokal ausführt

1. Erstelle einen Ordner
2. Erstelle eine virtuelle Python-Umgebung
`python -m venv .venv`
3. Aktiviere die virtuelle Umgebung
`.venv\Scripts\activate`
4. Installiere Jupyter und pandas
`pip install jupyter pandas`
5. Lade diese Datei in deinen Ordner herunter
6. Starte den lokalen Jupyter-Server
`jupyter-lab`

6.0.1 Kontext

In der realen Welt sind Daten fast nie in einer einzigen Tabelle gespeichert. Kundendaten liegen getrennt von Bestelldaten, Klassenlisten getrennt von Noten. Um sinnvolle Analysen zu machen, müssen wir Tabellen **verknüpfen** — das nennt man einen **JOIN**.

6.0.2 Das Wichtigste auf einen Blick

Was ich will	SQL
Nur übereinstimmende Zeilen	<code>SELECT ... FROM A INNER JOIN B ON A.key = B.key</code>
Alle Zeilen der linken Tabelle	<code>SELECT ... FROM A LEFT JOIN B ON A.key = B.key</code>
Fehlende Einträge finden	<code>... LEFT JOIN ... WHERE B.spalte IS NULL</code>

Dauer: ca. 40–50 Minuten

Voraussetzungen: SQL-Grundlagen (SELECT, FROM, WHERE, Aggregatfunktionen), grundlegende Python-Kenntnisse

6.0.3 Setup

Führe die folgende Zelle aus, um die Datenbank vorzubereiten. **Nicht verändern.**

```
1 # --- Setup (nicht verändern) ---
2 import sqlite3
3 import pandas as pd
4
5 # Hilfsfunktion: Führt eine SQL-Abfrage aus und gibt ein DataFrame
6   ↪ zurück.
7 def sql(query):
8     cursor.execute(query)
9     cols = [d[0] for d in cursor.description]
10    return pd.DataFrame(cursor.fetchall(), columns=cols)
11
12 # Erstelle eine SQLite-Datenbank im Arbeitsspeicher (RAM)
13 conn = sqlite3.connect(':memory:')
14 cursor = conn.cursor()
15
16 cursor.executescript("""
17 -- Tabellen für Aufgaben 1-2: Die Prüfungsliste
18 CREATE TABLE klassenliste (
19     schueler_id INTEGER PRIMARY KEY,
20     name        TEXT,
21     klasse      TEXT
22 );
23 CREATE TABLE noten (
24     schueler_id INTEGER,
25     note        REAL,
26     FOREIGN KEY (schueler_id) REFERENCES klassenliste(schueler_id)
27 );
28 INSERT INTO klassenliste VALUES
29     (1, 'Lea',    '2fP'),
30     (2, 'Jan',    '2fP'),
31     (3, 'Mira',   '2fP'),
32     (4, 'Noah',   '2fP'),
33     (5, 'Elena',  '2fP'),
34     (6, 'Luca',   '2fP');
35 INSERT INTO noten VALUES
36     (1, 5.5),
37     (2, 4.0),
38     (5, 5.0);
```



```

39 -- Tabellen für Aufgabe 3: Der Online-Shop (gleiche Daten wie im
    ↳ SQL-Aggregatfunktionen-Notebook!)
40 CREATE TABLE products (
41     product_id    INTEGER PRIMARY KEY,
42     product_name  TEXT,
43     category      TEXT,
44     price         REAL
45 );
46 CREATE TABLE orders (
47     order_id      INTEGER PRIMARY KEY,
48     product_id    INTEGER,
49     quantity      INTEGER,
50     customer      TEXT,
51     FOREIGN KEY (product_id) REFERENCES products(product_id)
52 );
53 INSERT INTO products VALUES
54     (1, 'Laptop', 'Electronics', 1200.00),
55     (2, 'Mouse', 'Electronics', 25.00),
56     (3, 'Desk', 'Furniture', 450.00),
57     (4, 'Chair', 'Furniture', 320.00),
58     (5, 'Headset', 'Electronics', 80.00);
59 INSERT INTO orders VALUES
60     (1, 1, 2, 'Anna'),
61     (2, 2, 5, 'Ben'),
62     (3, 3, 1, 'Clara'),
63     (4, 1, 1, 'Daniel'),
64     (5, 4, 3, 'Anna'),
65     (6, 5, 2, 'Ben'),
66     (7, 2, 8, 'Clara'),
67     (8, 3, 2, 'Evi'),
68     (9, 5, 1, 'Anna');
69
70 -- Tabellen für Aufgabe 4: Sportverein
71 CREATE TABLE mitglieder (
72     mitglied_id   INTEGER PRIMARY KEY,
73     name          TEXT,
74     sportart      TEXT
75 );
76 CREATE TABLE turnier_ergebnisse (
77     mitglied_id   INTEGER,
78     turnier       TEXT,
79     rang          INTEGER,
80     FOREIGN KEY (mitglied_id) REFERENCES mitglieder(mitglied_id)
81 );
82 INSERT INTO mitglieder VALUES
83     (1, 'Anna', 'Fussball'),

```

```

84     (2, 'Ben',      'Tennis'),
85     (3, 'Chiara',  'Fussball'),
86     (4, 'David',   'Schwimmen'),
87     (5, 'Eva',     'Tennis'),
88     (6, 'Finn',    'Fussball');
89 INSERT INTO turnier_ergebnisse VALUES
90     (1, 'Kantonalmeisterschaft', 2),
91     (2, 'Vereinsturnier', 5),
92     (3, 'Kantonalmeisterschaft', 8),
93     (5, 'Vereinsturnier', 3);
94
95 -- Tabellen für Aufgabe 5: Bibliothek
96 CREATE TABLE buecher (
97     buch_id INTEGER PRIMARY KEY,
98     titel   TEXT,
99     autor   TEXT,
100    genre   TEXT
101 );
102 CREATE TABLE ausleihen (
103     ausleihe_id INTEGER PRIMARY KEY,
104     buch_id      INTEGER,
105     schueler     TEXT,
106     datum        TEXT,
107     FOREIGN KEY (buch_id) REFERENCES buecher(buch_id)
108 );
109 INSERT INTO buecher VALUES
110     (1, 'Harry Potter', 'J.K. Rowling', 'Fantasy'),
111     (2, 'Gregs Tagebuch', 'Jeff Kinney', 'Humor'),
112     (3, 'Die drei ???', 'Verschiedene', 'Krimi'),
113     (4, 'Eragon', 'C. Paolini', 'Fantasy'),
114     (5, 'Tintenherz', 'C. Funke', 'Fantasy'),
115     (6, 'Rico, Oskar und die Tieferschatten', 'A. Steinhöfel',
116 ↵  'Krimi');
116 INSERT INTO ausleihen VALUES
117     (1, 1, 'Lea',      '2026-03-10'),
118     (2, 2, 'Jan',      '2026-03-12'),
119     (3, 1, 'Mira',     '2026-03-15'),
120     (4, 3, 'Noah',     '2026-03-18'),
121     (5, 2, 'Elena',    '2026-03-20');
122 """)
123 conn.commit()
124 print('Datenbank bereit. Viel Erfolg!')

```

6.0.4 Aufgabe 1: INNER JOIN — Wer hat die Prüfung geschrieben?

Tabellen:

Tabelle	Spalten
klassenliste	schueler_id, name, klasse
noten	schueler_id, note

Frau Müller will nur die Schüler sehen, die die Prüfung **tatsächlich geschrieben haben**, zusammen mit ihren Namen und Noten.

Schreibe ein SELECT mit INNER JOIN, um klassenliste und noten über die Spalte schueler_id zu verknüpfen. Zeige name und note.

Erwartetes Ergebnis: 3 Zeilen (Lea, Jan, Elena).

```
1 # AUFGABE 1: INNER JOIN von klassenliste und noten.
2 # Nutze sql(""" ... """) um deine Abfrage auszuführen.
3
4 result1 = sql("""
5 -- Deine Abfrage hier
6 """)
7 print('Aufgabe 1:\n', result1)
```

Lösung anzeigen

```
1 SELECT klassenliste.name, noten.note
2 FROM klassenliste
3 INNER JOIN noten
4     ON klassenliste.schueler_id = noten.schueler_id;
```

Ergebnis: 3 Zeilen — nur Lea (5.5), Jan (4.0) und Elena (5.0), weil nur diese in **beiden** Tabellen vorkommen.

6.0.5 Aufgabe 2: LEFT JOIN — Die vollständige Übersicht

Frau Müller braucht **alle 6 Schüler** in ihrer Übersicht — auch die, die nicht an der Prüfung teilgenommen haben. Bei fehlenden Noten soll NULL stehen.

2a) Schreibe ein SELECT mit LEFT JOIN, um alle Schüler mit ihren Noten zu sehen.

2b) Erweitere die Abfrage mit einer WHERE-Klausel, um nur die Schüler zu finden, die **keine Note** haben (IS NULL).

2c) Beantworte: Warum stehen dort NULL-Werte?

```

1 # AUFGABE 2a: LEFT JOIN von klassenliste und noten.
2
3 result2a = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 2a:\n', result2a)

```

Lösung anzeigen (2a)

```

1 SELECT klassenliste.name, noten.note
2 FROM klassenliste
3 LEFT JOIN noten
4     ON klassenliste.schueler_id = noten.schueler_id;

```

Ergebnis: 6 Zeilen. Mira, Noah und Luca haben None (= NULL) in der Notenspalte, weil sie nicht in der noten-Tabelle vorkommen.

```

1 # AUFGABE 2b: Nur Schüler OHNE Note finden (LEFT JOIN + WHERE IS NULL).
2
3 result2b = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 2b:\n', result2b)

```

Lösung anzeigen (2b)

```

1 SELECT klassenliste.name
2 FROM klassenliste
3 LEFT JOIN noten
4     ON klassenliste.schueler_id = noten.schueler_id
5 WHERE noten.note IS NULL;

```

Ausgabe: Mira, Noah, Luca — die drei Schüler, die krank waren und die Prüfung nicht geschrieben haben.

Aufgabe 2c: Warum stehen bei diesen Schülern NULL-Werte?

Deine Antwort:

6.0.6 Aufgabe 3: Online-Shop — JOIN + GROUP BY

Tabellen:

Tabelle	Spalten
products	product_id, product_name, category, price
orders	order_id, product_id, quantity, customer

Du kennst diese Tabellen bereits vom SQL-Aggregatfunktionen-Notebook. Jetzt verknüpfen wir sie mit einem JOIN!

3a) Schreibe einen `INNER JOIN`, um jede Bestellung mit dem zugehörigen Produktnamen und Preis anzuzeigen. Zeige: `product_name`, `quantity`, `price`, `customer`.

3b) Berechne den **Gesamtumsatz pro Kategorie**. Tipp: `Umsatz = price * quantity`. Nutze `SUM()` und `GROUP BY`.

```
1 # AUFGABE 3a: INNER JOIN von orders und products.
2
3 result3a = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 3a:\n', result3a)
```

Lösung anzeigen (3a)

```
1 SELECT products.product_name, orders.quantity, products.price,
   ↪   orders.customer
2 FROM orders
3 INNER JOIN products
4     ON orders.product_id = products.product_id;
```

Ergebnis: 9 Zeilen — jede Bestellung jetzt mit Produktname und Preis.

```
1 # AUFGABE 3b: Gesamtumsatz pro Kategorie (JOIN + GROUP BY).
2
3 result3b = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 3b:\n', result3b)
```

Lösung anzeigen (3b)

```

1 SELECT products.category,
2       SUM(orders.quantity * products.price) AS umsatz
3 FROM orders
4 INNER JOIN products
5   ON orders.product_id = products.product_id
6 GROUP BY products.category;

```

Ergebnis: Electronics = 4165.00, Furniture = 2310.00.

6.0.7 Aufgabe 4: Sportverein — Wer hat (nicht) am Turnier teilgenommen?

Tabellen:

Tabelle	Spalten
mitglieder	mitglied_id, name, sportart
turnier_ergebnisse	mitglied_id, turnier, rang

Der Sportverein will eine Übersicht über alle Mitglieder und ihre Turnierergebnisse. Nicht alle Mitglieder haben an einem Turnier teilgenommen!

4a) Zeige alle Mitglieder mit ihren Turnierergebnissen. Auch Mitglieder ohne Turnierergebnis sollen erscheinen. Welchen JOIN-Typ brauchst du?

4b) Finde heraus: Welche Mitglieder haben noch **nicht** an einem Turnier teilgenommen?

```

1 # AUFGABE 4a: Alle Mitglieder mit Turnierergebnissen (auch ohne!).
2
3 result4a = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 4a:\n', result4a)

```

Lösung anzeigen (4a)

```

1 SELECT mitglieder.name, mitglieder.sportart,
2       turnier_ergebnisse.turnier, turnier_ergebnisse.rang
3 FROM mitglieder
4 LEFT JOIN turnier_ergebnisse
5   ON mitglieder.mitglied_id = turnier_ergebnisse.mitglied_id;

```

Ergebnis: 6 Zeilen. David und Finn haben NULL bei Turnier und Rang.

```

1 # AUFGABE 4b: Mitglieder, die NIE an einem Turnier teilgenommen haben.
2
3 result4b = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 4b:\n', result4b)

```

Lösung anzeigen (4b)

```

1 SELECT mitglieder.name, mitglieder.sportart
2 FROM mitglieder
3 LEFT JOIN turnier_ergebnisse
4     ON mitglieder.mitglied_id = turnier_ergebnisse.mitglied_id
5 WHERE turnier_ergebnisse.turnier IS NULL;

```

Ergebnis: David (Schwimmen) und Finn (Fussball) — sie haben kein Turnierergebnis.

6.0.8 Aufgabe 5: Bibliothek — Welche Bücher wurden nie ausgeliehen?

Tabellen:

Tabelle	Spalten
buecher	buch_id, titel, autor, genre
ausleihen	ausleihe_id, buch_id, schueler, datum

Die Schulbibliothek will herausfinden, welche Bücher noch nie ausgeliehen wurden, um sie besser zu bewerben.

5a) Zeige alle Bücher mit ihren Ausleihen an (auch Bücher ohne Ausleihe).

5b) Finde die Bücher, die **noch nie** ausgeliehen wurden.

5c) Zähle, wie oft jedes Buch ausgeliehen wurde. Sortiere absteigend nach der Ausleihzahl. Tipp: Nutze COUNT(), GROUP BY und ORDER BY.

```

1 # AUFGABE 5a: Alle Bücher mit ihren Ausleihen (auch ohne!).
2
3 result5a = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 5a:\n', result5a)

```

Lösung anzeigen (5a)

```
1 SELECT buecher.titel, buecher.autor,
2        ausleihen.schueler, ausleihen.datum
3 FROM buecher
4 LEFT JOIN ausleihen
5     ON buecher.buch_id = ausleihen.buch_id;
```

Ergebnis: 8 Zeilen. Eragon, Tintenherz und Rico haben NULL bei Schüler und Datum.

```
1 # AUFGABE 5b: Bücher, die noch NIE ausgeliehen wurden.
2
3 result5b = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 5b:\n', result5b)
```

Lösung anzeigen (5b)

```
1 SELECT buecher.titel, buecher.autor
2 FROM buecher
3 LEFT JOIN ausleihen
4     ON buecher.buch_id = ausleihen.buch_id
5 WHERE ausleihen.ausleihe_id IS NULL;
```

Ergebnis: Eragon, Tintenherz und Rico, Oskar und die Tieferschatten — diese drei wurden noch nie ausgeliehen.

```
1 # AUFGABE 5c: Wie oft wurde jedes Buch ausgeliehen? (JOIN + GROUP BY +
   ↪ ORDER BY)
2
3 result5c = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Aufgabe 5c:\n', result5c)
```

Lösung anzeigen (5c)

```
1 SELECT buecher.titel,
2        COUNT(ausleihen.ausleihe_id) AS anzahl_ausleihen
3 FROM buecher
4 LEFT JOIN ausleihen
5     ON buecher.buch_id = ausleihen.buch_id
6 GROUP BY buecher.titel
7 ORDER BY anzahl_ausleihen DESC;
```


Ergebnis: Harry Potter und Gregs Tagebuch je 2×, Die drei ??? 1×, die restlichen drei 0×.

6.0.9 Bonus: JOIN über drei Tabellen

Für diese Aufgabe erstellen wir eine zusätzliche Tabelle. Führe zuerst die nächste Zelle aus.

```
1 # Bonus-Setup (nicht verändern)
2 cursor.executescript("""
3 CREATE TABLE hausaufgaben (
4     schueler_id INTEGER,
5     punkte      INTEGER,
6     FOREIGN KEY (schueler_id) REFERENCES klassenliste(schueler_id)
7 );
8 INSERT INTO hausaufgaben VALUES
9     (1, 18),
10    (2, 12),
11    (3, 20),
12    (4, 15),
13    (5, 17),
14    (6, 10);
15 """)
16 conn.commit()
17 print('Bonus-Tabelle erstellt!')
```

Tabellen:

Tabelle	Spalten
klassenliste	schueler_id, name, klasse
noten	schueler_id, note
hausaufgaben	schueler_id, punkte

Frau Müller will jetzt eine **Gesamtübersicht**: Name, Prüfungsnote UND Hausaufgabenpunkte — für alle 6 Schüler.

Tipp: Du kannst zwei JOINS hintereinander schreiben:

```
1 SELECT ...
2 FROM tabelle_a
3 LEFT JOIN tabelle_b ON ...
4 LEFT JOIN tabelle_c ON ...;
```

```

1 # BONUS: JOIN über drei Tabellen.
2
3 bonus = sql("""
4 -- Deine Abfrage hier
5 """)
6 print('Bonus:\n', bonus)

```

Lösung anzeigen

```

1 SELECT klassenliste.name, noten.note, hausaufgaben.punkte
2 FROM klassenliste
3 LEFT JOIN noten
4     ON klassenliste.schueler_id = noten.schueler_id
5 LEFT JOIN hausaufgaben
6     ON klassenliste.schueler_id = hausaufgaben.schueler_id;

```

Ergebnis: 6 Zeilen. Alle Schüler haben Hausaufgabenpunkte, aber nur 3 haben Noten. Bei den Kranken steht NULL in der Notenspalte.

6.1 Reflexion

Beantworte die folgenden Fragen in dieser Zelle:

1. Erkläre in eigenen Worten: Was ist der Unterschied zwischen einem INNER JOIN und einem LEFT JOIN?

Deine Antwort:

2. Nenne eine Situation aus deinem Alltag, in der du zwei Listen zusammenführen müsstest. Welchen JOIN-Typ würdest du verwenden?

Deine Antwort:

3. In Aufgabe 3 hast du einen JOIN mit GROUP BY kombiniert. Warum brauchst du zuerst den JOIN, bevor du gruppieren kannst?

Deine Antwort:

Teil V

Nachhaltige IT

7 Kernbotschaft:

Hinweis

Der ökologische Fußabdruck der IT reicht weit über den Energieverbrauch und Elektroschrott hinaus. Er beinhaltet verborgene soziale und ökologische Kosten, wie etwa gewaltsame Konflikte um Mineralien wie Coltan, die für unsere digitalen Geräte unerlässlich sind. Das Verständnis dieser indirekten Kosten ist entscheidend, um Technologie nachhaltiger zu gestalten.

7.1 Einführung: Was ist der ökologische Fußabdruck der IT?

Der ökologische Fußabdruck der IT umfasst den gesamten Lebenszyklus von Geräten und Dienstleistungen – von der Rohstoffgewinnung und Herstellung bis hin zur Nutzung und Entsorgung. Dieser Fußabdruck wird typischerweise in drei Hauptbereiche unterteilt: die Umweltkosten der Hardware, die Kosten des Computing und die indirekten Kosten, welche die oft übersehenen Folgen von Ressourcenkonflikten (wie etwa rund um Coltan) beinhalten.

Umweltkosten der Hardware

IT-Hardware – Computer, Smartphones, Server – hat erhebliche Auswirkungen auf die Umwelt, primär über drei Kanäle: **Transport**, **Bergbau** und **Konstruktion**.

7.2 Transport

Die globalisierte Natur der IT-Fertigung bedeutet, dass Rohstoffe aus der ganzen Welt bezogen, in Ländern wie China montiert und an Verbraucher überall versandt werden. Diese komplexe Lieferkette führt zu erheblichen Treibhausgasemissionen durch internationale Schifffahrt und Logistik.

7.3 Bergbau

Der Abbau essenzieller Mineralien wie Kupfer, Lithium und insbesondere Coltan (Columbit-Tantalit) ist einer der umweltschädlichsten Aspekte der IT-Hardwareproduktion. Coltan ist entscheidend für die Herstellung von Tantalkondensatoren, die in fast allen Smartphones und Laptops zu finden sind. Bergbaubetriebe führen oft zu Entwaldung, Wasserverschmutzung und der Zerstörung von

Lebensräumen für gefährdete Arten. In der Demokratischen Republik Kongo (DRK) hat der Coltan-Abbau beispielsweise seit dem Jahr 2000 zum Verlust von 8,6 % der Baumbedeckung des Landes beigetragen und die Populationen von Gorillas und anderen Wildtieren dezimiert.

7.4 Konstruktion

Der Herstellungsprozess für IT-Geräte ist energieintensiv und stützt sich auf gefährliche Chemikalien. Fabriken nutzen oft fossile Brennstoffe, was zur globalen Erwärmung beiträgt. Die kurze Lebensdauer vieler Geräte führt zu häufigem Ersatz, was die Nachfrage nach neuen Ressourcen erhöht und riesige Mengen an Elektroschrott (E-Waste) erzeugt. Weltweit werden nur etwa 22 % des Elektroschrotts ordnungsgemäß recycelt, während der Rest auf Mülldeponien landet oder unter unsicheren Bedingungen verarbeitet wird.

7.5 Umweltkosten des Computing

Über die Hardware hinaus hat der Betrieb von IT-Systemen – insbesondere Rechenzentren und Netzwerke – erhebliche Auswirkungen auf die Umwelt.

7.5.1 Energieverbrauch

Rechenzentren, die digitale Informationen speichern und verarbeiten, gehören weltweit zu den größten Stromverbrauchern. Im Jahr 2024 machten Rechenzentren etwa 1,5 % des weltweiten Stromverbrauchs aus; eine Zahl, die mit dem Wachstum von künstlicher Intelligenz und Cloud-Computing (wie dem Online-Betrieb von Jupyter Notebooks) voraussichtlich steigen wird. Der IT-Sektor insgesamt ist für 3–4 % der weltweiten Treibhausgasemissionen verantwortlich.

Power Usage Effectiveness (PUE)

Diese Kennzahl misst, wie effizient ein Rechenzentrum Energie nutzt. Niedrigere PUE-Werte weisen auf einen effizienteren Betrieb hin.

7.5.2 Grüne Rechenzentren

Um ihre Umweltauswirkungen zu reduzieren, investieren viele Organisationen in grüne Rechenzentren. Diese Einrichtungen nutzen erneuerbare Energiequellen, fortschrittliche Kühlsysteme und Abwärmenutzung, um den Energieverbrauch und die Emissionen zu minimieren.

Schweizer Beispiel

Das [Green Data Center Zürich](#) speist Abwärme in über 1.000 Haushalte ein und zeigt so, wie digitale Infrastruktur lokale Energiebedarfe unterstützen und gleichzeitig Verschwendung

reduzieren kann.

7.6 Nachhaltige Software

Software spielt ebenfalls eine Rolle beim Energieverbrauch. Effizienter Code und Algorithmen können die benötigte Rechenleistung verringern und so den Energieverbrauch und die Emissionen senken. Nachhaltige Softwareentwicklung konzentriert sich auf die Optimierung von Anwendungen, um weniger Ressourcen zu verbrauchen und die Lebensdauer der Hardware zu verlängern.

Indirekte Umweltkosten: Der Fall der Coltan-Konflikte

Die indirekten Kosten der IT sind oft weniger sichtbar, können aber schwerwiegender sein als direkte Umweltauswirkungen. Eines der bedeutendsten Beispiele ist der Konflikt um Coltan in der DRK.

7.7 Coltan: Ein kritisches Mineral mit hohem Preis

Coltan ist essenziell für die Produktion von Tantalkondensatoren, die in fast jedem modernen elektronischen Gerät verwendet werden. Die DRK hält bis zu 70 % der weltweiten Coltan-Reserven, was sie zu einem Schlüsselakteur in der globalen Lieferkette macht. Die Gewinnung und der Handel mit Coltan haben jedoch gewaltsame Konflikte, Menschenrechtsverletzungen und Umweltzerstörung geschürt.

7.7.1 Konflikte und Menschenrechte im Bergbausektor der DRK

Gewalt und Vertreibung: Die Sicherheitslage im Osten der DRK bleibt kritisch, da bewaffnete Gruppen, insbesondere die M23-Rebellen, die Kontrolle über strategische Coltan-Bergbauzentren wie Rubaya gefestigt haben, um ihre militärischen Operationen zu finanzieren. Dieser Konflikt hat eine humanitäre Katastrophe ausgelöst; bis Anfang 2026 erreichte die Zahl der Binnenvertriebenen erschütternde 7,3 Millionen Menschen, während Schätzungen von Menschenrechtsbeobachtern darauf hindeuten, dass jährlich weiterhin Tausende Zivilisten in Verbindung mit Gewalt und Territorialstreitigkeiten getötet werden.

Kinderarbeit und Ausbeutung: Der handwerkliche Bergbausektor ist weiterhin stark von Ausbeutung geprägt; schätzungsweise 40.000 Kinder arbeiten unter gefährlichen Bedingungen in den Minen der DRK für Hungerlöhne. Über Kinderarbeit hinaus haben internationale Beobachter wie Amnesty International systemische Menschenrechtsverletzungen dokumentiert, darunter Zwangsräumungen lokaler Gemeinschaften und weit verbreitete sexuelle Gewalt gegen Frauen und Kinder, was die verheerenden menschlichen Kosten der weltweiten Nachfrage nach Kobalt und Coltan unterstreicht.

7.7.2 Wirtschaftliche und ökologische Auswirkungen

Ressourcenfluch: Obwohl die DRK über enorme mineralische Reichtümer verfügt, bleibt der „Ressourcenfluch“ eine bittere Realität: Über 70 % der Bevölkerung leben weiterhin von weniger als 2,15 \$ pro Tag. Während die weltweite Nachfrage nach Elektronik hohe Preise für Coltan treibt, verdienen die meisten handwerklichen Bergleute nur wenige Dollar pro Tag, wobei der Großteil der Gewinne von bewaffneten Gruppen und korrupten Beamten abgeschöpft wird, um laufende Konflikte aufrechtzuerhalten.

Umweltzerstörung: Die Ausweitung des unregulierten Bergbaus hat zu katastrophalen Umweltfolgen geführt. Laut UNEP-Berichten hat der Bergbau großflächige Entwaldung im Kongobecken vorangetrieben, lokale Wasserquellen mit Schwermetallen kontaminiert und kritische Lebensräume für Wildtiere zerstört. Diese ökologischen Schäden verursachen langfristige indirekte Kosten, die die unmittelbaren wirtschaftlichen Gewinne aus der Mineralgewinnung bei weitem überwiegen.

7.7.3 Herausforderungen der globalen Lieferkette

- **Schmuggel und Betrug:** Coltan aus Konfliktgebieten wird oft in Nachbarländer geschmuggelt und mit legal gewonnenen Mineralien gemischt, was es für Hersteller schwierig macht, die Herkunft zurückzuverfolgen und eine Mitschuld an Menschenrechtsverletzungen zu vermeiden.
- **Zertifizierung und Regulierung:** Internationale Bemühungen wie die OECD-Leitsätze für die Erfüllung der Sorgfaltspflicht, der Dodd-Frank Act in den USA und die EU-Konfliktmineralienverordnung zielen darauf ab, die Transparenz der Lieferkette zu verbessern. Die Durchsetzung ist jedoch schwach, und Zertifizierungssysteme wurden durch Betrug und mangelnde Aufsicht untergraben.

i Wichtigste Erkenntnis

Die wahren Kosten unserer digitalen Geräte beinhalten nicht nur die Umweltzerstörung, sondern auch die Aufrechterhaltung von Konflikten und menschlichem Leid in Bergbauregionen.

Tabelle: Die wichtigsten ökologischen und sozialen Auswirkungen der IT

Bereich	Hauptauswirkungen	Lösungen/Trends
Hardware	Bergbau, Transport, Elektroschrott	Kreislaufwirtschaft, achthaltige Beschaffung
Computing	Hoher Energieverbrauch, Emissionen	Grüne Rechenzentren, effiziente Software
Indirekt	Konflikte, Menschenrechtsverletzungen, Umweltzerstörung	Politik, verantwortungsvolle Beschaffung, Verbraucherbewusstsein

Weiterführende Literatur:

- Amnesty International: “This is what we die for”
- OECD Due Diligence Guidance
- UN Reports on DRC

8 Green IT – Datenvisualisierungsprojekt

Im Unterricht wurde der ökologische Fussabdruck der IT besprochen. Zwei zentrale Erkenntnisse:

- Rechenzentren sind für ca. 1.5 % des weltweiten Stromverbrauchs verantwortlich.
- Die Energieeffizienz wird als Power Usage Effectiveness (PUE) gemessen.

$$(PUE = \frac{\text{Gesamtstromverbrauch RZ}}{\text{Stromverbrauch IT}})$$

8.1 Lernziele

Am Ende dieses Projekts können Sie: - PUE-Kennzahlen berechnen und interpretieren - Professionelle Datenvisualisierungen mit matplotlib erstellen - Trends bei der weltweiten Elektroschrott-Erzeugung und -Wiederverwertung analysieren - Datenanalyse mit realen Nachhaltigkeitsherausforderungen verknüpfen

In diesem Notebook berechnen Sie die PUE-Werte verschiedener Unternehmen und visualisieren (und interpretieren) Elektroschrott-Daten.

8.2 Aufgabe 1: Power Usage Effectiveness (PUE)

```
1 # --- Setup (nicht verändern) ---
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # --- Daten: PUE-Werte grosser Cloud-Anbieter ---
6 # Quellen: siehe Referenztabelle unten
7 data_centers = {
8     "Name": [
9         "Google (weltweit, 2024)",
10        "AWS (weltweit, 2024)",
11        "Microsoft (weltweit, 2022)",
12        "Meta (Luleå, 2024)",
13        "Branchendurchschnitt (2024)"
14    ],
15    "PUE": [
16        1.09,      # Google Environmental Report 2025
```

```

17         1.15,    # AWS Sustainability Report 2024
18         1.18,    # Microsoft Azure Blog (2022)
19         1.07,    # Meta/DCPulse (Luleå, Schweden)
20         1.56     # Uptime Institute Survey 2024
21     ]
22 }
23 df_dc = pd.DataFrame(data_centers)

```

8.2.1 Gute Visualisierungspraxis

Bevor Sie mit dem Programmieren beginnen, beachten Sie: 1. **Klare Beschriftungen**: Achsen immer beschriften und einen Titel hinzufügen. 2. **Farbbedeutung**: Farben zur Informationsvermittlung nutzen (grün = gut, rot = schlecht). 3. **Referenzlinien**: Linien für Richtwerte einzeichnen (z. B. PUE = 1.0). 4. **Legende**: Erklären, was Farben oder Symbole bedeuten.

```

1
2 # TODO 1: Erstellen Sie ein horizontales Balkendiagramm (barh) der
   ↪ PUE-Werte.
3 #
4 # HINWEISE:
5 # - Verwenden Sie df_dc["Name"] für die y-Achsen-Beschriftungen
6 # - Verwenden Sie df_dc["PUE"] für die x-Achsen-Werte
7 # - Für bedingte Einfärbung der Balken erstellen Sie eine Liste:
8 #   colors = ['green' if pue < 1.2 else 'orange' if pue < 1.4
9 #             else 'red' for pue in df_dc["PUE"]]
10 # - Fügen Sie eine vertikale Linie hinzu: plt.axvline(x=1.0,
   ↪ color='black',
11 #   linestyle='--', label='Theoretisches Optimum')
12 # - Vergessen Sie nicht: plt.xlabel(), plt.ylabel(), plt.title(),
   ↪ plt.legend()

```

8.3 Aufgabe 2: Elektroschrott-Analyse

```

1 # --- Daten: Weltweiter Elektroschrott ---
2 # Quelle: Global E-Waste Monitor 2024
3 # (UNITAR/ITU), inkl. Prognosen
4 data_ewaste = {
5     "Jahr": [
6         2010, 2014, 2019, 2022, 2030
7     ],
8     "Elektroschrott_Mio_Tonnen": [

```

```

9         34.0, 41.8, 53.6, 62.0, 82.0
10     ],
11     "Recyclingquote_Prozent": [
12         23.5, 22.5, 17.4, 22.3, 20.0
13     ]
14 }
15 df_ew = pd.DataFrame(data_ewaste)
16
17 # TODO 2: Erstellen Sie ein Liniendiagramm mit zwei y-Achsen:
18 # Elektroschrott (Mio. Tonnen) auf der linken Achse und
19 # Recyclingquote (%) auf der rechten Achse, über die Zeit.
20 #
21 # HINWEISE:
22 # - Verwenden Sie plt.plot() für beide Linien
23 # - Für die zweite y-Achse: ax2 = ax1.twinx()
24 # - Markieren Sie 2030 als Prognose mit einer gestrichelten
25 #   Linie oder einem anderen Marker (z. B. linestyle='--'
26 #   für Jahre >= 2030, oder plt.axvline(x=2030,
27 #   linestyle=':', label='Prognose'))
28 # - Beschriften Sie beide y-Achsen: ax1.set_ylabel("..."),
29 #   ax2.set_ylabel("...")
30 # - Fügen Sie eine Legende hinzu: ax1.legend(loc='upper left'),
31 #   ax2.legend(loc='upper right')
32
33
34 # TODO 3:
35 # (a) Berechnen Sie den nicht recycelten Elektroschrott 2022:
36 #     nicht_recycelt_tonnen = gesamt_2022 * (1 - recyclingquote / 100)
37 #
38 # (b) Berechnen Sie den Wert der nicht recycelten Materialien:
39 #     Wenn der Gesamtwert = 62 Mrd. USD und die Recyclingquote = 22.3
40 #     ↪ %,
41 #     dann nicht_recycelter_wert = 62 * (1 - 0.223) Mrd. USD
42 #
43 # Geben Sie beide Ergebnisse aus.

```

8.4 Quellenverzeichnis

Organisation	Bericht/Quelle	Jahr
Google	Environmental Report 2025	2025
AWS	Sustainability Report 2024	2024
Microsoft	Azure Blog (PUE-Kennzahlen)	2022
Meta	DCPulse (Rechenzentrum Luleå)	2024

Organisation	Bericht/Quelle	Jahr
Uptime Institute	Global Data Center Survey	2024
UNITAR/ITU	Global E-Waste Monitor 2024	2024

Links: - Google: <https://sustainability.google/reports/> - AWS: <https://sustainability.aboutamazon.com/> - Microsoft: <https://azure.microsoft.com/en-us/blog/> - Meta: <https://sustainability.fb.com/>
- Uptime Institute: <https://uptimeinstitute.com/> - UNITAR: <https://ewastemonitor.info/>

9 Open-Source-Software

Für die meisten Computernutzer gibt es zwei Alternativen hinsichtlich ihrer Arbeitsumgebung: Entweder man arbeitet an einem PC und nutzt Windows, oder man arbeitet an einem Apple-Gerät und nutzt macOS oder iOS. Beiden Varianten ist gemeinsam, dass man mit proprietärer Software arbeitet. Im Kontext von Software bedeutet proprietär, dass es sich um herstellerspezifische Software handelt, deren Quellcode geheim ist.

Neben diesen bekannten Betriebssystemen und den darauf aufgebauten Software-Ökosystemen existiert ein Paralleluniversum aus Open-Source-Software. Während Open-Source-Betriebssysteme auf Desktop-Computern ein Nischendasein fristen, stellen sie auf Servern die Mehrheit dar, was von der breiten Öffentlichkeit weitgehend unbemerkt bleibt. Auch im Bildungsbereich erfreut sich Open-Source-Software einer gewissen Beliebtheit.

Was ist Open-Source-Software

Open-Source-Software (kurz OSS) umfasst zwei Aspekte:

1. Der Quellcode der Software ist öffentlich. Dies erlaubt es, die Funktionsweise der Software zu untersuchen.
2. Open-Source-Software kann frei verwendet, weiterverbreitet **und** verändert werden.

Dies ist von Freeware zu unterscheiden. Freeware ist Software, die genutzt werden darf, ohne dafür bezahlen zu müssen. Der Quellcode ist jedoch nicht zugänglich und darf (was aufgrund des verborgenen Quellcodes ohnehin kaum möglich wäre) nicht verändert werden.

9.1 Nutzung von Open-Source-Software

Die bekannteste Nutzung von Open-Source-Software ist der Einsatz von Linux als Server-Betriebssystem. AWS, das Cloud-Computing-Produkt von Amazon, bietet Kunden nicht nur virtuelle Linux-Server an,¹ sondern läuft intern auf einem spezialisierten Linux-System.² Microsofts Gegenstück Azure läuft intern auf einem Windows-Server-System; die von Kunden am häufigsten gekauften Lösungen sind jedoch ebenfalls Linux-Server.³

¹«Amazon Linux 2023», in *Amazon Web Services, Inc.*, <https://aws.amazon.com/linux/amazon-linux-2023/>, 2023.

²«Leichter Hypervisor – AWS Nitro System – AWS», in *Amazon Web Services, Inc.*, <https://aws.amazon.com/de/ec2/nitro/>, 2026.

³suhuruli, *Einführung in den Azure Linux-Containerhost für AKS*, <https://learn.microsoft.com/de-de/azure/azure-linux/intro-azure-linux>, o. J.

Supercomputer wie der Alps-Supercomputer (derzeit – Februar 2026 – Platz 8 der Weltrangliste der leistungstärksten Computer) laufen ebenfalls mit Linux als Betriebssystem.

Android ist gleichermaßen eine Variante von Linux und basiert daher im Kern auf Open-Source-Software.

Besonders verbreitet ist Open-Source-Software jedoch in den unzähligen Chips von Smart-Geräten, die die Basis für das Internet der Dinge (Internet of Things) bilden.

9.2 Beispiele für Open-Source-Software

Bekannte Beispiele für etablierte Open-Source-Software sind der Firefox-Browser und die Programmiersprache Python. Diese beiden Beispiele zeigen, dass Open-Source-Software in vielen Bereichen ebenso leistungsfähig ist wie ihre proprietären Gegenstücke. Mit einem Marktanteil von ca. 3 % im Jahr 2025 ist Firefox zwar nur noch ein Schatten seiner selbst, dennoch bleibt Firefox ein bekannter und etablierter Browser. Im Gegensatz dazu ist Python laut dem Tiobe-Ranking⁴ mit einem Anteil von gut 20 % der Suchanfragen die mit Abstand bedeutendste Programmiersprache⁵.

9.3 Open-Source-Lizenzen

Man muss zwischen verschiedenen Open-Source-Lizenzen unterscheiden. Das strengste Prinzip wird als Copyleft bezeichnet. Lizenzen, die diesem Prinzip folgen, verlangen, dass alle Produkte, die auf einer so lizenzierten Software basieren, ebenfalls mit einem Copyleft versehen werden müssen. Dies steht im Gegensatz zum üblichen Verständnis von Copyright, mit dem Urheber in der Regel die Nutzung ihres geistigen Eigentums einschränken wollen. Ein Beispiel für diese Lizenz ist die GNU General Public License (GPL). Es gibt jedoch auch Open-Source-Lizenzen, die eine weitgehend freie Nutzung, Modifikation und Weiterverbreitung erlauben, insbesondere in proprietären Anwendungen. Ein Beispiel hierfür ist die MIT-Lizenz (benannt nach dem Massachusetts Institute of Technology, MIT).

9.4 Geschäftsmodelle auf Basis von Open-Source-Software

Entgegen weit verbreiteter Annahmen ist es möglich, mit Software Geld zu verdienen, die den Nutzern kostenlos zur Verfügung gestellt wird. Viele der entsprechenden Geschäftsmodelle basieren darauf, dass Unternehmen Beratung für den Einsatz ihrer IT-Infrastruktur benötigen – unabhängig davon, ob es sich um proprietäre Software oder Open-Source-Software handelt. Software muss fast immer an die spezifischen Gegebenheiten des jeweiligen Unternehmens angepasst werden. Durch diese Anpassungsleistungen kann Geld verdient werden.

⁴«TIOBE Index», in *TIOBE*, 2026.

⁵Dooley, John F. *History of Cryptography and Cryptanalysis: Codes, Ciphers, and Their Algorithms. History of Computing*. Cham: Springer International Publishing, 2024. <https://doi.org/10.1007/978-3-031-67485-3>. S. 76.

Ein weiteres Geschäftsmodell – wenn auch nicht spezifisch für Open-Source-Software – ist das sogenannte Freemium-Modell. Dabei wird ein Teil der Leistung kostenlos angeboten, während der volle Funktionsumfang kostenpflichtig ist. Zotero stellt ein Beispiel für dieses Modell dar. Zotero ist grundsätzlich kostenlos nutzbar. Möchte man jedoch mehr als ein bestimmtes Maximum an Daten synchronisieren, muss für den entsprechenden Speicherplatz eine Gebühr entrichtet werden.

Andere Modelle unterscheiden zwischen privater und kommerzieller Nutzung. Die private Nutzung ist dann kostenlos, während für die kommerzielle Nutzung eine Lizenzgebühr gezahlt werden muss.

9.5 Ausblick

Eine vertiefte Auseinandersetzung mit Open-Source-Software ergibt sich aus der Aufgabe im folgenden Abschnitt.

10 Redaktionsübung

10.1 Orientierung

Es geht bei dieser Aufgabe darum, dass Sie lernen, einen Text mit integrierten statistischen Auswertungen und Grafiken zu erstellen. Dabei sollen ausschliesslich Open Source Werkzeuge verwendet werden. Darüber hinaus sollen Sie sich mit den Vor- und Nachteilen des Einsatzes von Open Source Software auseinandersetzen.

10.2 Absicht

Ich will,

- dass Sie mit erster Priorität die Möglichkeiten von Markdown und Jupyter Notebooks nutzen, um Ihre Analyse durchzuführen;
- dass Sie mit zweiter Priorität eine nachvollziehbare Datenvisualisierung innerhalb Ihres Textes erstellen und
- dass Sie mit dritter Priorität Übersichtsgrafiken und schematische Darstellungen mit Werkzeugen aus der „Diagrams as Code“-Familie (Mermaid, PlantUML) erstellen können.

10.3 Aufgabe

Schreiben Sie eine Analyse der Chancen und Risiken des Einsatzes von Open-Source-Software sowohl im privaten als auch im unternehmerischen Umfeld.

10.4 Spezifische Anforderungen

10.4.1 Inhalt

- Der Begriff *Open-Source-Software* muss definiert werden.
- Die Analyse sollte auch quantitative Aspekte der Nutzung von Open-Source-Software behandeln. Nutzen Sie hierfür öffentlich zugängliche Quellen.
- Die Analyse muss auf die Kosten der Nutzung von Open-Source-Software eingehen.
- Die Analyse muss Sicherheitsfragen behandeln.

10.4.2 Formale Anforderungen

- Die Analyse muss alle notwendigen Verzeichnisse enthalten (Inhalts-, Quellen-, Tabellen-, Abbildungsverzeichnis).
- Alle Grafiken müssen innerhalb der Analyse erstellt werden.
- Quellen müssen zitiert werden.
- Der Einsatz von generativer KI muss deklariert werden.

10.4.3 Quellen

Die folgenden Quellen können für die quantitativen Aspekte der Analyse verwendet werden:

1. Nutzungsstatistiken & Marktanteile

- **GitHub – The State of the Octoverse (2025)**: Statistiken über die Anzahl der Repositories, die aktivsten Sprachen (Python, TypeScript) und das Wachstum von KI-Projekten.
- **Linux Foundation – World of Open Source Survey (2025)**: Fokus auf die Einführung in Unternehmen und Regierungen.
- **W3Techs – Web Technology Surveys**: Echtzeitdaten zur Nutzung von Webservern (Nginx, Apache) und Betriebssystemen im Web.

2. Ökonomische Aspekte & Kosten (TCO)

- **State of Open Source Report (Perforce/OSI)**: Jährliche Umfrage unter IT-Entscheidungsträgern zu den Gründen für den Einsatz von OSS.
- **Quandary Peak Research (2025)**: Wissenschaftliche Analyse des wirtschaftlichen Wertes von OSS.

3. Sicherheit & Risiken

- **Synopsys (Black Duck) – Open Source Security and Risk Analysis (OSSRA) Report**: Analyse von über 1.000 kommerziellen Codebasen auf Sicherheitsanfälligkeiten und Lizenzkonflikte.
- **Tidelift – State of the Open Source Maintainer Report**: Beleuchtet die „Risikoseite“ der Wartung (Supply Chain Security).

10.4.4 Bewertung

Die Arbeit wird nach den folgenden Kriterien bewertet:

Kriterium	Gewicht	3 Punkte (Sehr gut)	2 Punkte (Genügend)	1 Punkt (Ungenügend)
Nutzung von Markdown & Jupyter	25%	Notebook sauber strukturiert, Code/Text integriert, Ergebnisse reproduzierbar.	Notebook funktioniert, hat aber Mängel in Struktur oder Reproduzierbarkeit.	Notebook unvollständig, Code läuft nicht oder ist vom Text getrennt.
Datenvisualisierung	20%	Mind. zwei aussagekräftige Grafiken, korrekt beschriftet, im Text interpretiert und zitiert.	Grafiken vorhanden, aber mit Mängeln in Beschriftung, Interpretation oder Zitat.	Keine oder fehlerhafte Grafiken; fehlender Textbezug.
Diagrams as Code	10%	Mind. ein Diagramm (Mermaid/PlantUML) sinnvoll genutzt und korrekt gerendert.	Diagramm vorhanden, aber mit technischen oder inhaltlichen Mängeln.	Kein Diagramm oder nicht als Code erstellt.
Inhaltliche Qualität	25%	OSS definiert, Chancen/Risiken differenziert, Kosten/Sicherheit fundiert, Quellen korrekt.	Wesentliche Inhalte vorhanden, aber Argumente oberflächlich oder lückenhaft.	Zentrale Inhalte fehlen oder sind faktisch falsch.
Formale Anforderungen	10%	Alle Verzeichnisse vorhanden, Quellen vollständig, KI-Deklaration vorhanden.	Kleine Lücken in Verzeichnissen, Zitaten oder Umfang.	Wesentliche formale Anforderungen nicht erfüllt.
Sprache & Darstellung	10%	Klarer, präziser Ausdruck; korrekte Fachterminologie; gute Lesbarkeit.	Verständlich, aber sprachliche oder gestalterische Mängel.	Schwer verständlich oder durchgehend fehlerhaft.

10.4.5 Allgemeine Informationen

- **Abgabefrist:** 13. März 2026.
- **Tech Stack:** Jupyter Notebooks mit Python und Quarto.
- **Format:** Abgabe als PDF oder HTML inkl. aller Quelldateien (.qmd, .ipynb, .yaml, Bilder/Daten).

- **Konfiguration:** Eine `_quarto.yml` Datei mit der Basiskonfiguration für PDF wird bereitgestellt.

10.5 Kontakt

Fragen beantworte ich während des Unterrichts, via Teams oder nach Terminvereinbarung während der Sprechstunde.

Teil VI

Kryptologie

11 File Handling in Python

Um Textdateien in Python zu bearbeiten, schreiben wir eine Funktion, mit welcher der Inhalt einer Datei einer Variabel als String zugewiesen wird.

```
1 def file_reader(path : str) -> str:
2
3     with open(path, mode='r', encoding='utf-8') as f:
4         text = f.read()
5
6     return text
```

Um ver- oder entschlüsselte Texte in eine Datei zu schreiben, schreiben wir eine Funktion, die einen String in eine Datei schreibt.

```
1 def file_writer(path : str, text : str) -> None:
2     i = 0
3     grouped_text = ""
4     for c in text:
5         i += 1
6         if i % 50 == 0:
7             grouped_text += c + "\n"
8         elif i % 5 == 0:
9             grouped_text += c + " "
10        else:
11            grouped_text += c
12
13    with open(path, mode='w', encoding='utf-8') as f:
14        f.write(grouped_text)
```

Damit Texte ausschliesslich aus ASCII Grossbuchstaben bestehen, schreiben wir eine Funktion, die alle Kleinbuchstaben in Grossbuchstaben umwandelt und alle Umlaute in ihre aequivalenten Buchstaben umwandelt. Alle anderen Zeichen werden entfernt.

Damit alle Methoden für die Bearbeitung von Strings zur Verfügung stehen, muss das String-Modul importiert werden.

```
1 import string
```

```
1 def text_cleaning(text : str) -> str:
2     clean = text.upper() \
3         .replace('Ä', 'AE') \
4         .replace('Ö', 'OE') \
5         .replace('Ü', 'UE') \
6         .replace('ß', 'SS') \
7         .replace(' ', '') \
8
9     cleaned_text = ''
10
11     for c in clean:
12         if c.isalpha():
13             cleaned_text += c
14
15     return cleaned_text
```

12 Caesar Chiffre

In der Kryptologie wird mit Verschlüsselung jener Prozess beschrieben, der eine Nachricht so transformiert, dass sie idealerweise nur von autorisierten Parteien gelesen werden kann. Dieser Prozess wandelt die ursprüngliche Darstellung der Informationen, bekannt als Klartext, in eine alternative Form, bekannt als Chiffretext, um. Trotz seines Ziels verhindert die Verschlüsselung nicht selbst die Interferenz, sondern verweigert dem potenziellen Abhörer den verständlichen Inhalt.

Aus [Wikipedia](#)

Das erste Beispiel für eine Chiffre, das wir uns ansehen werden, ist die Caesar-Chiffre.

Die Caesar-Chiffre ist eine einfache Substitutionsverschlüsselungstechnik, bei der jeder Buchstabe des zu verschlüsselnden Textes durch einen Buchstaben ersetzt wird, der sich um eine feste Anzahl von Positionen im Alphabet verschiebt. Zum Beispiel würde bei einer Verschiebung um vier Positionen nach rechts A durch E ersetzt werden, und das Wort CIPHER würde zu GMTLIV. Die Technik ist nach Julius Caesar benannt, der sie in seinen Briefen verwendete. Die Einfachheit der Caesar-Chiffre macht sie zu einer beliebten Quelle für Freizeit-Kryptogramme.

Aus [Encyclopedia Britannica](#)

12.1 Python Implementation

Um die Caesar-Chiffre in Python zu implementieren, bauen wir auf den von Python angebotenen String-Methoden auf. Die Funktion `ord` gibt den Unicode-Codepunkt für ein gegebenes Zeichen zurück, und die Funktion `chr` gibt das Zeichen zurück, das dem gegebenen Unicode-Codepunkt entspricht.

```
1 # Was ist der Unicode Wert des Buchstabens 'A'?
```

12.1.1 Simple (naive) Implementation

Als erstes definieren wir eine Funktion, die einen gegebenen Klartext verschlüsselt, indem sie jeden Buchstaben um eine bestimmte Anzahl von Positionen im Alphabet verschiebt.

```
1 plain = "CIPHER"  
2 shift = 4
```

```

1 def caesar_encode(plain, shift):
2     cipher = ""
3
4     for char in plain:
5         shifted = ord(char) + shift
6         cipher += chr(shifted)
7
8     return cipher

```

12.1.2 Verbesserte Implementation

Was geschieht aber, wenn wir das Ende des Alphabets erreichen? Zum Beispiel, wenn wir Z um 4 Positionen verschieben, würden wir über Z hinausgehen. Um dies zu handhaben, können wir den Modulo-Operator % verwenden, um im Alphabet von vorne zu beginnen. Der Modulo-Operator gibt den Rest einer Division zurück, was es uns ermöglicht, wieder bei Null zu beginnen, wenn wir die Länge des Alphabets überschreiten. Im Fall der Caesar-Chiffre können wir ihn verwenden, um sicherzustellen, dass unsere verschobenen Positionen innerhalb der Grenzen des Alphabets bleiben.

Daher verwenden wir modulo 26 (die Anzahl der Buchstaben im lateinischen Alphabet), um sicherzustellen, dass unsere verschobenen Positionen korrekt "herumwickeln". Wenn wir Z um 4 Positionen verschieben, landen wir bei D. Dieses Wickelverhalten ist für die Funktionsweise der Caesar-Chiffre von entscheidender Bedeutung. Die Berechnung der neuen Position eines Buchstabens kann wie folgt ausgedrückt werden:

$$x' = (x + n) \mod 26$$

oder in einer kompakteren Form mit dem Modulo-Additionsoperator:

$$x' = x \oplus_{26} n$$

In Python können wir dies wie folgt implementieren:

```

1 def caesar_encrypt_mod(plain, shift):
2     cipher = ""
3
4     for char in plain:
5         shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
6         cipher += chr(shifted)
7
8     return cipher

```

Weil


```
1 ord('A')
```

65 zurückgibt, müssen wir 65 von dem Ergebnis von `ord('A')` subtrahieren, um 0 zu erhalten. Dies gibt uns den 0-basierten Index des Buchstabens im Alphabet, der für unsere Berechnungen nützlich ist.

Daher die oben gezeigte Berechnung.

Für die Entschlüsselung können wir einfach den Verschiebungswert subtrahieren anstatt ihn zu addieren:

```
1 def caesar_decrypt_mod(plain, shift):
2     cipher = ""
3
4     for char in plain:
5         shifted = (ord(char) - ord('A') - shift) % 26 + ord('A')
6         cipher += chr(shifted)
7
8     return cipher
```

Für die Bequemlichkeit implementieren wir eine Funktion die sowohl ver- als auch entschlüsselt.

```
1 def caesar(text : str, shift : int, encrypt=True) -> str:
2     text = text.upper()
3     result = ""
4
5     if encrypt:
6         for char in text:
7             shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
8             result += chr(shifted)
9     else:
10        for char in text:
11            shifted = (ord(char) - ord('A') - shift) % 26 + ord('A')
12            result += chr(shifted)
13
14    return result
```

12.2 Caesar Chiffre brechen

Es gibt zwei Hauptmethoden, um eine Caesar-Chiffre zu brechen:

1. Brute Force Attack: Versuchen Sie alle möglichen Verschiebungen (1-25) und sehen Sie, welche eine sinnvolle Ausgabe erzeugt.
2. Frequency Analysis: Analysieren Sie die Häufigkeit von Buchstaben im Chiffretext und vergleichen Sie sie mit der erwarteten Häufigkeit von Buchstaben in der Sprache.

12.2.1 Brute Force Attack

```
1 def caesar_bruteforce(ciphertext: str) -> None:
2     for shift in range(1, 26):
3         decrypted_text = caesar(ciphertext, shift, encrypt=False)
4         print(f"Shift {shift}: {decrypted_text}")
```

12.2.2 Häufigkeitsanalyse

In der deutschen Sprache erscheinen bestimmte Buchstaben häufiger als andere. Zum Beispiel ist der Buchstabe 'E' der häufigste Buchstabe in deutschen Texten, gefolgt von 'N', 'I', 'S', 'R' und 'A'. Durch die Analyse der Häufigkeit von Buchstaben im Chiffretext können wir fundierte Vermutungen darüber anstellen, welche Buchstaben im Klartext welchen im Chiffretext entsprechen.

Für eine genauere Analyse siehe die folgende Häufigkeitstabelle:

Platz	Buchstabe	Relative Häufigkeit
1.	E	17,40 %
2.	N	9,78 %
3.	I	7,55 %
4.	S	7,27 %
5.	R	7,00 %
6.	A	6,51 %
7.	T	6,15 %
8.	D	5,08 %
9.	H	4,76 %
10.	U	4,35 %
11.	L	3,44 %
12.	C	3,06 %
13.	G	3,01 %
14.	M	2,53 %
15.	O	2,51 %
16.	B	1,89 %
17.	W	1,89 %
18.	F	1,66 %
19.	K	1,21 %
20.	Z	1,13 %
21.	P	0,79 %
22.	V	0,67 %
23.	J	0,27 %
24.	Y	0,04 %
25.	X	0,03 %
26.	Q	0,02 %

Quelle: [Wikipedia](#)

Wir brauchen also eine Funktion, die die Häufigkeit jedes Buchstabens im Chiffretext zählt. Das Ergebnis sollte ein Dictionary sein, bei dem die Schlüssel die Buchstaben und die Werte die Zählungen jedes Buchstabens sind. Oder noch besser, die Häufigkeiten in Prozent.

Um die Sache einfach zu halten, nehmen wir an, dass der Eingabetext nur Großbuchstaben von A bis Z und keine Leerzeichen oder Satzzeichen enthält.

```
1 def letter_frequency(text: str) -> dict:
2     frequency = {}
3     total_letters = 0
4
5     for char in text:
6         if char not in frequency:
7             frequency[char] = 1
8         else:
9             frequency[char] += 1
10            total_letters += 1
11
12    for key, value in frequency.items():
13        frequency[key] = (value / total_letters) * 100
14
15
16    return frequency
```

Nachdem wir den häufigsten Buchstaben im Chiffretext gefunden haben, können wir annehmen, dass er dem Buchstaben 'E' im Klartext entspricht. Indem wir die Verschiebung berechnen, die erforderlich ist, um den häufigsten Buchstaben in 'E' zu verwandeln, können wir dann den gesamten Chiffretext mit diesem Verschiebungswert entschlüsseln.

```
1 def find_shift(char: str, e = 'E') -> int:
2     return (ord(char) - ord(e)) % 26
```

13 Polyalphabetische Chiffren

13.1 Vigenère Chiffre

Bereits im 9. Jahrhundert wurde im islamischen Raum die grosse Schwachstelle monoalphabetischer Chiffren (Caesar-Chiffre) erkannt. Die Verteilung der Buchstaben folgt in jeder Sprache einem spezifischen aber konstanten Muster. Für die deutsche Sprache ist die Verteilung der folgenden Tabelle zu entnehmen.

Buchstabe	relative Häufigkeit	Buchstabe	relative Häufigkeit	Buchstabe	relative Häufigkeit
a	0.0651	l	0.0344	w	0.0189
b	0.0189	m	0.0253	x	0.0003
c	0.0306	n	0.0978	y	0.0004
d	0.0508	o	0.0251	z	0.0113
e	0.1740	p	0.0079		
f	0.0166	q	0.0002		
g	0.0301	r	0.0700		
h	0.0476	s	0.0727		
i	0.0755	t	0.0615		
j	0.0027	u	0.0435		
k	0.0121	v	0.0067		

Um zu zeigen, dass dies sich mit (längeren) Texten deckt, wurde ein Kapitel aus dem Roman 'Der Zauberberg' von Thomas Mann ausgewertet. Die sich aus dieser Auswertung ergebende Verteilung wird in der folgenden Grafik der Verteilung aus der Tabelle gegenübergestellt.

```
1 import pdfplumber
2 import string
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import numpy as np
6 with pdfplumber.open("Mann_1989_Der Zauberberg - Roman.pdf") as pdf:
7     pages = pdf.pages
8     text = ""
9     for i, pg in enumerate(pages):
10         if i >= 23 and i < 42:
11             text += pages[i].extract_text() + "\n"
```

```

12 def text_cleaning(text : str) -> str:
13     clean = text.upper() \
14         .replace('Ä', 'AE') \
15         .replace('Ö', 'OE') \
16         .replace('Ü', 'UE') \
17         .replace('ß', 'SS') \
18         .replace(' ', '') \
19
20     cleaned_text = ''
21
22     for c in clean:
23         if c.isalpha():
24             cleaned_text += c
25
26     return cleaned_text
27
28 def file_writer(path : str, text : str) -> None:
29     i = 0
30     grouped_text = ""
31     for c in text:
32         i += 1
33         if i % 50 == 0:
34             grouped_text += c + "\n"
35         elif i % 5 == 0:
36             grouped_text += c + " "
37         else:
38             grouped_text += c
39
40     with open(path, mode='w', encoding='utf-8') as f:
41         f.write(grouped_text)
42     zauberberg = text_cleaning(text)
43     file_writer('zauberberg_bereinigt.txt', zauberberg)
44 def letter_frequency(text: str) -> dict:
45     frequency = {}
46     total_letters = 0
47
48     for char in text:
49         if char not in frequency:
50             frequency[char] = 1
51         else:
52             frequency[char] += 1
53         total_letters += 1
54
55     for key, value in frequency.items():
56         frequency[key] = (value / total_letters) * 100
57

```

```

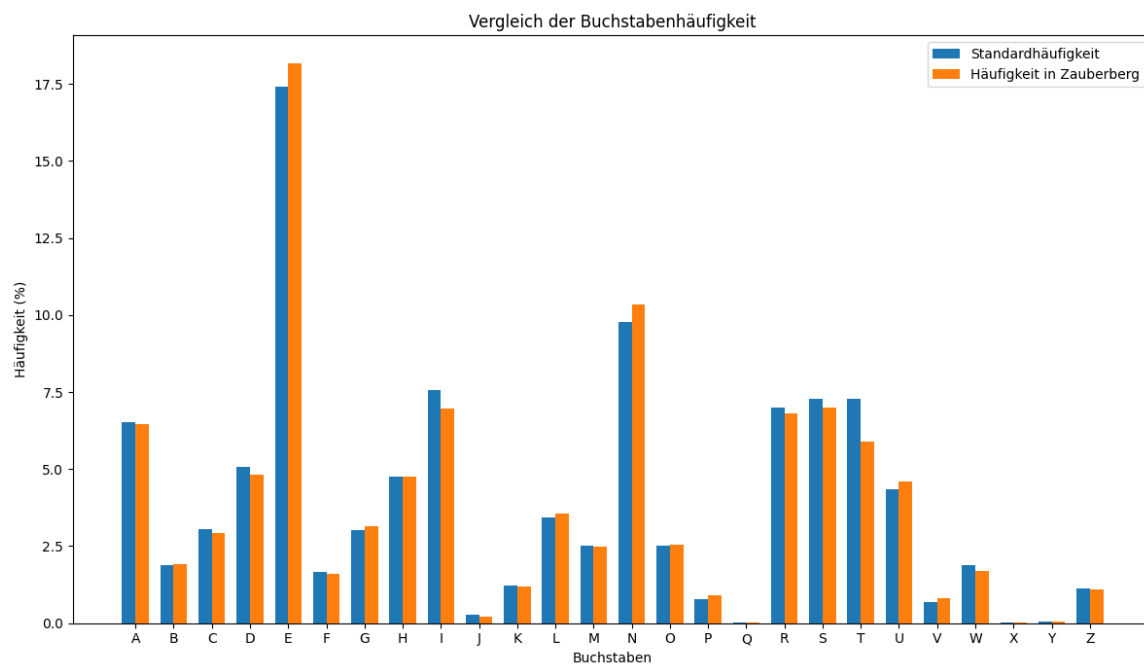
58
59     return frequency
60
61 frequency_zauberberg = letter_frequency(zauberberg)
62
63
64 standard_frequency = {
65     'E': 17.4,
66     'T': 7.27,
67     'A': 6.51,
68     'O': 2.51,
69     'I': 7.55,
70     'N': 9.78,
71     'S': 7.27,
72     'R': 7.0,
73     'H': 4.76,
74     'D': 5.08,
75     'L': 3.44,
76     'C': 3.06,
77     'U': 4.35,
78     'M': 2.53,
79     'W': 1.89,
80     'F': 1.66,
81     'G': 3.01,
82     'Y': 0.04,
83     'P': 0.79,
84     'B': 1.89,
85     'V': 0.67,
86     'K': 1.21,
87     'J': 0.27,
88     'X': 0.03,
89     'Q': 0.02,
90     'Z': 1.13
91 }
92
93
94 df = pd.DataFrame.from_dict([standard_frequency, frequency_zauberberg])
95 df.index = ['Standard Frequency', 'Zauberberg Frequency']
96 dft = df.T
97 dft = dft.sort_index()
98 dft['Standard Frequency'] = dft['Standard Frequency'].astype(float)
99 dft['Zauberberg Frequency'] = dft['Zauberberg Frequency'].astype(float)
100 # --- Erstellen Sie das Side-by-Side-Balkendiagramm ---
101 # Legen Sie die Breite der Balken fest
102 bar_width = 0.35
103

```

```

104 # Verwenden Sie den Index des transponierten DataFrames (dft)
105 x = np.arange(len(dft.index))
106
107 fig, ax = plt.subplots(figsize=(12, 7))
108
109 # Zeichnen Sie die Balken für die beiden Spalten aus dft
110 ax.bar(x - bar_width/2, dft['Standard Frequency'], bar_width,
111       ↪ label='Standardhäufigkeit')
112 ax.bar(x + bar_width/2, dft['Zauberberg Frequency'], bar_width,
113       ↪ label='Häufigkeit in Zauberberg')
114
115 ax.set_xticks(x)
116 ax.set_xticklabels(dft.index)
117
118 ax.set_xlabel('Buchstaben')
119 ax.set_ylabel('Häufigkeit (%)')
120 ax.set_title('Vergleich der Buchstabenhäufigkeit')
121 ax.legend()
122 plt.tight_layout()
123 plt.show()

```



Die Grafik zeigt, dass bei einer Textlänge von 51'396 Buchstaben die Verteilung in einem literarischen Text nahezu identisch ist mit der allgemeinen Häufigkeitsverteilung in der deutschen Sprache.

Die nächste Grafik zeigt, was mit der Verteilung der Buchstaben geschieht, wenn der gleiche Text

mit einer Caesar-Chiffre verschlüsselt worden ist.

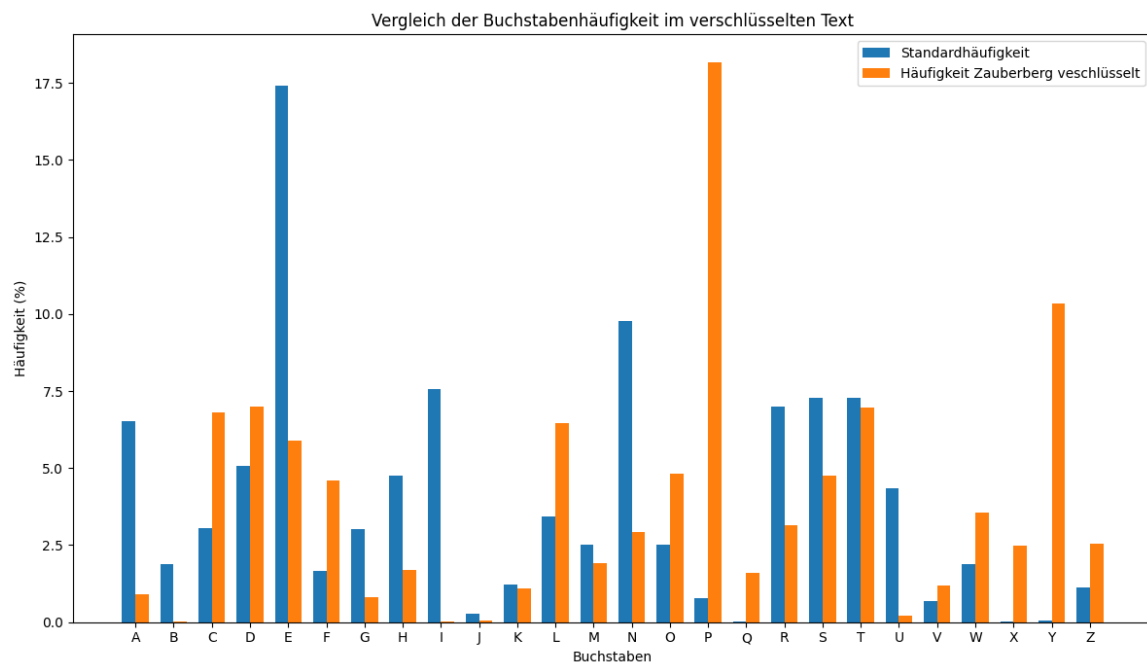
```
1 def caesar(text : str, shift : int, encrypt=True) -> str:
2     text = text.upper()
3     result = ""
4
5     if encrypt:
6         for char in text:
7             shifted = (ord(char) - ord('A') + shift) % 26 + ord('A')
8             result += chr(shifted)
9     else:
10        for char in text:
11            shifted = (ord(char) - ord('A') - shift) % 26 + ord('A')
12            result += chr(shifted)
13
14    return result
15
16
17 zauberberg_verschluesst = caesar(zauberberg, 11, encrypt=True)
18 frequency_zauberberg_verschluesst =
19     ↪ letter_frequency(zauberberg_verschluesst)
20 dft['Zauberberg Frequency Encrypted'] =
21     ↪ pd.Series(frequency_zauberberg_verschluesst)
22
23 # --- Erstellen Sie das Side-by-Side-Balkendiagramm ---
24 # Legen Sie die Breite der Balken fest
25 bar_width = 0.35
26
27 # Verwenden Sie den Index des transponierten DataFrames (dft)
28 x = np.arange(len(dft.index))
29
30 fig, ax = plt.subplots(figsize=(12, 7))
31
32 # Zeichnen Sie die Balken für die beiden Spalten aus dft
33 ax.bar(x - bar_width/2, dft['Standard Frequency'], bar_width,
34     ↪ label='Standardhäufigkeit')
35 ax.bar(x + bar_width/2, dft['Zauberberg Frequency Encrypted'],
36     ↪ bar_width, label='Häufigkeit Zauberberg verschlüsselt')
37
38 ax.set_xticks(x)
39 ax.set_xticklabels(dft.index)
40
41 ax.set_xlabel('Buchstaben')
42 ax.set_ylabel('Häufigkeit (%)')
43 ax.set_title('Vergleich der Buchstabenhäufigkeit im verschlüsselten
44     ↪ Text')
```



```

40 ax.legend()
41 plt.tight_layout()
42
43 plt.show()

```



Es ist deutlich zu erkennen, dass die Verteilung dem gleichen Muster folgt - verschoben um elf Positionen. Diese Auswertung ermöglicht die Entschlüsselung des Textes, ohne alle möglichen Schlüsselalphabete durchzuprobieren.

13.2 Vigenère Chiffre

Bei der Vigenère Chiffre handelt es sich um eine polyalphabetische Chiffre. Das Verfahren ist nach Blaise de Vigenère (1523 - 1596) benannt. Polyalphabetisch heisst, dass zur Verschlüsselung nicht eine Verschiebung vorgenommen wird, sondern - nach jedem Buchstaben wechselnd - mehrere Verschiebungen vorgenommen werden.

Um das zu erreichen, verwendet man ein sogenanntes Vigenère-Quadrat wie unten abgebildet.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Für die Verschlüsselung eines Klartextes braucht das Vigenère Verfahren ein Schlüsselwort. Das Schlüsselwort sollte möglichst lang sein. Das folgende Beispiel soll zeigen, wie das Vigenère Verfahren funktioniert. Der zu verschlüsselnde Klartext lautet 'Kryptologie ist spannend' und der Schlüssel 'Buelrain'. Als Hilfestellung werden Text und Schlüssel in einer Tabelle dargestellt.

```
kryptologieistspannend
buelrainbuelrainbuelra
```

Der Schlüssel wird dabei ohne Wortabstand so oft wiederholt, bis die Buchstabenfolge des Schlüssels gleich lang ist, wie die Buchstabenfolge, welche zu verschlüsseln ist.

Als nächstes wird der zu verschlüsselnde Buchstabe in der Kopfzeile des Vigenère Quadrates gesucht. Damit wird die Spalte mit dem verschobenen Alphabet identifiziert. Der chiffrierte Buchstabe

ergibt sich, indem in der Spalte mit den Zeilenköpfen der unter dem zu chiffrierenden Buchstaben befindliche Buchstabe des Schlüssels gesucht wird. Der Schnittpunkt der Zeile mit der vorher gefundenen Spalte entspricht dem chiffrierten Buchstaben.

kryptologieistspannend
buelrainbuelrainbuelra

LLCAKOTBHCITJTACBHRPED

Alternativ kann eine Verschlüsselung mit der Vigenère Chiffre auch mit modularer Arithmetik umgesetzt werden. Dazu wird jedem Buchstaben ein Zahlenwert nach dem Muster $a = 0, b = 1, \dots, z = 25$ zugewiesen. Die Verschlüsselung erfolgt anschliessend nach der 'Formel' $C_i = (P_i + K_i) \bmod 26$ wobei die Buchstaben C für den chiffrierten Text, P für den Klartext (Englisch *plain text*) und K für den Schlüssel (Englisch *key*) stehen. Der Index i steht für den i -ten Buchstaben in der Textfolge.

Das obige Beispiel stellt sich dann folgendermassen dar:

k	r	y	p	t	o	l	o	g	i	e	i	s	t	s	p	a	n	n	e	n	d
10	17	24	15	19	14	11	14	06	08	04	08	18	19	18	15	00	13	13	04	13	03
b	u	e	l	r	a	i	n	b	u	e	l	r	a	i	n	b	u	e	l	r	a
01	20	04	11	17	00	08	13	01	20	04	11	17	00	08	13	01	20	04	11	17	00
11	37	28	26	36	14	19	27	07	28	08	19	35	19	26	28	01	33	17	15	30	03
11	11	02	00	10	14	19	01	07	02	08	19	09	19	00	02	01	07	17	15	04	03
L	L	C	A	K	O	T	B	H	C	I	T	J	T	A	C	B	H	R	P	E	D

Für die Entschlüsselung wird die 'Formel' folgendermassen umgekehrt: $P_i = (C_i - K_i + 26) \bmod 26$. Die Addition von 26 in der Klammer erfolgt, um negative Zahlen zu vermeiden.

Wie sich die Vigenère Verschlüsselung auf die Verteilung der Buchstaben auswirkt, kann untenstehender Grafik entnommen werden.

```

1 def vigenere_chiffre(text: str, key: str, encrypt=True) -> str:
2     """
3     Implementiert die Vigenère-Verschlüsselung für einen gegebenen
    ↪ Klartext und
4     Schlüssel.
5
6     Args:
7         klartext (str): Der zu verschlüsselnde Text schluessel (str):
    ↪ Das
8         Schlüsselwort für die Verschlüsselung
9

```

```

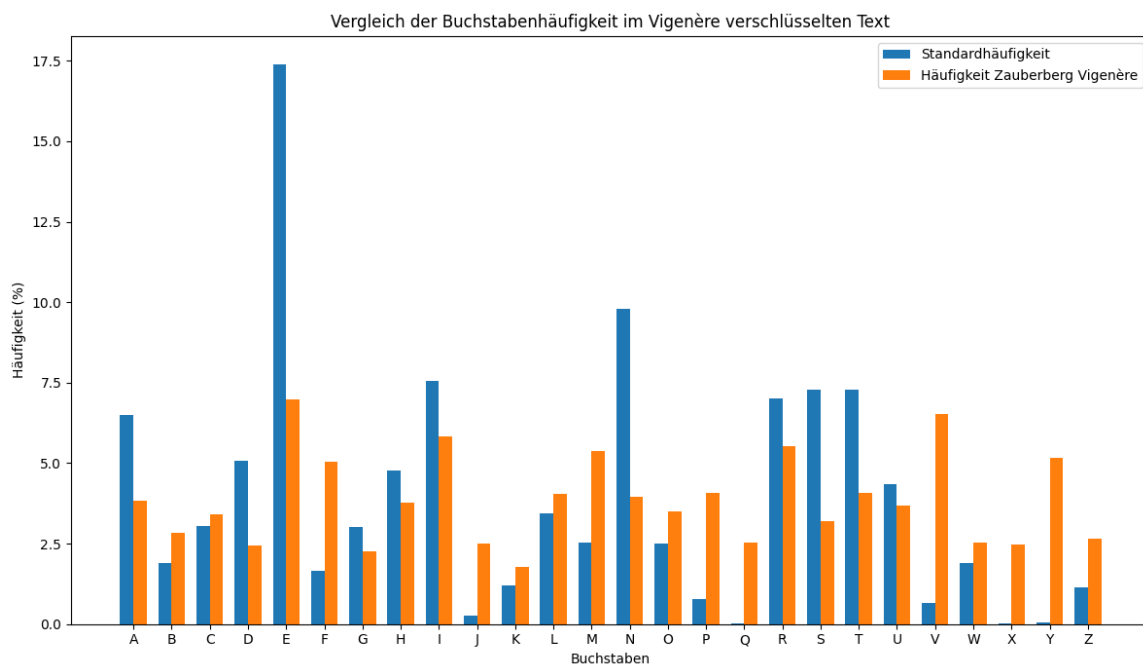
10 Returns:
11     str: Der verschlüsselte Text
12     ""
13
14 # initialisiere den resultierenden Text
15 resulting_text = ''
16
17 # bestimme die Schlüssellänge für die anschliessende
18     ↪ Modulo-Operation
19 key_length = len(key)
20
21 # iteriere über den Eingabetext unter gleichzeitiger Erfassung des
22     ↪ Index
23 for i, char in enumerate(text):
24     # berechne den Zahlwert des Buchstabens aus der ascii Tabelle
25     char_no = ord(char) - 97
26     key_no = ord(key[i % key_length]) - 97
27
28     if encrypt == True:
29         # berechne den Zahlwert des verschlüsselten Buchstabens
30         ciph_no = (char_no + key_no) % 26
31     else:
32         # berechne den Zahlwert des entschlüsselten Buchstabens
33         ciph_no = (char_no + (26 - key_no)) % 26
34
35     # übernehme das Zeichen aufgrund seines Zahlwertes aus der
36     ↪ ascii Tabelle
37 ciph = chr(ciph_no + 97)
38
39 # füge den Buchstaben am resultierenden Text an
40 resulting_text += ciph
41 return resulting_text
42
43 zauberberg_vigenere = vigenere_chiffre(zauberberg.lower(), 'buelrain',
44     ↪ encrypt=True)
45
46 zauberberg_vigenere_frequency =
47     ↪ letter_frequency(zauberberg_vigenere.upper())
48 dft['Zauberberg Frequency Vigenere'] =
49     ↪ pd.Series(zauberberg_vigenere_frequency)
50
51 # --- Erstellen Sie das Side-by-Side-Balkendiagramm ---
52 # Legen Sie die Breite der Balken fest
53 bar_width = 0.35
54
55 # Verwenden Sie den Index des transponierten DataFrames (dft)

```

```

50 x = np.arange(len(dft.index))
51
52 fig, ax = plt.subplots(figsize=(12, 7))
53
54 # Zeichnen Sie die Balken für die beiden Spalten aus dft
55 ax.bar(x - bar_width/2, dft['Standard Frequency'], bar_width,
56       ↪ label='Standardhäufigkeit')
57
58 ax.bar(x + bar_width/2, dft['Zauberberg Frequency Vigenere'],
59       ↪ bar_width, label='Häufigkeit Zauberberg Vigenère')
60
61 ax.set_xticks(x)
62 ax.set_xticklabels(dft.index)
63
64 ax.set_xlabel('Buchstaben')
65 ax.set_ylabel('Häufigkeit (%)')
66 ax.set_title('Vergleich der Buchstabenhäufigkeit im Vigenère
67       ↪ verschlüsselten Text')
68
69 ax.legend()
70 plt.tight_layout()
71
72 plt.show()

```



Wie unschwer zu erkennen ist, stellt sich die Verteilung der Buchstaben in einem polyalphabetisch verschlüsselten Text deutlich anders dar, als dies in normalen Text der Fall ist. Die Vigenère Chiffre galt daher während ungefähr 300 Jahren als 'la chiffre indéchiffable'.

Ein Spezialfall der Vigenère-Chiffre lässt sich tatsächlich nicht entschlüsseln. Das ist dann der Fall, wenn der Schlüssel länger ist als der Klartext. Man spricht in diesem Fall vom One-Time Pad.

14 Vigenère Chiffre Implementierung in Python

In diesem Notebook wird eine mögliche Implementierung der Vigenère Chiffre in Python vorgestellt.

```
1 def vigenere(text: str, key: str, mode: str) -> str:
2     """Encrypts or decrypts a given text using the Vigenère cipher.
3
4     The function processes a string of uppercase alphabetic characters
5     using a provided key for encryption or decryption.
6
7     Args:
8         text: The string to be encrypted or decrypted. It should only
9     ↪ contain
10             uppercase alphabetic characters (A-Z).
11         key: The key for the cipher. It should also only contain
12             uppercase alphabetic characters (A-Z).
13         mode: The operation to perform, must be 'encrypt' or 'decrypt'.
14
15     Returns:
16         The resulting ciphertext or plaintext.
17
18     Raises:
19         ValueError: If the mode is not 'encrypt' or 'decrypt'.
20     """
21
22     # Ensure the mode is valid before proceeding.
23     if mode not in ['encrypt', 'decrypt']:
24         raise ValueError("Mode must be 'encrypt' or 'decrypt'")
25
26     key_length = len(key)
27
28     if mode == 'encrypt':
29         cipher = ''
30         # Iterate through each character of the input text.
31         for i, char in enumerate(text):
32             # Convert the current text character and the corresponding
33             ↪ key
```

```

32         # character to a number (0-25).
33         # The key character is determined using modulo to cycle
34         ↪ through
35         # the key.
36         char_num = ord(char) - ord('A')
37         key_num = ord(key[i % key_length]) - ord('A')
38
39         # Calculate the new character's number using the Vigenère
40         ↪ encryption formula.
41         # The modulo operator ensures the result stays within the
42         ↪ range 0-25.
43         cipher_num = (char_num + key_num) % 26
44
45         # Convert the resulting number back to an uppercase
46         ↪ character and append it.
47         cipher += chr(cipher_num + ord('A'))
48     return cipher
49 else: # mode == 'decrypt'
50     plain = ''
51     # Iterate through each character of the input text.
52     for i, char in enumerate(text):
53         # Convert the current text character and the corresponding
54         ↪ key
55         # character to a number (0-25).
56         char_num = ord(char) - ord('A')
57         key_num = ord(key[i % key_length]) - ord('A')
58
59         # Calculate the new character's number using the Vigenère
60         # decryption formula.
61         # The modulo operator handles negative results, ensuring
62         ↪ the
63         # result is correct.
64         plain_num = (char_num - key_num) % 26
65
66         # Convert the resulting number back to an uppercase
67         # character and append it.
68         plain += chr(plain_num + ord('A'))
69     return plain

```

Die Funktion `vigenere()` wird im Folgenden detailliert erklärt.

Die Signatur der Funktion

```

1 def vigenere(text: str, key: str, mode: str) -> str:

```


zeigt, dass die Funktion drei Parameter erwartet: * text: Der zu verschlüsselnde oder zu entschlüsselnde Text als String. * key: Der Schlüssel zur Verschlüsselung oder Entschlüsselung als String. * mode: Der Modus, der angibt, ob der Text verschlüsselt oder entschlüsselt werden soll. Mögliche Werte sind 'encrypt' für Verschlüsselung und 'decrypt' für Entschlüsselung (dies ist allerdings nicht direkt aus der Signatur ersichtlich).

Anschliessend an die Signatur folgt ein ausführlicher Docstring, der die Funktion und ihre Parameter beschreibt. Dieser ist - obwohl in Englisch abgefasst - aus sich selber heraus verständlich.

Der Docstring wird gefolgt durch die Prüfung, ob zulässige Werte für den mode-Parameter übergeben wurden. Ist dies nicht der Fall, wird eine ValueError-Exception ausgelöst.

```
1 if mode not in ['encrypt', 'decrypt']:
2     raise ValueError("Mode must be 'encrypt' or 'decrypt'")
```

Um diese Prüfung durchzuführen, werden die zulässigen Werte in einer Liste zur Verfügung gestellt. Geprüft wird dann, ob der mode zugewiesene Wert in der Liste enthalten ist. Falls dies nicht der Fall ist, wird eine Fehlermeldung ausgegeben und die Ausführung der Funktion abgebrochen.

Sofern der mode-Parameter einen zulässigen Wert enthält, beginnt die eigentliche Funktionslogik zu spielen.

Als erstes wird die Länge des Schlüssels in

```
1 key_length = len(key)
```

der Variablen key_length zugewiesen. Diese Information wird weiter unten benötigt, um in einer besonderen Art von Schleife über den Text des Schlüssels zu iterieren.

Nach dieser Zuweisung teilt sich die Funktion in die Zweige Ver- bzw. Entschlüsselung.

Zuerst wird die Verschlüsselung - eingeleitet mit `if mode == 'encrypt':` - betrachtet.

Innerhalb dieses Blocks wird als erstes ein leerer String cipher initialisiert, der später die verschlüsselten Zeichen aufnehmen wird.

Anschliessend wird mit einer for-Schleife über den zu verschlüsselnden Text iteriert.

```
1 for i, char in enumerate(text):
```

In dieser Schleife wird die Funktion `enumerate()` verwendet. Diese Funktion liefert ein Tupel aus Index und dem jeweiligen Element der Struktur, über die iteriert wird. Die Werte des Tupels werden den Variablen i und char zugewiesen.

Die Variablen i und char werden innerhalb der Schleife verwendet, um die einzelnen Zeichen des Textes in eine Zahl umzuwandeln.

```

1 char_num = ord(char) - ord('A')
2 key_num = ord(key[i % key_length]) - ord('A')

```

Die Funktion `ord()` wandelt einen Buchstaben in die entsprechende Zahl aus der ASCII-Tabelle um. Damit die Zahlen im Bereich von 0 bis 25 liegen, wird der ASCII-Wert des Buchstabens 'A' abgezogen. Der Buchstabe 'A' wird verwendet, da die zu verschlüsselnden Zeichen in Grossbuchstaben vorgegeben sind.

Da der Schlüssel kürzer als der zu verschlüsselnde Text sein kann, wird mit `i % key_length` über den Schlüssel iteriert. Der Modulo-Operator `%` sorgt dafür, dass der Index Wert des verwendeten Index immer zwischen 0 und der Länge des Schlüssels `key_length` bleibt. Dadurch wird sichergestellt, dass der Schlüssel wieder von vorne begonnen wird, sobald das Ende des Schlüssels erreicht ist. Anschliessend wird mit den einzelnen Buchstaben des Schlüssels genauso verfahren wie mit den Buchstaben des Textes.

Nachdem die Buchstaben des Textes sowie des Schlüssels in Zahlen umgewandelt wurden, wird die eigentliche Verschlüsselung gemäss der Formel $C_i = (P_i + K_i) \bmod 26$ durchgeführt.

```

1 cipher_num = (char_num + key_num) % 26

```

Die Zahlenwerte des verschlüsselten Textes werden anschliessend wieder in Buchstaben umgewandelt und an den String `cipher` angehängt. Dazu wird die Funktion `chr()` verwendet, die eine Zahl in den entsprechenden Buchstaben der ASCII-Tabelle umwandelt. Da die Zahlenwerte im Bereich von 0 bis 25 liegen, wird der ASCII-Wert des Buchstabens 'A' addiert.

```

1 cipher += chr(cipher_num + ord('A'))

```

Der unter `cipher` abgelegte String wird am Ende des Blocks zurückgegeben.

Im zweiten Block wird die Entschlüsselung - eingeleitet mit `else:` - durchgeführt. Der Prozess ist dem der Verschlüsselung sehr ähnlich, jedoch wird hier die umgekehrte Formel ($P_i = (C_i - K_i + 26) \bmod 26$) verwendet.

```

1 plain_num = (char_num - key_num + 26) % 26
2 plain += chr(plain_num + ord('A'))

```

Alles andere entspricht dem Vorgehen bei der Verschlüsselung.

15 Die Vigenère-Verschlüsselung brechen

Die entscheidende Schwäche der Vigenère-Verschlüsselung wurde Mitte des 19. Jahrhunderts von zwei Männern unabhängig voneinander entdeckt. Einer von ihnen war Charles Babbage, ein britischer Universalgelehrter. Er entdeckte die Schwachstelle, um eine Wette zu gewinnen. Der andere war Friedrich Wilhelm Kasiski, ein pensionierter Major der preußischen Armee, der nach seiner Pensionierung seine Zeit mit Schreiben verbrachte. Die Methode zum Brechen der Vigenère-Verschlüsselung ist nach dem preußischen Major Kasiski benannt.

15.1 Die Kasiski-Methode – Zusammenfassung

Die **Kasiski-Methode** ist eine Technik zum Brechen der Vigenère-Verschlüsselung, bei der wiederholte Muster im Geheimtext ausgenutzt werden. So funktioniert sie:

15.1.1 Grundprinzip

Wenn dieselbe Klartextsequenz mit demselben Teil des Schlüssels verschlüsselt wird, entstehen identische Geheimtextsequenzen. Indem wir diese Wiederholungen finden, können wir die Schlüssellänge bestimmen.

15.1.2 Schritt-für-Schritt-Anleitung

1. Wiederholte Sequenzen finden

- Den Geheimtext nach identischen Sequenzen von 3+ Zeichen durchsuchen.
- Die Positionen aufzeichnen, an denen diese Sequenzen auftreten.

2. Abstände berechnen

- Den Abstand zwischen wiederholten Sequenzen messen.
- Die Schlüssellänge muss ein Teiler dieser Abstände sein.

3. Schlüssellänge bestimmen

- Den größten gemeinsamen Teiler (ggT) aller Abstände finden.
- Oder nach dem häufigsten gemeinsamen Teiler suchen.
- Dies ergibt die wahrscheinliche Schlüssellänge.

4. Häufigkeitsanalyse

- Den Geheimtext basierend auf der Schlüssellänge in Gruppen aufteilen.
- Jede Gruppe wurde mit demselben Schlüsselzeichen verschlüsselt.
- Eine Häufigkeitsanalyse auf jede Gruppe separat anwenden.
- Die Buchstabenhäufigkeiten mit den erwarteten Mustern der Sprache abgleichen.

15.1.3 Beispiel

Wenn „THE“ mehrmals im Klartext an Positionen erscheint, an denen sich der Schlüssel wiederholt, sind die entsprechenden Geheimtextsequenzen identisch. Wenn diese Sequenzen 10 Positionen voneinander entfernt sind, ist die Schlüssellänge wahrscheinlich ein Teiler von 10 (1, 2, 5 oder 10).

Für ein konstruiertes Beispiel, wenn der Klartext „the codes in the word and the message“ lautet und der Schlüssel „crypt“ ist, erhalten wir den folgenden Geheimtext.

```

1 p: t h e c o d e s i n t h e w o r d a n d t h e m
2 k: c r y p t c r y p t c r y p t c r y p t c r y p
3 c: V Y C r h f v q x g V Y C l h t u y c w V Y C b

```

Beachten Sie, dass das Geheimtextmuster VYC dreimal im Geheimtext vorkommt und jedes Mal ein Abstand von 10 Buchstaben zwischen dem Anfang eines VYC und dem nächsten liegt. Dies geschieht, weil dasselbe Klartextmuster, „the“, jedes Mal auf denselben Teil des Schlüssels, „cry“, trifft, was zum selben Geheimtext führt. Die Wiederholung des Schlüsselworts ist hier die Schwachstelle. Die Duplikate im Geheimtext sind alle 10 Buchstaben voneinander entfernt. Babbage und Kasiski schlussfolgerten, dass dies bedeutet, dass die Länge des Schlüssels ein Teiler von 10 ist. Die Teiler von 10 sind 10, 5 und 2. Man könnte argumentieren, dass ein Schlüssel der Länge 2 zu kurz ist, um viel Sicherheit zu bieten, sodass ein Schlüssel der Länge 5 oder 10 wahrscheinlicher ist. Ein Schlüssel der Länge 10 hätte in einem so kurzen Geheimtext wahrscheinlich nicht so viele Wiederholungen, daher wird der Kryptoanalytiker seine Arbeit mit einer angenommenen Schlüssellänge von 5 beginnen.

Eine Schlüssellänge von 5 bedeutet, dass der 1., 6., 11., 16. usw. Buchstabe alle mit demselben Schlüsselbuchstaben und somit mit demselben Alphabet aus der Vigenère-Tabelle verschlüsselt werden. In ähnlicher Weise werden der 2., 7., 12., 17. usw. Buchstabe alle mit dem nächsten Schlüsselalphabet verschlüsselt. Wenn wir also das Kryptogramm in 5 Buchstabengruppen aufteilen, haben wir 5 monoalphabetische Substitutions-Geheimtexte. Wir können dann eine Häufigkeitsanalyse jeder Gruppe durchführen und jede Gruppe separat lösen. Und in einem standardmäßig verschobenen Alphabet wie in der normalen Vigenère-Tabelle haben wir, wenn wir einen einzigen Geheimtext-Alphabetbuchstaben finden können, das gesamte Alphabet.¹

¹Dooley, John F. History of Cryptography and Cryptanalysis: Codes, Ciphers, and Their Algorithms. History of Computing. Cham: Springer International Publishing, 2024. <https://doi.org/10.1007/978-3-031-67485-3>. S. 76.

15.1.4 Warum es funktioniert

Die Schwäche der Vigenère-Verschlüsselung liegt in der Wiederholung des Schlüssels. Sobald die Schlüssellänge bekannt ist, wird die Verschlüsselung zu mehreren einfachen Caesar-Verschlüsselungen, die mittels Häufigkeitsanalyse gebrochen werden können.

16 Public-Key-Kryptographie

Obwohl das One-Time-Pad theoretisch eine absolut sichere Verschlüsselung ermöglicht, ist es in der Praxis kaum praktikabel. Neben der Tatsache, dass der Schlüssel mindestens so lang sein muss, wie die Nachricht selbst, braucht es ein Verfahren den bzw. die Schlüssel sicher zwischen Sender und Empfänger zu teilen.

Um das Problem des Schlüsselaustausches zu lösen, verwendet man spezielle mathematische Funktionen. Diese Funktionen nennt man "Einwegfunktionen mit Hintertüren". Eine Einwegfunktion $f(x) = x'$ lässt sich leicht berechnen, aber ihre Umkehrung $f^{-1}(x') = x$ ist sehr schwierig zu berechnen. Die "Hintertür" ist ein Geheimwissen, mit dem die Umkehrfunktion dann doch einfach berechnet werden kann.

Die Einwegfunktion kann veröffentlicht werden, damit ein Absender mit deren Hilfe eine Botschaft verschlüsseln kann. Nur der Empfänger ist dann noch in der Lage, die Botschaft mit wenig Aufwand zu entschlüsseln. Allfällige 'Lauscher' können die Umkehrfunktion nicht innert nützlicher Frist berechnen.

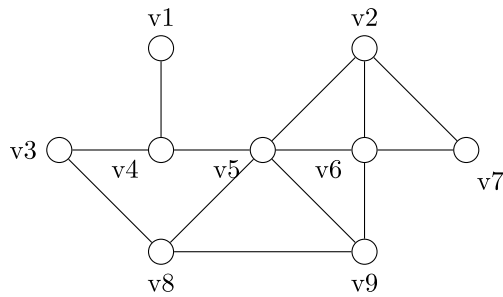
Um ein Beispiel einer solchen Einwegfunktion zu zeigen, wird der Umstand genutzt, dass Text als Zahlenfolge dargestellt werden kann. Ein als Zahlenfolge dargestellter Text kann dann mit Hilfe einer mathematischen Funktion verschlüsselt werden. Im folgenden soll ein Modell für ein solches Verschlüsselungsverfahren vorgestellt werden. In diesem Modell werden Graphen für die Modellierung einer Einwegfunktion verwendet.

16.1 Verschlüsselung mit Hilfe eines Graphen

Ein Graph besteht aus Knoten und Kanten. Die Knoten sind durch Kanten miteinander verbunden. Damit die Verschlüsselung mit Hilfe eines Graphen erfolgen kann, muss der Graph öffentlich bekannt und die Knoten nummeriert sein. Die Verschlüsselung erfolgt in den unten dargestellten Schritten.

1. Der Klartext wird als Folge von Zahlen dargestellt, welche folgendermassen verschlüsselt werden:
2. Die einzelnen Zahlen der Zahlenfolge werden in Summanden zerlegt. Die Zahl der Summanden entspricht der Anzahl der Knoten im Graphen.
3. Jeder Summand wird einem Knoten zugeordnet.
4. Der 'Wert' eines Knotens berechnet sich als Summe des dem Knoten zugeordneten Summanden und allen Summanden der Nachbarknoten.
5. Der verschlüsselte Wert der Zahl ist die Folge der Werte der Knoten.

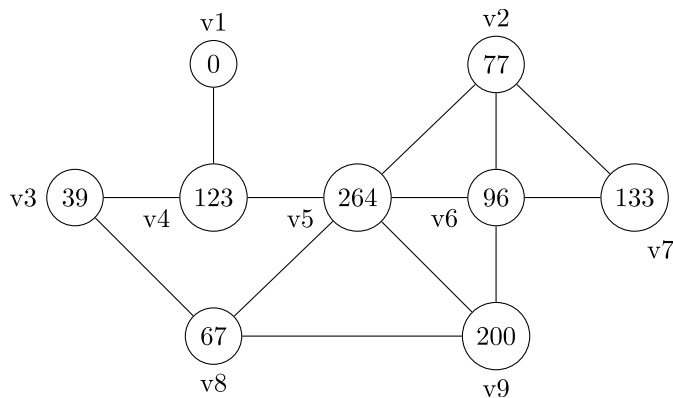
Das Vorgehen soll an einem Beispiel verdeutlicht werden. Dem Beispiel liegt der folgende Graph zu Grunde.



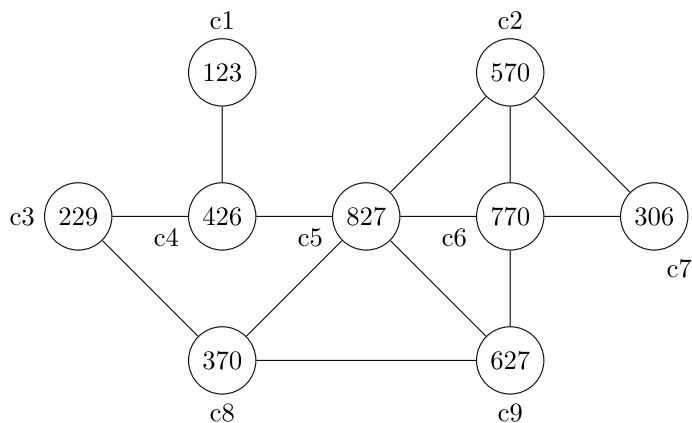
In diesem Graphen wird die Zahl 999 verschlüsselt. Die Aufteilung in Summanden sieht folgendermassen aus:

$$999 = 0 + 77 + 39 + 123 + 264 + 96 + 133 + 67 + 200$$

Die Summanden werden folgendermassen in den Graphen eingetragen:



Nach der Addition der Nachbarn stellt sich der Graph folgendermassen dar:



Der verschlüsselte Wert kann jetzt als Zahlenfolge 123/570/229/426/827/770/306/370/627 übermittelt werden.

Um die ursprüngliche Zahl zu rekonstruieren, muss ein unberechtigter Lauscher das folgende Gleichungssystem lösen:

$$\begin{aligned}c_1 &= v_1 + v_4 \\c_2 &= v_2 + v_5 + v_6 + v_7 \\c_3 &= v_3 + v_4 + v_8 \\c_4 &= v_1 + v_3 + v_4 + v_5 \\c_5 &= v_2 + v_4 + v_5 + v_6 + v_8 + v_9 \\c_6 &= v_2 + v_5 + v_6 + v_7 + v_9 \\c_7 &= v_2 + v_6 + v_7 \\c_8 &= v_3 + v_5 + v_8 \\c_9 &= v_5 + v_6 + v_8 + v_9\end{aligned}$$

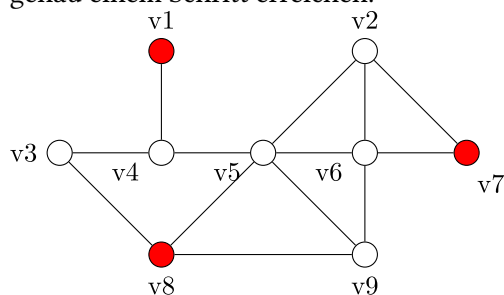
Wobei c_n die jeweilige übermittelte Zahl und v_n der Summand im jeweiligen Knoten darstellt. Dieses Gleichungssystem ist noch innerhalb nützlicher Frist lösbar. Wenn der Graph aber grösser wird, stösst man bald an zeitliche Grenzen.

Viel einfacher ist die Lösung, wenn man den Graphen in seine dominierende Menge zerlegt. Eine dominierende Menge ist eine Auswahl von Knoten, sodass jeder andere Knoten entweder in dieser Menge liegt oder einen Nachbarn darin hat. Dies so, dass innerhalb des Subgraphen die untereinander verbundenen Knoten mit einem einzigen Knoten verbunden sind.

16.2 Definition Dominierende Menge

i Definition Dominierende Menge

Eine dominierende Menge in einem Graphen ist eine Teilmenge von Knoten mit einer besonderen Eigenschaft: Jeder Knoten des Graphen, der nicht zu dieser Teilmenge gehört, ist mit mindestens einem Knoten aus dieser Teilmenge verbunden. Anders ausgedrückt: Von den Knoten der dominierenden Menge aus kann man jeden anderen Knoten des Graphen mit genau einem Schritt erreichen.



Die Kenntnis der dominierenden Menge stellt in unserem Verschlüsselungsverfahren die “Hintertür” dar. Wer diese Menge kennt, kann das komplizierte Gleichungssystem umgehen und die ursprünglichen Werte deutlich einfacher rekonstruieren.

In den Knoten v_1 , v_7 und v_8 sind alle Summanden enthalten. Um die ursprüngliche Zahl zu rekonstruieren, reicht es also, die Werte dieser drei Knoten zu summieren. Das Finden der dominierenden Menge ist aber ein schwierig zu lösendes Problem und stellt daher für den Lauscher eine unüberwindbare Schranke dar.¹

¹Karin Freiermuth, *Einführung in die Kryptologie: Lehrbuch für Unterricht und Selbststudium*, 2., überarb. Aufl, Einführung in die Kryptologie (Vieweg, 2014), 244 ff.

17 RSA

RSA ist ein Kryptografie-System für asymmetrische Verschlüsselung und Signatur. Es wurde Jahrzehnte lang verwendet und wird nun stetig von neueren Systemen abgelöst.

17.1 Lernziele

1. Sie entschlüsseln und verschlüsseln eine Zahl mit dem gegebenen öffentlichen, beziehungsweise mit dem privaten Schlüssel.
2. Sie wissen, auf welcher mathematischen Schwierigkeit RSA beruht. Anders ausgedrückt: Was muss man machen, um vom öffentlichen auf den privaten Schlüssel zu kommen.
3. Sie erklären das Prinzip der Signatur. Sie berechnen eine RSA Signatur. Sie überprüfen, ob eine Nachricht zur RSA Signatur passt.

17.2 Die Mathematik hinter RSA

Wie RSA funktioniert, betrachten wir mit dieser interaktiven Webseite www.cryptool.org/de/cto/rsa-step-by-step/.

17.2.1 Tipps

Der Taschenrechner kann nicht gut modulo mit grossen Zahlen rechnen.

Python ist da besser geeignet. Wenn Sie $c = 88^{17} \bmod 143$ rechnen möchten, können Sie es in Python wie folgt berechnen:

```
1 c = 88**17 % 143
2 print(c)
3 # 121
```

Zusätzlicher Hintergrund zum Rechnen

Python kann 88000^{80021} nicht ausrechnen. Im Gegensatz dazu ist die Berechnung von $88000^{80021} \bmod 89911$ möglich.

88000^{80021} ist zu gross für den Speicher. In der zweiten Berechnung berechnet Python immer wieder den Rest. Das braucht viel weniger Speicher und geht erst noch schneller.

17.2.2 Signieren

Betrachten wir die Situation, dass Alice mitteilen möchte, wann Sie am Bahnhof ankommt. Sie kann es nicht direkt an Bob schreiben, sondern Mallory. Sie möchte nicht, dass Bob zu einer falschen Uhrzeit am Bahnhof wartet. Alice hat Bob deshalb im Vorfeld ihren öffentlichen Schlüssel geteilt.

- Privater Schlüssel: $(n = 143, d = 113)$
- Öffentlicher Schlüssel: $(n = 143, e = 17)$

Mallory darf den öffentlichen Schlüssel auch kennen. Der private Schlüssel kennt nur Alice.

Was Alice macht, ist sie gibt die Stunde bekannt: Sie kommt um 15 Uhr an. Nun berechnet Alice zusätzlich

$$15^d \bmod n = 15^{113} \bmod 143 = 97$$

. Sie legt diese Zahl 97 als Signatur bei.

Wenn Mallory nichts ändert, bekommt Bob "Uhrzeit 15, Signatur 97". Bob kann nun den Inhalt überprüfen, indem er

$$signature^e \bmod n = 97^{17} \bmod 143 = 15$$

berechnet. Er merkt, die Zahl stimmt überein.

Wenn Mallory eine falsche Zeit an Bob geben möchte und die Zeit zu 14 anpasst, merkt Alice, dass die Zeit nicht mit der Signatur übereinpasst. Wenn Mallory die Signatur auch anpassen möchte, müsste sie die Signatur finden, sodass

$$x = 14^e \bmod n.$$

Dies ist genau so schwierig, wie das Entschlüsseln der Nachricht.



Aufgabe

1. Erstellen Sie auf der [bekannten cryptool Seite](#) ein Schlüsselpaar.
2. Wählen Sie (zufällig) 3 Zahlen.
3. Erstellen Sie zu jeder Zahl die Signatur.
4. Verändern Sie bei einem Tupel die Zahl, bei einem anderen die Signatur.
5. Tauschen Sie den öffentlichen Schlüssel mit den 3 Tupeln mit Ihrem Nachbar aus.
6. Testen Sie die Tupel, welches stimmt.

Teil VII

Computernetzwerke

18 Computernetzwerke

Computernetzwerke sind Systeme, die es Computern und anderen Geräten ermöglichen, miteinander zu kommunizieren. Ursprünglich ging es vor allem um den Austausch von Nachrichten. Heute ermöglichen Netzwerke zusätzlich die gemeinsame Nutzung von Ressourcen wie Druckern, Dateien oder Internetzugängen. Sie bilden die Grundlage für viele alltägliche Anwendungen – von E-Mails über Videokonferenzen bis hin zu Cloud-Diensten.

Ein Computernetzwerk besteht aus mehreren miteinander verbundenen Geräten (z. B. Computer, Smartphones, Server), die über verschiedene Übertragungsmedien wie Kabel oder Funk kommunizieren. Wichtige Ziele eines Netzwerks sind: - effiziente Kommunikation - zuverlässige Verfügbarkeit von Informationen - Sicherheit der Datenübertragung (in der Praxis meist durch Verfahren auf höheren Schichten, z. B. Verschlüsselung über TLS)

Um die Kommunikation zu ermöglichen, werden in Netzwerken Regeln und Protokolle verwendet. Ein bekanntes Referenzmodell zur Beschreibung dieser Abläufe ist das OSI-Schichtenmodell. Es unterteilt die Netzwerkkommunikation in sieben aufeinander aufbauende Schichten – von der physischen Übertragung der Daten bis hin zur Anwendungsebene, auf der Programme wie Webbrowser oder E-Mail-Clients arbeiten.

18.1 Das OSI- bzw. TCP/IP-Modell

Das OSI-Modell (Open Systems Interconnection Model) ist ein Referenzmodell. Es hilft, die komplexen Abläufe der Datenübertragung zu strukturieren und zu verstehen. Die sieben Schichten (mit typischen Dateneinheiten, sogenannten PDUs) im Überblick:

1) Physical Layer

- Überträgt einzelne Bits als elektrische, optische oder Funksignale über das Übertragungsmedium (z. B. Kabel, Funk).
- PDU: Bits

2) Data Link Layer

- Sorgt für lokale, rahmenbasierte Übertragung zwischen direkt verbundenen Geräten; regelt den Zugriff auf das Medium (MAC).
- Bietet Fehlererkennung (z. B. CRC) und – je nach Technik – ggf. einfache Fehlerbehandlung/Wiederholungen (z. B. bei WLAN).
- PDU: Frames/Rahmen

3) Network Layer

- Leitet Pakete über mehrere Netzwerke hinweg weiter (Routing); Adressierung auf Netzwerkebene (z. B. IP-Adressen).
- Beispielprotokoll: Internet Protocol (IP).
- PDU: Packets/Pakete

4) Transport Layer

- Stellt Ende-zu-Ende-Transport zwischen Anwendungen bereit. Unterschiedliche Protokolle bieten unterschiedliche Dienste:
 - TCP: zuverlässig, geordnet, verbindungsorientiert; inkl. Fluss- und Staukontrolle.
 - UDP: verbindungslos, ohne Garantie für Zuverlässigkeit oder Reihenfolge.
- PDU: Segmente (TCP) bzw. Datagramme (UDP)

5) Session Layer

- Baut Sitzungen zwischen Anwendungen auf, verwaltet und beendet sie (z. B. Sitzungsverwaltung, Dialogsteuerung).

6) Presentation Layer

- Übersetzt Daten zwischen Darstellungsformaten (z. B. Zeichencodierung, Serialisierung) und kann Daten komprimieren/verschlüsseln.

7) Application Layer

- Stellt die Schnittstelle zu Anwendungsprogrammen und Anwendungsprotokollen bereit (z. B. HTTP, SMTP, DNS).

Das OSI-Schichtenmodell wurde in seiner reinen Form nicht als Protokollstapel umgesetzt. In der Praxis hat sich das TCP/IP-Modell durchgesetzt (TCP = Transmission Control Protocol, IP = Internet Protocol).

Das TCP/IP-Modell besteht aus vier Schichten, die Funktionen des OSI-Modells zusammenfassen:

1) Netzwerkzugangsschicht (Link/Network Access)

- Entspricht OSI Physical + Data Link (Schichten 1 und 2).
- Beispiele: Ethernet, WLAN; MAC-Adressen; Switches arbeiten typischerweise hier.
- PDU: Frames/Rahmen

2) Internetschicht

- Entspricht OSI Network Layer (Schicht 3).
- Beispiele: IP, Routing; Router arbeiten hier; IP-Adressen.
- PDU: Packets/Pakete

3) Transportschicht

- Entspricht OSI Transport Layer (Schicht 4).

- Beispiele: TCP (zuverlässig, geordnet), UDP (verbindungslos); Ports identifizieren Dienste.
- PDU: Segmente (TCP) / Datagramme (UDP)

4) Anwendungsschicht

- Fasst OSI Session, Presentation und Application (Schichten 5–7) zusammen.
- Beispiele: HTTP/HTTPS, SMTP, DNS. Verschlüsselung wie TLS liegt typischerweise oberhalb des Transports und wird Anwendungen zugeordnet.

Das TCP/IP-Modell ist heute die Grundlage für die Kommunikation im Internet und in den meisten Computernetzwerken.

18.2 Analogie zur Kapselung in der objektorientierten Programmierung (OOP)

Das Schichtenmodell in der Netzwerktechnik lässt sich gut mit dem Prinzip der Kapselung in der objektorientierten Programmierung vergleichen. In beiden Fällen werden komplexe Aufgaben in klar abgegrenzte, eigenständige Einheiten (Schichten bzw. Klassen) unterteilt. Jede Schicht im Netzwerkmodell hat eine genau definierte Aufgabe und kommuniziert nur mit der direkt darüber- oder darunterliegenden Schicht – ähnlich wie eine Klasse in der OOP ihre inneren Details verbirgt und nur über definierte Schnittstellen mit anderen Klassen interagiert.

Durch diese Kapselung wird die Komplexität reduziert, Änderungen können leichter vorgenommen werden und die einzelnen Komponenten bleiben unabhängig voneinander wartbar. So wie in der OOP eine Klasse ihre Daten und Methoden kapselt, kapselt im Schichtenmodell jede Ebene die Details ihrer Funktion und stellt nur die notwendigen Dienste für die nächste Schicht bereit.

19 IP Adressen und DNS

19.1 IP Adressen

Damit Computer in einem Netzwerk erreichbar sind, benötigen sie eine eindeutige Adresse. Diese Adresse wird als IP-Adresse (Internet Protocol Address) bezeichnet. Aktuell - das heisst seit ungefähr zwei Jahrzehnten - wird nach und nach vom IPv4 System auf IPv6 umgestellt.

IPv4 sind 32-Bit-Adressen, die in vier Oktetten dargestellt werden (z.B. 192.0.2.1). Eine Länge von 32 Bit ergibt 2^{32} , das heisst etwas mehr als 4.29 Milliarden mögliche Adressen. Das zeigt, dass IPv4-Adressen begrenzt sind. Diese Begrenztheit hat zur Entwicklung von IPv6 geführt.

IPv6 Adressen sind 128 Bit lang. Sie werden in Hexadezimaldarstellung angezeigt, wobei acht Gruppen von vier hexadezimalen Ziffern (z.B. 2001:0db8:85a3:0000:0000:8a2e:0370:7334) verwendet werden. Die Länge von 128 Bit ermöglicht 2^{128} Adressen, was in dezimaler Schreibweise $3.4 \cdot 10^{38}$, einer Zahl mit 39 Stellen, entspricht. Auf jeden Quadratmeter der Erdoberfläche kämen etwa $6.7 \cdot 10^{23}$ IPv6-Adressen – das ist ungefähr eine Avogadro-Zahl pro Quadratmeter.

19.2 DNS Lookup

Menschen können sich IP-Adressen nur schwer merken. Daher wurden Domainnamen eingeführt, die leichter zu erinnern sind. Ein Beispiel für einen Domainnamen ist www.google.com. Eine mögliche IP-Adresse für den Domainname von Google ist 172.217.168.68; oft bestehen mehrere IP-Adressen für einen Domainnamen. Für die "Übersetzung" eines Domainname in eine IP-Adresse ist das Domain Name System (DNS) verantwortlich.

19.2.1 Funktionsweise

1. **Anfrage:** Wenn ein Benutzer eine Website besucht, sendet der Browser eine DNS-Anfrage an einen DNS-Server, um die IP-Adresse der gewünschten Domain zu ermitteln.
2. **Auflösung:** Der DNS-Server überprüft seinen Cache auf eine vorhandene Zuordnung. Wenn keine Zuordnung gefunden wird, leitet der Server die Anfrage an andere DNS-Server weiter, bis die IP-Adresse ermittelt wird.
3. **Antwort:** Der DNS-Server sendet die gefundene IP-Adresse zurück an den Browser, der dann eine Verbindung zur Website herstellen kann.

19.2.2 Typen von DNS-Einträgen

- **A-Record:** Verknüpft einen Domainnamen mit einer IPv4-Adresse.
- **AAAA-Record:** Verknüpft einen Domainnamen mit einer IPv6-Adresse.
- **CNAME-Record:** Alias für einen anderen Domainnamen.
- **MX-Record:** Gibt den Mailserver für eine Domain an.

19.2.3 Bedeutung

Der Domainname ist entscheidend für die Benutzerfreundlichkeit des Internets, da er es ermöglicht, Websites über leicht merkbare Namen anstelle von numerischen IP-Adressen zu erreichen. Ohne DNS müssten Benutzer die IP-Adressen aller Websites kennen, die sie besuchen möchten.

20 Network Address Translation (NAT)

20.1 Was ist NAT?

Network Address Translation (NAT) ist ein Verfahren zur automatischen und transparenten Übersetzung von IP-Adressen in Datenpaketen. NAT ermöglicht es, dass mehrere Geräte in einem privaten Netzwerk über eine einzige öffentliche IP-Adresse mit dem Internet kommunizieren können. Diese Technik wurde entwickelt, um die Knappheit von IPv4-Adressen zu bewältigen und gleichzeitig die Sicherheit privater Netzwerke zu erhöhen.

In der Praxis bedeutet dies: Wenn Sie zu Hause mehrere Geräte (Laptop, Smartphone, Smart-TV) mit dem Internet verbinden, teilen sich alle diese Geräte eine einzige öffentliche IP-Adresse, die Ihnen von Ihrem Internetanbieter zugewiesen wurde.

20.2 Funktionsweise von NAT

20.2.1 Grundprinzip

NAT arbeitet auf der Schicht 3 des OSI-Modells und ändert die Adressen im IP-Header von Datenpaketen. Der Prozess läuft folgendermassen ab:

1. **Ausgehende Verbindung:** Ein Gerät im privaten Netzwerk (z.B. 192.168.1.10) sendet eine Anfrage an einen Server im Internet.
2. **Adressübersetzung:** Der NAT-Router ersetzt die private Quell-IP-Adresse durch seine öffentliche IP-Adresse (z.B. 203.0.113.5).
3. **Tabelleneintrag:** Der Router speichert die Zuordnung in einer NAT-Tabelle.
4. **Antwort:** Wenn der Server antwortet, verwendet der Router die NAT-Tabelle, um die Antwort an das richtige Gerät im privaten Netzwerk weiterzuleiten.

20.2.2 NAT-Tabelle

Die NAT-Tabelle ist das Herzstück des Verfahrens. Sie enthält folgende Informationen:

- **Private IP-Adresse** (z.B. 192.168.1.10)
- **Privater Port** (z.B. 54321)
- **Öffentliche IP-Adresse** (z.B. 203.0.113.5)
- **Öffentlicher Port** (z.B. 12345)
- **Ziel-IP-Adresse** (z.B. 93.184.216.34)
- **Ziel-Port** (z.B. 80)

20.3 Private IP-Adressbereiche

Für private Netzwerke sind spezielle IP-Adressbereiche reserviert, die nicht im öffentlichen Internet geroutet werden:

- **10.0.0.0 bis 10.255.255.255** (10.0.0.0/8) – 16'777'216 Adressen
- **172.16.0.0 bis 172.31.255.255** (172.16.0.0/12) – 1'048'576 Adressen
- **192.168.0.0 bis 192.168.255.255** (192.168.0.0/16) – 65'536 Adressen

Diese Bereiche können in privaten Netzwerken beliebig oft wiederverwendet werden, da sie nur lokal gültig sind.

20.4 Arten von NAT

20.4.1 1. Static NAT (One-to-One NAT)

Bei Static NAT wird eine private IP-Adresse fest einer öffentlichen IP-Adresse zugeordnet. Dies wird verwendet, wenn ein Server im privaten Netzwerk dauerhaft aus dem Internet erreichbar sein soll.

Beispiel: - Privat: 192.168.1.100 ↔ Öffentlich: 203.0.113.10

20.4.2 2. Dynamic NAT

Dynamic NAT ordnet private IP-Adressen dynamisch aus einem Pool öffentlicher IP-Adressen zu. Die Zuordnung erfolgt bei Bedarf und wird nach einer gewissen Zeit wieder freigegeben.

20.4.3 3. Port Address Translation (PAT)

20.4.3.1 Was ist ein Port?

Um zu verstehen, wie PAT funktioniert, müssen wir zuerst das Konzept der Ports verstehen. Eine IP-Adresse identifiziert einen Computer im Netzwerk – aber auf einem Computer laufen gleichzeitig viele Programme, die alle Netzwerkverbindungen nutzen möchten.

Analogie: Stellen Sie sich die IP-Adresse als Postadresse eines Hochhauses vor. Die Portnummer entspricht dann der Wohnungsnummer. So wie die Post weiss, in welche Wohnung ein Brief gehört, weiss der Computer durch die Portnummer, an welches Programm die Daten geliefert werden sollen.

Ports sind 16-Bit-Zahlen (0 bis 65'535) und werden in drei Kategorien unterteilt: - **Well-known Ports (0-1023):** Reserviert für Standarddienste - Port 80: HTTP (Webseiten) - Port 443: HTTPS (verschlüsselte Webseiten) - Port 22: SSH (sichere Verbindung) - Port 25: SMTP (E-Mail-Versand)

- **Registered Ports (1024-49'151):** Für registrierte Dienste - **Dynamic/Private Ports (49'152-65'535):** Frei verwendbar

20.4.3.2 Funktionsweise von PAT

PAT, auch NAT-Overload genannt, ist die häufigste Form von NAT. Hier teilen sich viele private IP-Adressen eine einzige öffentliche IP-Adresse, wobei die Unterscheidung über Portnummern erfolgt.

Der Router merkt sich nicht nur, welche private IP-Adresse eine Verbindung aufgebaut hat, sondern auch über welchen Port. Dadurch können mehrere Geräte gleichzeitig über dieselbe öffentliche IP-Adresse kommunizieren.

Rechenbeispiel: Mit 16-Bit-Portnummern stehen theoretisch $2^{16} = 65'536$ Ports zur Verfügung. Abzüglich der reservierten Ports (0-1023) bleiben etwa 64'512 nutzbare Ports für gleichzeitige Verbindungen. In der Praxis bedeutet dies, dass ein Heimrouter theoretisch über 64'000 gleichzeitige Verbindungen verwalten könnte.

20.5 Vorteile und Nachteile

20.5.1 Vorteile

- **IP-Adressen-Einsparung:** Ein Haushalt mit 20 Geräten benötigt nur eine öffentliche IP-Adresse
- **Sicherheit:** Private IP-Adressen sind von aussen nicht direkt erreichbar
- **Flexibilität:** Interne Netzwerkstruktur kann geändert werden, ohne die öffentliche Adresse zu beeinflussen

20.5.2 Nachteile

- **Ende-zu-Ende-Konnektivität:** Direktverbindungen zwischen Geräten werden erschwert
- **Komplexität:** Bestimmte Anwendungen (z.B. VoIP, Online-Gaming) benötigen spezielle Konfigurationen
- **Performance:** Die Adressübersetzung benötigt Rechenleistung und kann zu Verzögerungen führen

20.6 Praktisches Beispiel

Betrachten wir einen typischen Heimrouter:

1. **Ihr Laptop** (192.168.1.15) öffnet die Webseite www.example.com
2. **HTTP-Anfrage:**

- Quelle: 192.168.1.15:45678
- Ziel: 93.184.216.34:80

3. NAT-Router übersetzt:

- Quelle: 85.5.123.45:23456 (öffentliche IP)
- Ziel: 93.184.216.34:80

4. Server antwortet:

- Quelle: 93.184.216.34:80
- Ziel: 85.5.123.45:23456

5. Router übersetzt zurück:

- Quelle: 93.184.216.34:80
- Ziel: 192.168.1.15:45678

20.7 Bedeutung für die Zukunft

Mit der Einführung von IPv6 und seinen 2^{128} möglichen Adressen wird NAT theoretisch überflüssig. In der Praxis wird NAT jedoch weiterhin verwendet werden, da:

- Viele Netzwerke noch auf IPv4 basieren
- NAT zusätzliche Sicherheit bietet
- Die Umstellung auf IPv6 schrittweise erfolgt

NAT bleibt somit eine wichtige Technologie für das moderne Internet, die sowohl die Adressknappheit löst als auch zur Netzwerksicherheit beiträgt.

21 Top Level Domains und ihre Vergabe

21.1 Was sind Top Level Domains?

Top Level Domains (TLDs) sind der letzte Teil eines Domainnamens – der Teil nach dem letzten Punkt. Bei der Adresse `www.example.com` ist “.com” die Top Level Domain. TLDs bilden die oberste Hierarchieebene im Domain Name System (DNS) und sind essentiell für die Organisation des Internets.

Jeder Domainname folgt einer hierarchischen Struktur, die von rechts nach links gelesen wird: - **www.kbw.ch** - TLD: .ch (Schweiz) - Second Level Domain: kbw - Subdomain: www

21.2 Arten von Top Level Domains

21.2.1 1. Generic Top Level Domains (gTLDs)

Die generischen TLDs sind nicht länderspezifisch und können weltweit registriert werden:

- **.com** – Commercial (ursprünglich für Unternehmen)
- **.org** – Organization (für Organisationen)
- **.net** – Network (ursprünglich für Netzerkanbieter)
- **.edu** – Education (nur für akkreditierte Bildungseinrichtungen)
- **.gov** – Government (nur für US-Regierungsstellen)
- **.mil** – Military (nur für US-Militär)

21.2.2 2. Country Code Top Level Domains (ccTLDs)

Jedes Land erhält einen zweistelligen Code nach ISO 3166-1:

- **.ch** – Schweiz (Confoederatio Helvetica)
- **.de** – Deutschland
- **.fr** – Frankreich
- **.uk** – Vereinigtes Königreich
- **.us** – Vereinigte Staaten

Insgesamt existieren 249 ccTLDs für Länder und Territorien.

21.2.3 3. New Generic Top Level Domains (ngTLDs)

Seit 2013 wurden über 1'200 neue TLDs eingeführt:

- **.shop, .blog, .app**
- **.berlin, .paris, .swiss**
- **.google, .apple** (Marken-TLDs)

21.3 Die Vergabestelle: ICANN und IANA

Die **Internet Corporation for Assigned Names and Numbers (ICANN)** ist die zentrale Organisation für die Verwaltung des Domain Name Systems. Sie wurde 1998 gegründet und hat ihren Hauptsitz in Los Angeles, USA.

Die **Internet Assigned Numbers Authority (IANA)**, eine Abteilung von ICANN, verwaltet die Root Zone des DNS. Diese enthält alle TLDs und ihre zugehörigen Nameserver.

21.3.1 Hierarchie der Vergabe

1. **ICANN/IANA** → Verwaltet alle TLDs
2. **Registry** → Betreibt eine spezifische TLD (z.B. SWITCH für .ch)
3. **Registrar** → Verkauft Domains an Endkunden
4. **Registrant** → Der Domaininhaber

21.4 Der Vergabeprozess für neue gTLDs

21.4.1 Phase 1: Bewerbung

Organisationen können sich bei ICANN um neue TLDs bewerben. Die letzte grosse Bewerbungsrunde war 2012:

- **Bewerbungsgebühr:** 185'000 US-Dollar
- **Bewerbungszeitraum:** 4 Monate
- **Eingegangene Bewerbungen:** 1'930

21.4.2 Phase 2: Evaluation

ICANN prüft jede Bewerbung auf: - Technische Kompetenz - Finanzielle Stabilität - Keine Markenrechtsverletzungen - Öffentliches Interesse

21.4.3 Phase 3: Delegation

Nach erfolgreicher Prüfung wird die TLD in die Root Zone eingetragen. Der gesamte Prozess dauert typischerweise 18-24 Monate.

21.5 Kosten und Gebühren

21.5.1 Für Registry-Betreiber

- **Jährliche ICANN-Gebühr:** 25'000 US-Dollar pro TLD
- **Transaktionsgebühr:** 0.25 US-Dollar pro Domainregistrierung
- **Betriebskosten:** Technische Infrastruktur, Personal, Marketing

21.5.2 Für Endkunden (Beispiele)

- **.ch Domain:** CHF 10-20 pro Jahr
- **.com Domain:** CHF 15-25 pro Jahr
- **Premium ngTLDs:** CHF 50-500+ pro Jahr

21.6 Besondere Regelungen

21.6.1 Restricted TLDs

Einige TLDs haben spezielle Registrierungsbedingungen:

- **.edu** – Nur für akkreditierte US-Hochschulen
- **.museum** – Nur für Museen
- **.aero** – Nur für die Luftfahrtindustrie
- **.swiss** – Verbindung zur Schweiz erforderlich

21.6.2 Internationalized Domain Names (IDN)

Seit 2010 sind TLDs in nicht-lateinischen Schriften möglich:

- China: .中国
- Russland: .рф
- Griechenland: .ελ

21.7 Statistische Betrachtung

Stand 2025 gibt es:

- **Aktive TLDs:** ~1'500
- **Registrierte Domains weltweit:** ~370 Millionen
- **Grösste TLD:** .com mit ~160 Millionen Domains
- **Schweizer .ch Domains:** ~2.5 Millionen

Die Verteilung folgt einem Potenzgesetz: Die 10 grössten TLDs beherbergen etwa 75% aller Domains.

21.8 Bedeutung und Ausblick

Top Level Domains sind mehr als technische Bezeichnungen – sie sind digitale Identitäten. Eine .ch-Domain signalisiert Schweizer Herkunft, eine .edu-Domain steht für Bildung. Mit der kontinuierlichen Expansion des Internets wächst auch die Bedeutung einer durchdachten TLD-Struktur.

Die nächste ICANN-Bewerbungsrunde für neue gTLDs ist für 2026 geplant. Experten erwarten weitere 500-1'000 neue TLDs, darunter vermehrt Städte- und Markennamen. Die Herausforderung wird sein, die Balance zwischen Innovation und Übersichtlichkeit zu wahren.

22 Beobachten von Internetverbindungen

Die folgenden Ausführungen basieren auf der Analyse von Netzwerkpaketen, welche mit Wireshark aufgezeichnet wurden. Dafür ist die Installation von Wireshark erforderlich. Die Website von Wireshark stellt dazu den entsprechenden Download zur Verfügung.

22.1 Aufzeichnen von Netzwerkpaketen

Um Netzwerkpakete aufzuzeichnen, wird Wireshark gestartet. Das Startfenster von Wireshark stellt sich folgendermassen dar:

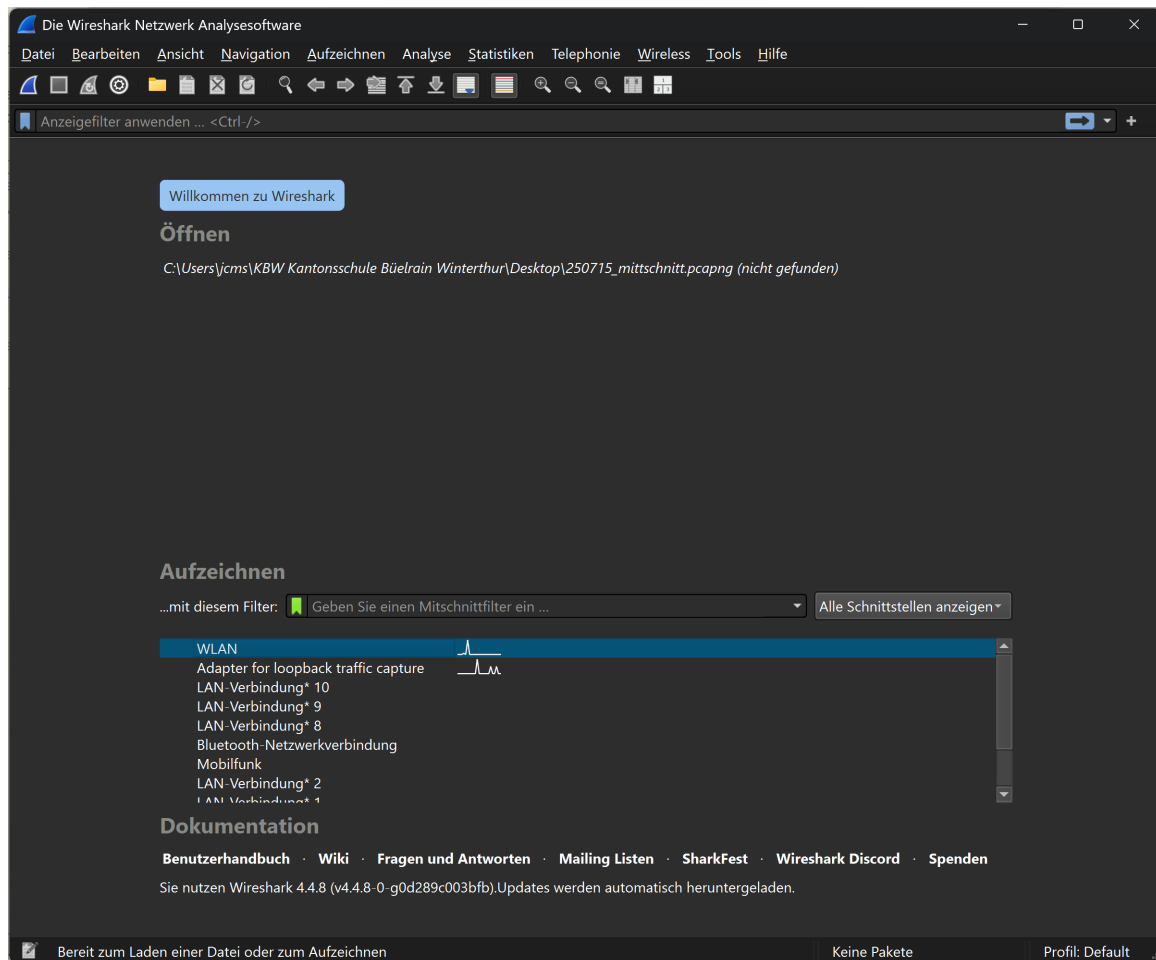


Abbildung 22.1: Wireshark Startfenster

Unter dem Titel “Aufzeichnen” kann der gewünschte Netzwerkadapter ausgewählt und die Aufzeichnung gestartet werden. Die erfassten Pakete werden in Echtzeit angezeigt und können später analysiert werden. In der Schule wird die Verbindung zum Internet per WLAN hergestellt. Entsprechend ist der WLAN-Adapter auszuwählen. Sobald der entsprechende Adapter ausgewählt ist, kann die Aufzeichnung gestartet werden. Gestartet wird die Aufzeichnung durch einen Klick auf das blaue Haiifischflossen-Symbol in der Symbolleiste. Die Aufzeichnung startet umgehend. Angehalten wird die Aufzeichnung mit einem Klick auf das rote Quadrat-Symbol in der Symbolleiste. Die Aufzeichnung kann entweder über das Menü Datei > Speichern, durch einen Klick auf das Dateisymbol oder durch die Tastenkombination **Strg + S** gespeichert werden.

22.2 Beobachten der DNS-Anfragen

22.2.1 Filtern der Aufzeichnung

Um die DNS-Anfragen zu beobachten, wird bei laufender Wireshark Aufzeichnung eine beliebige Website aufgerufen. Dadurch werden die entsprechenden DNS-Anfragen erfasst und können in Wireshark analysiert werden. Nach dem Aufruf der Website kann die Aufzeichnung angehalten und die erfassten Pakete analysiert werden (wenn die Aufzeichnung weiterläuft, bewegen sich die Pakete im Anzeigefenster ständig weiter).

Damit die relevanten Datenpakete angezeigt werden, kann der aufgezeichnete Datenverkehr gefiltert werden. Der Filter wird in der Eingabefeld für "Anzeigefilter" eingegeben.

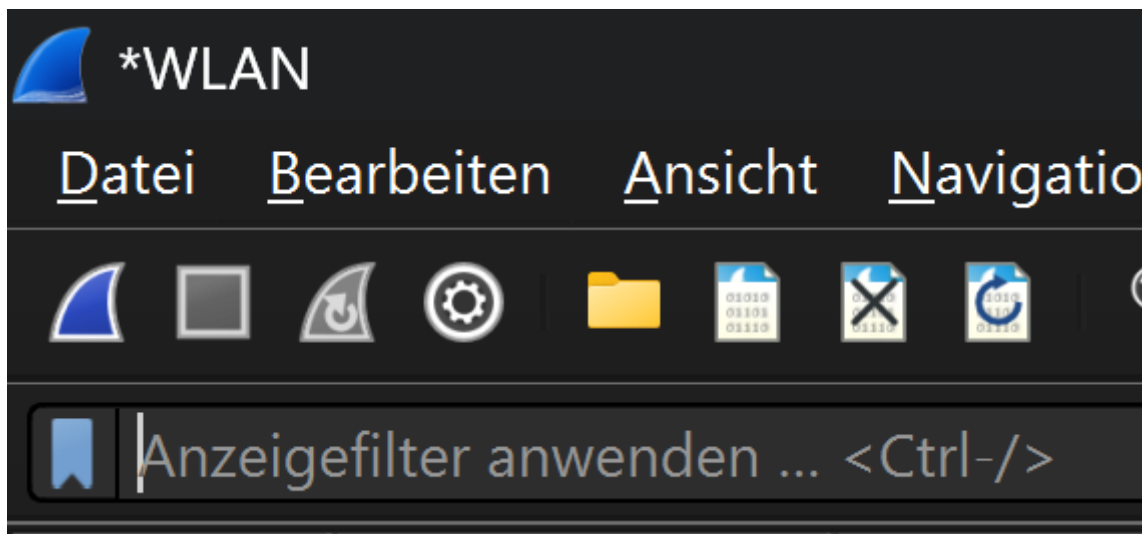


Abbildung 22.2: Wireshark Anzeigefilter

Der entsprechende Filter für DNS-Anfragen zu einer gegebenen Website lautet

```
1 dns.qry.name == "www.example.com"
```

Der angewendete Filterbefehl ist relativ einfach nachzuvollziehen. An erster Stelle steht hier das Protokoll, nach dem gefiltert wird. Weil nach den DNS-Anfragen gefiltert wird, ist das hier `dns`. `dns` alleine wäre bereits ein gültiger Filter. Allerdings werden dann alle DNS-Pakete angezeigt. Der Filter wird daher zu `dns.qry.name` ergänzt. Dabei steht `qry` als Abkürzung für `query` - Anfrage. Die Ergänzung `name` steht für den Domain Name, der Abgefragt wird. `==` ist die logische Verknüpfung, nach der gefiltert wird und bedeutet hier "ist gleich". Zwischen den Anführungszeichen steht der String, nach dem gesucht wird.

Sofern die Seite während der Aufzeichnung genau einmal aufgerufen wurde, wird der Filter zwei Pakete anzeigen: eine DNS-Anfrage und eine DNS-Antwort.

dns.qry.name == "www.deutschegrammophon.com"					
No.	Time	Source	Destination	Protocol	Length Info
697	5.235094	192.168.1.108	192.168.1.1	DNS	86 Standard query 0x1f7a A www.deutschegrammophon.com
723	5.257969	192.168.1.1	192.168.1.108	DNS	102 Standard query response 0x1f7a A www.deutschegrammophon.com A 85.236.46.65

Abbildung 22.3: Gefilterte Wireshark-Pakete

Das Bild zeigt als erstes Paket die DNS-Anfrage für `www.deutschegrammophon.com` und als zweites Paket die entsprechende Antwort.

22.2.2 Analyse der gefilterten Pakete

Für eine genaue Analyse der Kommunikation kann ein einzelnes Paket durch anklicken ausgewählt werden. Dadurch wird das Paket im unteren Bereich von Wireshark detailliert angezeigt und kann genauer untersucht werden.

```

▶ Frame 697: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF_{409C3DA7-
▶ Ethernet II, Src: Intel_e8:b0:93 (04:56:e5:e8:b0:93), Dst: Arcadyan_a6:82:80 (a0:b5:49:a6:82:80)
▶ Internet Protocol Version 4, Src: 192.168.1.108, Dst: 192.168.1.1
▶ User Datagram Protocol, Src Port: 53586, Dst Port: 53
▶ Domain Name System (query)

```

Abbildung 22.4: Inhalt des Ausgewählten Pakets

Dass es sich hier im Bild um die Details des ausgewählten Paketes handelt, ist an der übereinstimmenden Paketnummer zu erkennen. Die Zeilen in der Detailansicht entsprechen den einzelnen Protokoll-Header-Feldern des ausgewählten Paketes. Das bildet auch das TCP/IP Schichtenmodell ab.

Die Detailansicht kann durch einen Klick auf die Dreiecke am Anfang der einzelnen Protokoll-Header-Felder erweitert werden. Dadurch werden weitere Informationen zu den jeweiligen Feldern angezeigt. Hier werden jedoch nur die Zusammenfassungen der Header-Felder erläutert.

Im Beispiel wird als erstes der Inhalt des Headers des Internetlayers erläutert.

```

1
2 Internet Protocol Version 4, Src: 192.168.1.108, Dst: 192.168.1.1

```

In der Zusammenfassung werden die Quell- und Ziel-Adressen des IP-Pakets angezeigt. Im vorliegenden Fall sind das jeweils die Privaten IP-Adressen 192.168.1.108 und 192.168.1.1. 192.168.1.108 ist die Quell-Adresse, erkennbar an der Abkürzung "Src" und 192.168.1.1 die Ziel-Adresse, erkennbar an der Abkürzung "Dst". Beide Geräte befinden sich damit im gleichen LAN. Der Rechner mit der IP-Adresse 192.168.1.1 ist der Router. Dieses Gerät stellt die Internetverbindung her und kann DNS-Anfragen aus seinem Cache beantworten.

Im Header für das User Datagram Protocol (UDP) werden die Quell- und Ziel-Ports angezeigt.

```
1 User Datagram Protocol, Src Port: 53586, Dst Port: 53
```

Der Quellport wurde mit 53586 automatisch und weit oberhalb der sog. “Well-Known Ports” (0-1023) gewählt. Die “Well-Known Ports” sind Ports, die von bestimmten Anwendungen oder Diensten standardmässig verwendet werden. Entsprechend wurde der Zielpport auf 53 gewählt, da dies der standardmässige Port für DNS-Anfragen ist. Eine Liste der “Well-Known Ports” findet sich in der offiziellen IANA-Portdatenbank. Der Quellport ermöglicht es dem Zielsystem, die Antwort an den korrekten Absender zurückzusenden. NAT-Geräte (vgl. [Abschnitt Network Address Translation \(NAT\)](#)) nutzen diese Port-Informationen zusätzlich für die Zuordnung zwischen privaten und öffentlichen Adressen.

Der zuunterst dargestellte Layer in der Detailansicht, beinhaltet die eigentliche Anfrage für die Übersetzung des Domainnamens in eine IP-Adresse.

```
1 Domain Name System (query)
2   Transaction ID: 0x1f7a
3   Flags: 0x0100 Standard query
4   Questions: 1
5   Answer RRs: 0
6   Authority RRs: 0
7   Additional RRs: 0
8   Queries
9       www.deutschegrammophon.com: type A, class IN
10  [Response In: 723]
```

Aus diesem Grund wird dieser Teil der Analyse hier auch aufgefaltet dargestellt.

Unter dem Stichwort Queries wird die gesuchte Adresse `www.deutschegrammophon.com` angezeigt. Das Stichwort `type A` zeigt an, dass es sich hier um eine Anfrage nach einer IPv4-Adresse handelt. IPv4-Adressen werden mit `A` bezeichnet, IPv6 mit `AAAA`. Das letzte Element in dieser Zeile ist die Klasse der Anfrage, in diesem Fall `IN` für das Internet. Obwohl heute fast ausschliesslich das Internet als Netzwerktyp verwendet wird, ist das Feld für die Klasse (`IN`) aus historischen Gründen weiterhin Teil jeder DNS-Anfrage.

Der entsprechende Inhalt der Antwort sieht folgendermassen aus:

```
1 Domain Name System (response)
2   Transaction ID: 0x1f7a
3   Flags: 0x8180 Standard query response, No error
4   Questions: 1
5   Answer RRs: 1
6   Authority RRs: 0
7   Additional RRs: 0
8   Queries
9       www.deutschegrammophon.com: type A, class IN
10  Answers
11      www.deutschegrammophon.com: type A, class IN, addr 85.236.46.65
```

Das Paket wiederholt die Frage und liefert die Antwort des DNS-Servers. Der Domainname `www.deutschegrammophon.com` ist mit der IPv4-Adresse `85.236.46.65` verknüpft.

Damit kann die Verbindung zur Website `www.deutschegrammophon.com` hergestellt werden.

22.3 Beobachtung des Verbindungsaufbaus

Der Verbindungsaufbau zwischen Client (lokaler Rechner) und Server (Rechner im Internet) erfolgt in mehreren Schritten, die im sogenannten “Three-Way Handshake” zusammengefasst werden. Dieser Prozess stellt sicher, dass beide Seiten bereit sind, Daten zu senden und zu empfangen. Die folgende Abbildung zeigt eine schematische Darstellung des “Three-Way Handshake”.

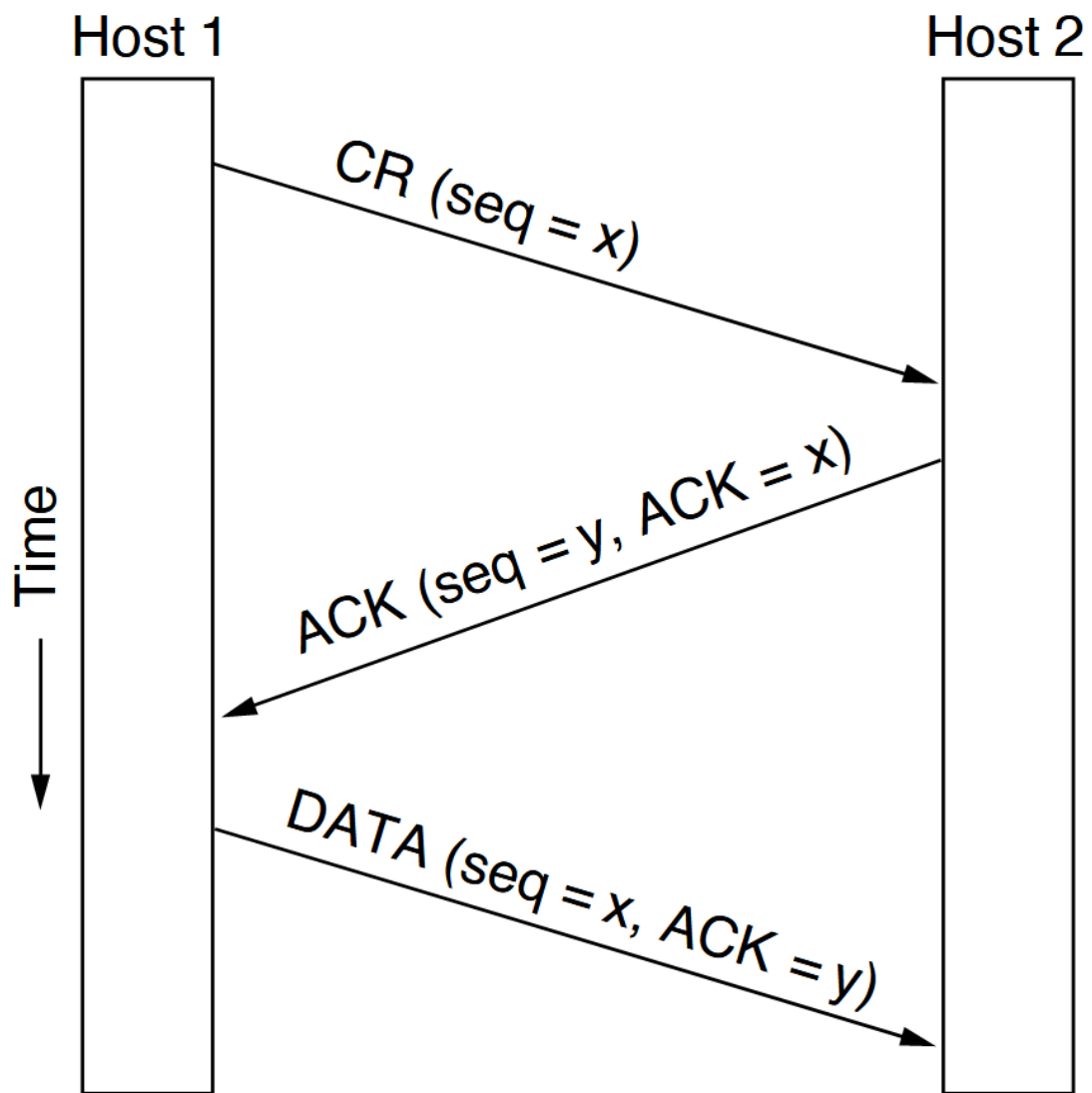


Abbildung 22.5: Schema Three-Way Handshake

Der Client sendet ein SYN-Paket an den Server, um eine Verbindung anzufordern. Dieser antwortet mit einem SYN-ACK-Paket. Das heißt, er bestätigt die Anfrage mit einem ACK und fragt seinerseits mit einem SYN nach, ob der Client (immer noch) bereit ist, die Verbindung aufzubauen. Damit klar ist, dass sich das ACK im SYN-ACK-Paket auf das ursprüngliche SYN-Paket bezieht, werden die einzelnen Pakete mit einer Sequenznummer (Sequence Number) versehen. Das ACK gibt die Sequenznummer des SYN-Paketes plus eins zurück.

Dieser Vorgang kann mit Wireshark beobachtet werden. Dafür braucht es einen kombinierten Wireshark Anzeigefilter. Als Beispiel wird der Verbindungsaufbau zwischen dem lokalen Rechner und der Website von www.deutschegrammophon.com betrachtet. Der erste Teil des Filters soll nur jene Pakete anzeigen, welche mit der IP-Adresse des Servers von www.deutschegrammophon.com (85.236.46.65) kommunizieren. Dieser Filter lautet


```
1 ip.addr == 85.236.46.65
```

Dieser Filter alleine zeigt jedoch noch zu viele Pakete an.

No.	Time	Source	Destination	Protocol	Length	Info
725	5.259654	192.168.1.108	85.236.46.65	TCP	66	27845 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
726	5.260265	192.168.1.108	85.236.46.65	TCP	66	7465 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
747	5.271721	85.236.46.65	192.168.1.108	TCP	62	443 → 7465 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 WS=64
749	5.271721	85.236.46.65	192.168.1.108	TCP	62	443 → 27845 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 WS=64
750	5.271842	192.168.1.108	85.236.46.65	TCP	54	7465 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
751	5.271919	192.168.1.108	85.236.46.65	TCP	54	27845 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
753	5.272544	192.168.1.108	85.236.46.65	TLSv1.3	1933	Client Hello (SNI=www.deutschegrammophon.com)
754	5.273677	192.168.1.108	85.236.46.65	TLSv1.3	1869	Client Hello (SNI=www.deutschegrammophon.com)
768	5.285991	85.236.46.65	192.168.1.108	TCP	58	443 → 7465 [ACK] Seq=1 Ack=1880 Win=40512 Len=0
770	5.286082	85.236.46.65	192.168.1.108	TLSv1.3	283	Server Hello, Change Cipher Spec, Application Data, Application Data
771	5.286552	192.168.1.108	85.236.46.65	TLSv1.3	118	Change Cipher Spec, Application Data
773	5.287014	192.168.1.108	85.236.46.65	TLSv1.3	1825	Application Data
774	5.287943	85.236.46.65	192.168.1.108	TCP	58	443 → 27845 [ACK] Seq=1 Ack=1816 Win=48128 Len=0
775	5.288129	85.236.46.65	192.168.1.108	TLSv1.3	283	Server Hello, Change Cipher Spec, Application Data, Application Data
776	5.288441	192.168.1.108	85.236.46.65	TLSv1.3	118	Change Cipher Spec, Application Data
778	5.299064	85.236.46.65	192.168.1.108	TLSv1.3	133	Application Data
779	5.299851	85.236.46.65	192.168.1.108	TCP	58	443 → 7465 [ACK] Seq=309 Ack=3715 Win=38720 Len=0
780	5.300388	85.236.46.65	192.168.1.108	TLSv1.3	593	Application Data
781	5.300388	85.236.46.65	192.168.1.108	TLSv1.3	133	Application Data
782	5.300417	192.168.1.108	85.236.46.65	TCP	54	7465 → 443 [ACK] Seq=3715 Ack=848 Win=64512 Len=0
784	5.304111	192.168.1.108	85.236.46.65	TLSv1.3	1828	Application Data
793	5.315887	85.236.46.65	192.168.1.108	TCP	58	443 → 7465 [ACK] Seq=848 Ack=5489 Win=36992 Len=0
794	5.316865	85.236.46.65	192.168.1.108	TLSv1.3	583	Application Data
809	5.355585	192.168.1.108	85.236.46.65	TCP	54	27845 → 443 [ACK] Seq=1880 Ack=309 Win=65024 Len=0
810	5.359311	192.168.1.108	85.236.46.65	TCP	54	7465 → 443 [ACK] Seq=5489 Ack=1377 Win=64000 Len=0
1140	6.371999	192.168.1.108	85.236.46.65	TLSv1.3	2010	Application Data
1141	6.388175	85.236.46.65	192.168.1.108	TCP	58	443 → 7465 [ACK] Seq=1377 Ack=7445 Win=37568 Len=0
1142	6.388211	85.236.46.65	192.168.1.108	TLSv1.3	645	Application Data
1143	6.439248	192.168.1.108	85.236.46.65	TCP	54	7465 → 443 [ACK] Seq=7445 Ack=1968 Win=65280 Len=0

Abbildung 22.6: ip.addr Filter

Um die Resultate weiter einzuschränken sollen nur jene Pakete angezeigt werden, welche entweder das SYN-Flag oder das ACK-Flag (oder beides) gesetzt haben. Dies kann mit folgendem Filter erreicht werden:

```
1 ip.addr == 85.236.46.65 and (tcp.flags.syn == 1 or tcp.flags.ack == 1 )
```

Das sind allerdings immer noch zu viele Pakete. Daher sollen nur jene Pakete angezeigt werden, welche die Antworten auf eine SYN-Anfrage sind. Das kann erreicht werden, in dem ein Paket mit dem SYN-Flag mit der rechten Maustaste angeklickt wird und die Option "Follow" > "TCP Stream" ausgewählt wird. Dadurch wird der gesamte TCP-Stream, in dem dieses Paket steht, (die aufeinanderfolgenden Pakete) angezeigt. Der Filter wird automatisch angepasst.

Das folgende Listing zeigt den ganzen Filterbefehl für die Anzeige der Pakete, die zu diesem TCP-Stream gehören:

```
1 ip.addr == 85.236.46.65 and (tcp.flags.syn == 1 or tcp.flags.ack == 1)
  ↪ and !(tcp.stream eq 8)
```

No.	Time	Source	Destination	Protocol	Length	Info
725	5.259654	192.168.1.108	85.236.46.65	TCP	66	27845 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
749	5.271721	85.236.46.65	192.168.1.108	TCP	62	443 → 27845 [SYN, ACK] Seq=0 Ack=1 Win=42340 Len=0 MSS=1460 WS=64
751	5.271919	192.168.1.108	85.236.46.65	TCP	54	27845 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Abbildung 22.7: Three-Way Handshake Pakete

Dass die Pakete die Kommunikationsfolge des Three-Way Handshakes zeigen, ist an den gesetzten Flags zu erkennen. Im ersten Schritt sendet der Client ein SYN-Paket, worauf der Server mit einem SYN-ACK-Paket antwortet. Der Client bestätigt dies mit einem ACK-Paket. Diese drei Schritte sind im Wireshark-Filter sichtbar. Gut zu erkennen sind die jeweiligen Portnummern, die in den TCP-Paketen verwendet werden.

Anschliessend an diesen "Three-Way Handshake" kann der Client mit dem Server kommunizieren.

23 Python Sockets: Ein Einfaches Client-Server Beispiel

In diesem Abschnitt untersuchen wir, wie man eine einfache Client-Server-Anwendung mit Python-Sockets erstellt. Dieses Beispiel wird die grundlegenden Konzepte der Socket-Programmierung demonstrieren, einschliesslich der Herstellung einer Verbindung sowie des Sendens und Empfangens von Daten.

Network Socket

Ein Netzwerk-Socket ist ein Endpunkt innerhalb eines Hosts, der zum Senden und Empfangen von Daten über ein Netzwerk verwendet wird.

23.1 Struktur der Verbindung

Das Beispiel besteht aus einem Server, der auf eingehende Verbindungen wartet, und einem Client, der sich mit dem Server verbindet. Wir werden die Kommunikation als einfachen Chatdienst modellieren.

23.2 Server Code

Sie benötigen keine virtuelle Umgebung. Das Beispiel verwendet nur die Python-Standardbibliothek. Unten ist das funktionierende Codebeispiel für den Socket-Server:

```
1 # socket_server.py
2
3 import socket
4
5 def server_program():
6     # define host name and port
7     host = 'localhost'      # for communication on the local machine
8     port = 5000             # use a port number above 1024
9
10    # create socket
11    server_socket = socket.socket()
12
```

```

13     # bind the socket to the host and port
14     server_socket.bind((host, port))
15
16     # set the server to listen for connections
17     server_socket.listen(2) # start listening
18                             # with a backlog of 2
19                             # (max queued connections)
20
21     # accept new connection from client
22     conn, address = server_socket.accept()
23     print("Connection from: " + str(address))
24
25     while True:
26         # receive up to 1024 bytes per call
27         data = conn.recv(1024).decode()
28         if not data:
29             # if no data is received, break
30             break
31         print("Received from connected client: " + str(data))
32         # prompt the user to enter a message
33         data = input(' -> ')
34         # send data to the client as bytes
35         conn.send(data.encode())
36
37     # close the connection
38     conn.close()
39
40 if __name__ == '__main__':
41     server_program()

```

Um das Skript direkt auszuführen, öffnen Sie ein Terminal im Verzeichnis, in dem sich das Skript befindet, und führen Sie den folgenden Befehl aus:

```

1 python socket_server.py

```

Was geschieht, wenn das Skript ausgeführt wird, wird in den folgenden Abschnitten erläutert.

Der Code besteht aus zwei Hauptteilen: der Funktion `server_program` und dem Block `if __name__ == '__main__':`. Die Funktion `server_program` enthält die Hauptlogik für den Server, während der `if`-Block verwendet wird, um den Servercode auszuführen, wenn das Skript direkt ausgeführt wird.

Wenn Sie das Skript ausführen, importiert Python zuerst das `socket`-Modul.

```
1 import socket
```

Als nächstes wird die Funktion `server_program` aufgerufen. Innerhalb dieser Funktion wird als erstes die Hostadresse und der Portnummer definiert. `localhost` ist eine spezielle Adresse, die auf die lokale Maschine verweist. `localhost` entspricht der IPv4-Adresse `127.0.0.1`. Das bedeutet, dass der Server nur Verbindungen von Clients akzeptiert, die auf derselben Maschine ausgeführt werden. Dies ist eine Simulation eines Netzwerks.

```
1 def server_program():
2     # define host name and port
3     host = 'localhost'      # for communication on the local machine
4     port = 5000             # use a port number above 1024
```

Im nächsten Code-Snippet wird der `server_socket` mit der Funktion `socket.socket()` erstellt, die ein neues Socket-Objekt erzeugt.

```
1 server_socket = socket.socket()
```

Dieses Socket wird verwendet, um auf eingehende Verbindungen von Clients zu hören. Damit die Verbindung funktioniert, muss der `server_socket` mit der Host- und Portnummer über die `bind()`-Methode verbunden werden.

```
1 server_socket.bind((host, port))
```

Als nächstes wird der Server so eingestellt, dass er mit der `listen()`-Methode auf eingehende Verbindungen wartet.

```
1 server_socket.listen(2) # start listening
2                         # with a backlog of 2
3                         # (max queued connections)
```

Der Server erstellt ein backlog von 2 ausstehenden Verbindungen. Der Code akzeptiert jedoch genau eine Client-Verbindung (`accept()` wird einmal aufgerufen). Um mehrere Clients zu verarbeiten, rufen Sie `accept()` in einer Schleife auf und verwenden Sie `Threads` oder `asyncio`. Dieses Limit wird festgelegt, um zu verhindern, dass der Server von zu vielen Verbindungen auf einmal überwältigt wird. Die `accept()`-Methode wird verwendet, um eine neue Verbindung von einem Client zu akzeptieren. Diese Methode gibt ein Tupel mit zwei Elementen zurück: `conn`, ein neues Socket-Objekt für die Kommunikation mit dem Client, und `address`, die Adresse des Clients. Der nächste Abschnitt zeigt, wie eine Nachricht ausgegeben wird, um die Adresse des verbundenen Clients anzuzeigen.

```

1 conn, address = server_socket.accept()
2 print("Connection from: " + str(address))

```

Die while-Schleife ermöglicht es dem Server, auf Daten vom Client zu warten. Die `recv()`-Methode wird verwendet, um Daten vom Client zu empfangen. Jeder `recv()`-Aufruf liest bis zu 1024 Bytes; zusätzliche Daten bleiben im Puffer und können durch nachfolgende Aufrufe gelesen werden. Wenn keine Daten empfangen werden, bricht der Server die Schleife ab und schliesst die Verbindung.

```

1 while True:
2     # receive up to 1024 bytes per call
3     data = conn.recv(1024).decode()
4     if not data:
5         # if no data is received, break
6         break
7

```

Wenn der Server Daten empfängt, gibt er die Daten in der Konsole aus und fordert den Benutzer auf, eine Antwort einzugeben. Die Antwort wird mit der `send()`-Methode an den Client zurückgesendet.

```

1 print("Received from connected client: " + str(data))
2     # prompt the user to enter a message
3     data = input(' -> ')
4     # send data to the client as bytes
5     conn.send(data.encode())

```

23.3 Client Code

Unten ist der funktionierende Code des Client-Beispiels:

```

1 # socket_client.py
2
3 import socket
4
5 def client_program():
6     # define host name and port (of the server to connect to)
7     host = 'localhost' # as both pieces of code are
8                       # running on the same machine
9     port = 5000        # socket server port number
10
11     # create socket
12     client_socket = socket.socket()

```

```

13
14     # connect to the server
15     client_socket.connect((host, port))
16
17     # prompt the user to enter a message
18     message = input(" -> ") # take input
19
20     # message loop
21     while message.lower().strip() != 'bye':
22         # send message to the server as bytes
23         client_socket.send(message.encode())
24
25         # receive response from the server
26         data = client_socket.recv(1024).decode()
27
28         print('Received from server: ' + data)
29
30         # prompt the user to enter a new message
31         message = input(" -> ") # take new input
32
33     # close the connection
34     client_socket.close()
35
36 if __name__ == '__main__':
37     client_program()

```

Um dieses Skript auszuführen, befolgen Sie die gleichen Schritte wie für das Server-Skript. Öffnen Sie ein Terminal im Verzeichnis, in dem sich das Skript befindet, und führen Sie den folgenden Befehl aus:

```

1 python socket_client.py

```

Starten Sie den Server, bevor Sie den Client starten. Wenn der Server nicht läuft, wird `client_socket.connect(...)` einen `ConnectionRefusedError` auslösen.

Wenn Sie das Skript ausführen, importiert Python zuerst das Modul `socket`.

Als nächstes wird die Funktion `client_program` aufgerufen. Innerhalb dieser Funktion wird die erste Aktion die Definition der Hostadresse und der Portnummer. Hier müssen die Hostadresse und die Portnummer mit denen im Server-Skript übereinstimmen.

Nachdem die Host- und Portinformationen definiert sind, erstellt der Client ein Socket-Objekt mit der Funktion `socket.socket()`. Dieses Socket wird verwendet, um eine Verbindung zum Server herzustellen. Die Methode `connect()` wird auf dem Socket-Objekt aufgerufen, um eine Verbindung zum Server herzustellen.

Sobald die Verbindung hergestellt ist, betritt der Client eine Schleife, in der er Nachrichten an den Server senden und Antworten empfangen kann. Der Client fordert den Benutzer auf, eine Nachricht einzugeben, die dann mit der Methode `send()` an den Server gesendet wird. Der Client wartet auch auf eine Antwort vom Server, indem er die Methode `recv()` verwendet.

Wenn der Benutzer `bye` eingibt, verlässt der Client die Schleife und schliesst die Verbindung zum Server.

23.4 Verbindung im LAN

Um die beiden Skripte im lokalen Netzwerk (LAN) zu verwenden, muss in beiden Skripten die IPv4-Adresse des Servers anstelle von `localhost` verwendet werden.

Die IPv4-Adresse des Servers kann durch Ausführen des folgenden Befehls im Terminal des Servers ermittelt werden:

```
1 C:\Users\username>ipconfig
2 Windows IP Configuration
3 ...
4 Wireless LAN adapter Wi-Fi:
5     Connection-specific DNS Suffix  . . . . :
6     IPv4 Address . . . . . : 192.168.1.2
7     Subnet Mask . . . . . : 255.255.255.255
8     Default Gateway . . . . . : 192.168.1.1
9     ...
```

Die Ausgabe des Befehls `ipconfig` zeigt alle Netzwerkadapter des entsprechenden Computers an. Die dargestellte Ausgabe hier im Beispiel wurde gekürzt.

Die IPv4-Adresse des Servers ist in diesem Fall `192.168.1.2`.

Diese ist die Adresse, welche in beiden Skripten anstelle von `localhost` verwendet werden muss. Dann kann innerhalb des lokalen Netzwerks eine Verbindung hergestellt werden.

23.5 Fazit

Die vorgestellten Codebeispiele veranschaulichen, wie Pythons integriertes Socket-Modul (Standardbibliothek) eine unkomplizierte Implementierung von netzwerkbasierenden Anwendungen ermöglicht, ohne externe Abhängigkeiten zu erfordern. Der Fokus des Tutorials auf die Kommunikation über `localhost` bietet eine sichere, kontrollierte Umgebung für Experimente und Lernen, während der bidirektionale Nachrichtenaustausch reale Kommunikationsmuster demonstriert, die in Produktionssystemen zu finden sind.

Zu den wichtigsten Erfolgen dieser Implementierung gehören:

- Erfolgreiche Herstellung von TCP-Socket-Verbindungen zwischen separaten Prozessen

- Implementierung einer nachrichtenbasierten Kommunikation mit ordnungsgemässer Kodierung/Dekodierung
- Demonstration des Verbindungslebenszyklusmanagements von der Herstellung bis zur Beendigung
- Erstellung einer einfachen Befehlszeilenschnittstelle für sowohl Server- als auch Clientanwendungen

Das Wissen, das aus diesem Tutorial gewonnen wurde, dient als solide Grundlage für die Entwicklung anspruchsvollerer netzwerkbasierter Anwendungen. Zukünftige Verbesserungen könnten die Implementierung von Multithreading für die gleichzeitige Verarbeitung mehrerer Clients, die Hinzufügung von Fehlerbehandlungs- und Wiederverbindungslogik oder die Erweiterung des Kommunikationsprotokolls zur Unterstützung strukturierter Datenformate umfassen.

Teil VIII

Datenbanken

24 Einführung in Datenbanken

Die Einführung in Datenbanken basiert auf dem Einführungskapitel aus dem Buch Abraham Silberschatz, Henry F. Korth und S. Sudarshan; Database system concepts; Seventh edition; New York 2020.

24.1 Einführung

Die bisher besprochenen Datenstrukturen Dictionary, Stack, Queue und Binary Search Tree dienen der Bearbeitung von Daten im Arbeitsspeicher. Sie sind daher auf einen beschränkten Umfang von Datensätzen ausgelegt. Ausserdem dienen sie nicht der permanenten Ablage von Daten.

Im Gegensatz dazu dienen Datenbanken der dauerhaften Ablage grosser Datensätze. Darüber hinaus sollen sie die effiziente Verfügbarkeit und die Integrität der Daten sicherstellen.

24.2 Charakteristika von Datenbanken

Ein wichtiges Merkmal von Datenbanken ist es, dass die gespeicherten Daten nur einmal abgelegt werden. Damit kann verhindert werden, dass mehrfach abgespeicherte Daten (redundante Daten) lediglich an einer Stelle modifiziert werden und damit Widersprüche entstehen. Der Entwurf von Datenbanken muss dem Rechnung tragen. Ein Hilfsmittel für den Entwurf von Datenbanken ist das ER-Diagramm (Entity-Relationship-Diagramm). Das ER-Diagramm ist eine grafische Darstellung der Datenbankstruktur. Es zeigt die Entitäten (durch die Datenbank modellierte Dinge der realen Welt), die in der Datenbank gespeichert werden, mit ihren Attributen (Eigenschaften der modellierten Dinge) sowie die Beziehungen (Relationship) zwischen den Entitäten.

Um einen Eintrag in der Datenbank eindeutig identifizieren zu können, wird jedem Eintrag ein Primärschlüssel zugeordnet. Der Primärschlüssel ist ein Attribut oder eine Kombination von Attributen, die den Eintrag eindeutig identifiziert.

In den Beziehungen werden die Primärschlüssel der Entitäten als Fremdschlüssel verwendet. Ein Fremdschlüssel ist ein Attribut oder eine Kombination von Attributen, die auf den Primärschlüssel einer anderen Entität verweisen. So kann eine Beziehung zwischen zwei Entitäten hergestellt werden.

Die untenstehende Graphik zeigt eine Skizze eines ER-Diagramms, in welchem die Beziehungen zwischen Schülern, Klassen und Lehrern dargestellt wird. Ausserdem wird gezeigt, welche Attribute als Primär- bzw. Fremdschlüssel verwendet werden.

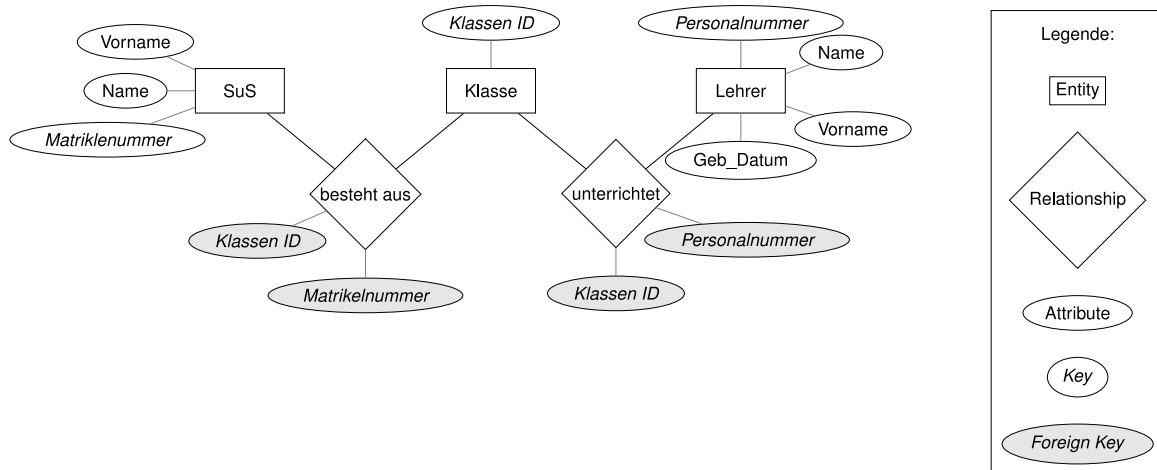


Abbildung 24.1: ER-Diagramm

Jede Datenbank ist immer eine Vereinfachung der Wirklichkeit und daher immer unvollständig bzw. erweiterbar. Das obige ER-Diagramm kann daher um eine zusätzliche Entität Fach erweitert werden und sieht dann folgendermassen aus:

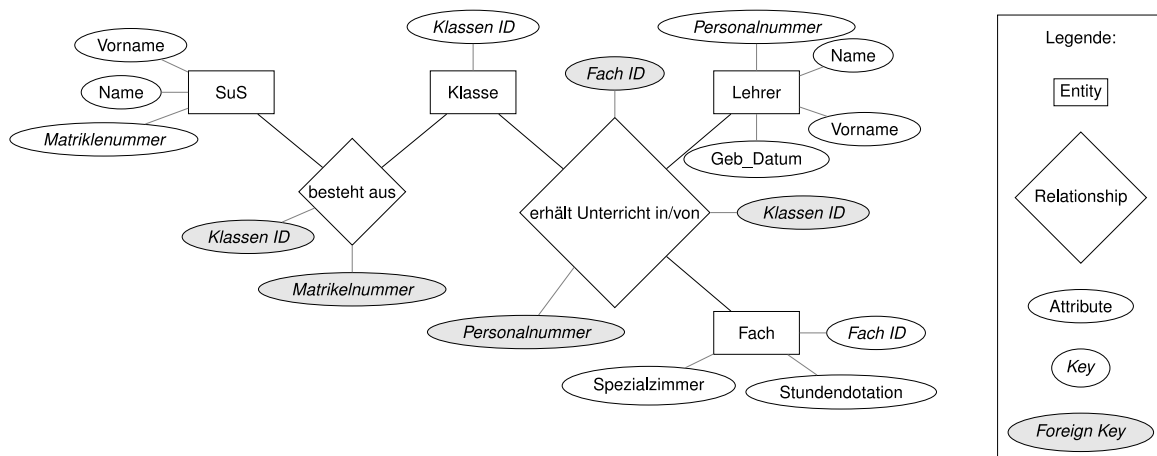


Abbildung 24.2: ER-Diagramm

Diese Grafiken können in Datenbanktabellen übersetzt werden. In den Tabellen werden die Primärschlüssel unterstrichen. Die Primärschlüssel sind in der Regel die ersten Spalten der Tabellen. Für das Beispiel werden Tabellen für die Entitäten Klasse, Fach und Lehrer erstellt.

Die einfachste Tabelle ist die Tabelle Klasse. Sie enthält lediglich ein Attribut (den Primärschlüssel).

Klasse
<u>Klassen ID</u>
aW_24-28
bW_24-28
cW_24-28
dP_24-28
eW_24-28
fP_24-28

Abbildung 24.3: Klasse

Etwas umfangreicher sind die Tabellen Fach und Lehrer. Sie enthalten jeweils drei bzw. vier Attribute.

Fach		
<u>Fach ID</u>	Stundendotation	Spezialzimmer
Deutsch	4	—
Französisch	4	—
Englisch	4	—
Mathe	4	—
Biologie	2	Praktikumszimmer B
Physik	2	Praktikumszimmer P
Chemie	2	Chemielabor
Geographie	2	—
Geschichte	2	—
Informatik	2	—
WR	4	—
PPP	4	—
BG	2	Zeichensaal
Musik	2	—
Sport	3	Turnhalle

Abbildung 24.4: Fach

Lehrer			
<u>Personalnummer</u>	Name	Vorname	Geburtsdatum
0001	Schiller	Friedrich	10.11.1759
0002	Balzac	Honoré de	20.05.1799
0003	Sand	George	01.07.1804
0004	Gauss	Johann Carl Friedrich	30.04.1777
0005	Darwin	Charles Robert	12.02.1809
0006	Curie	Marie	07.11.1867
0007	Friedman	Milton	31.07.1912
0008	Redlich	Oswald	17.09.1858
0009	Dufour	Guillaume-Henri	15.09.1787
0010	Kollwitz	Käthe	08.07.1867
0011	Callas	Maria	02.12.1923
0012	Lovlace	Ada	10.12.1815
0013	Piaget	Jean	09.08.1896
0014	Montessori	Maria	31.08.1870

Abbildung 24.5: Lehrer

Im Folgenden Abschnitt sollen die Daten aus den Tabellen mit Hilfe von SQL Statements abgefragt werden.

25 Abfrage von Daten

Datenbanken werden mit einer spezifischen Datenbanksprache angesprochen. Im Gegensatz zur bisher im Unterricht verwendeten Programmiersprache Python ist die Datenbanksprache SQL (Structured Query Language) eine deklarative Sprache. In Python werden die Befehle grundsätzlich der Reihe nach abgearbeitet. In SQL wird das gewünschte Resultat beschrieben. Wie diese Beschreibung abgearbeitet wird, ist in den Grundlagen der Datenbank programmiert.

25.1 Grundstruktur einer SQL Abfrage

Die Grundstruktur einer SQL Abfrage ist im untenstehenden Code Snippet dargestellt.

```
1 SELECT <Spalten>
2 FROM <Tabelle>
3 WHERE <Bedingung>;
```

Das Schlüsselwort SELECT gibt an, welche Spalte(n) aus der Tabelle ausgegeben werden soll(en). Das Schlüsselwort FROM gibt an, aus welcher Tabelle die Daten ausgelesen werden. Das Schlüsselwort WHERE gibt die Bedingung an, die erfüllt sein muss, damit die Daten angezeigt werden. Dass die Schlüsselwörter in Grossbuchstaben geschrieben werden, ist technisch nicht nötig, entspricht aber der Konvention. Die Abfrage wird mit einem Semikolon abgeschlossen.

25.1.1 Einfache Abfrage

In einem ersten Beispiel sollen alle Vornamen aller Lehrer aus der Tabelle Lehrer aus dem vergangenen Abschnitt angezeigt werden:

```
1 SELECT Vorname
2 FROM Lehrer;
```

In diesem Beispiel wurde auf die Formulierung einer Bedingung verzichtet. Wenn die Ausgabe zusätzlich eine Bedingung erfüllen soll, wird diese mit dem Schlüsselwort WHERE angegeben. Im folgenden Beispiel sollen nur die Vornamen der Lehrer angezeigt werden, die vor dem Jahr 1800 geboren sind.

25.1.2 Abfrage mit Bedingung

```
1 SELECT Vorname
2 FROM Lehrer
3 WHERE Geburtsdatum < '1800-01-01';
```

Diese Abfrage führt zu folgendem Ergebnis:

Vorname
Friedrich
Honore de
Johann Carl Friedrich
Guillaume-Henri

25.1.3 Sortierung der Ausgabe

Falls die Ausgabe nicht nur die Vornamen, sondern auch die Nachnamen und das Geburtsdatum enthalten soll und die Ausgabe nach dem Geburtsdatum aufsteigend sortiert werden soll, wird die Abfrage entsprechend angepasst:

```
1 SELECT Name, Vorname, Geburtsdatum
2 FROM Lehrer
3 WHERE Geburtsdatum < '1800-01-01'
4 ORDER BY Geburtsdatum;
```

Diese Abfrage führt zu folgendem Ergebnis:

Name	Vorname	Geburtsdatum
Schiller	Friedrich	10.11.1759
Gauss	Johann Carl Friedrich	30.04.1777
Dufour	Guillaume-Henri	15.09.1787
Balzac	Honoré de	20.05.1799

Es können dem Schlüsselwort SELECT mehrere Spalten übergeben werden. Zusätzlich wurde in der Anfrage das Schlüsselwort ORDER BY verwendet. Mit diesem kann angegeben werden, nach welchem Kriterium die Ausgabe sortiert werden soll. Standardmässig wird aufsteigend sortiert. Mit dem Schlüsselwort DESC kann die Sortierung absteigend erfolgen. Die Abfrage sieht dann folgendermassen aus:


```

1 SELECT Name, Vorname, Geburtsdatum
2 FROM Lehrer
3 WHERE Geburtsdatum < '1800-01-01'
4 ORDER BY Geburtsdatum DESC;

```

Die Sortierreihenfolge wird hinter das Kriterium geschrieben. Wenn nach mehreren Kriterien sortiert werden soll, werden die zusätzlichen Kriterien mit einem Komma an das erste Kriterium angehängt.

25.2 Abfrage aus mehreren Tabellen

Interessanter, als die Abfrage von Daten aus einer einzigen Tabelle, ist die Abfrage aus mehreren Tabellen. So ist es im Beispiel möglich, Abzufragen, wer Deutsch unterrichtet. Aus diesem Grund wurde die Tabelle erhält Unterricht in/von angelegt.

erhält Unterricht in/von		
<u>Fach ID</u>	<u>Klassen ID</u>	<u>Personalnummer</u>
Deutsch	aW_24-28	0001
Deutsch	bW_24-28	0001
Deutsch	cW_24-28	0001
Deutsch	dP_24-28	0001
Deutsch	eW_24-28	0001
Deutsch	fP_24-28	0001
Französisch	aW_24-28	0002
...	...	...

Abbildung 25.1: erhält Unterricht in/von

Um abzufragen, wer Deutsch unterrichtet, müssen die Daten aus den Tabellen Lehrer, Fach und erhält Unterricht in/von zusammengeführt werden. Dies geschieht mit dem Schlüsselwort JOIN. Das Schlüsselwort JOIN kann unterschiedlich verwendet werden. Im vorliegenden Beispiel wird die Variante INNER JOIN verwendet.

```

1 SELECT DISTINCT l.Name, l.Vorname
2 FROM Lehrer AS l
3 INNER JOIN erhält_Unterricht_in AS u ON l.Personalnummer =
   ↪ u.Personalnummer
4 WHERE u.Fach_ID = 'Deutsch';

```

Das Resultat dieser Abfrage sieht wie folgt aus:

Name	Vorname
Schiller	Friedrich

In Ergänzung zu den bisherigen Abfragen, kommt neu das Schlüsselwort `DISTINCT` zum Einsatz. Dieses bewirkt, dass Daten, die mehrfach vorkommen, nur einmal ausgegeben werden. In diesem Beispiel wäre dies nicht nötig, da es nur einen Lehrer gibt, der Deutsch unterrichtet.

Unter dem Schlüsselwort `FROM` wird die Tabelle `Lehrer` mit dem Alias `l` angegeben. Der Alias wird verwendet, um die Abfrage leserlicher zu machen. Wenn mehrere Tabellen abgefragt werden, muss jede Spalte, die ausgegeben werden soll, mit der Tabelle, aus der sie stammt, angegeben werden. Mit dem Alias kann dies abgekürzt werden. Das Schlüsselwort `AS` für den Alias ist nicht nötig, dient aber der besseren Lesbarkeit.

Mit dem Schlüsselwort `INNER JOIN` werden die Datensätze aus den beiden Tabellen `Lehrer` und `erhält_Unterricht_in` basierend auf übereinstimmenden Werten in der Spalte `Personalnummer` miteinander verbunden. Dabei entsteht eine neue Ergebnismenge, die alle Spalten beider Tabellen enthält, jedoch nur für diejenigen Zeilen, bei denen die `Personalnummer` in beiden Tabellen übereinstimmt.

Aus dieser Schnittmenge werden aus der Tabelle `erhält_Unterricht_in` von der Lehrer ausgewählt, die Deutsch unterrichten. Dies geschieht mit dem Schlüsselwort `WHERE` und dem Kriterium `u.Fach_ID = 'Deutsch'`.

Die Abfrage, wer die Klasse `fP_24-28` in `PPP` unterrichtet, sieht wie folgt aus:

```

1 SELECT l.Name, l.Vorname
2 FROM Lehrer AS l
3 INNER JOIN erhält_Unterricht_in AS u ON l.Personalnummer =
   ↪ u.Personalnummer
4 WHERE u.Fach_ID = 'PPP'
5 AND u.Klassen_ID = 'fP_24-28';

```

Die Abfrage gibt folgendes Resultat zurück:

Name	Vorname
Piaget	Jean

Gegenüber der Abfrage, wer Deutsch unterrichtet, wurde mit dem Schlüsselwort `AND` die zusätzliche Bedingung `u.Klassen_ID = 'fP_24-28'` hinzugefügt.

25.3 Ausblick

Der nächste Abschnitt dient dazu, SQL zu üben. Als Übungsplattform wird `SQL Island` genutzt. Diese Plattform ist unter sql-island.informatik.uni-kl.de zu finden.

26 Lernziele für die Prüfung vom 21. Mai 2025

Ich erwarte, dass Sie in der Lage sind

- die grundsätzlichen Unterschiede zwischen Python und SQL zu erklären;
- die Aufgaben einer Datenbank aufzuzählen;
- die Elemente einer grundlegenden SQL Abfrage aufzuzählen sowie
- eine SQL Abfrage in umgangssprachliches Deutsch zu übersetzen.

Für Fragen stehe ich Ihnen im Kanal *Allgemein* zur Verfügung.

Teil IX

Datenstrukturen und Codierung

27 Datenstrukturen Stack und Queue

Zwei wichtige Datenstrukturen zur Lösung alltäglicher Probleme sind der Stack (Stapelspeicher) und Queue (Warteschlange). In beiden Datenstrukturen können mehrere Werte zwischengespeichert werden. Die beiden Speicherformen sollen hier kurz erläutert werden.

In einem Stack werden die gespeicherten Werte *gestapelt*. Wie in einem Stapel in der realen Welt, wird der letzte gespeicherte Wert zuoberst auf den Stapel gelegt. Ebenso wie in einem realen Stapel, können die gespeicherten Werte nur in umgekehrter Reihenfolge zu ihrer Speicherung wieder abgerufen werden (last in - first out; LIFO). Mit Bezug auf Abbildung 27.1 bedeutet dies, dass der letzte Brotlaib, der auf den Stapel gelegt wurde, auch der erste ist, der wieder vom Stapel weggenommen werden kann.

Ein Stack kann dazu verwendet werden, um die Verarbeitungsreihenfolge von Rechenoperationen in einem Programm abzubilden.

In einer Queue werden die gespeicherten Werte in einer Warteschlange *eingereiht*. Wie in einer Warteschlange in der realen Welt (vgl. Abbildung 27.2), wird jeder neu gespeicherte Wert hinten eingereiht. Beim Abrufen der gespeicherten Werte wird der zuerst gespeicherte Wert zuerst verarbeitet (first in - first out; FIFO).

Eine Queue kann dazu verwendet werden, eine Warteschlange abzubilden wie sie in Netzwerken für die Übermittlung von Datenpaketen gebraucht wird.

Für die Übungen finden Sie hier eine [Klasse Node](#) und eine [Klasse Linked List](#) zum Download.

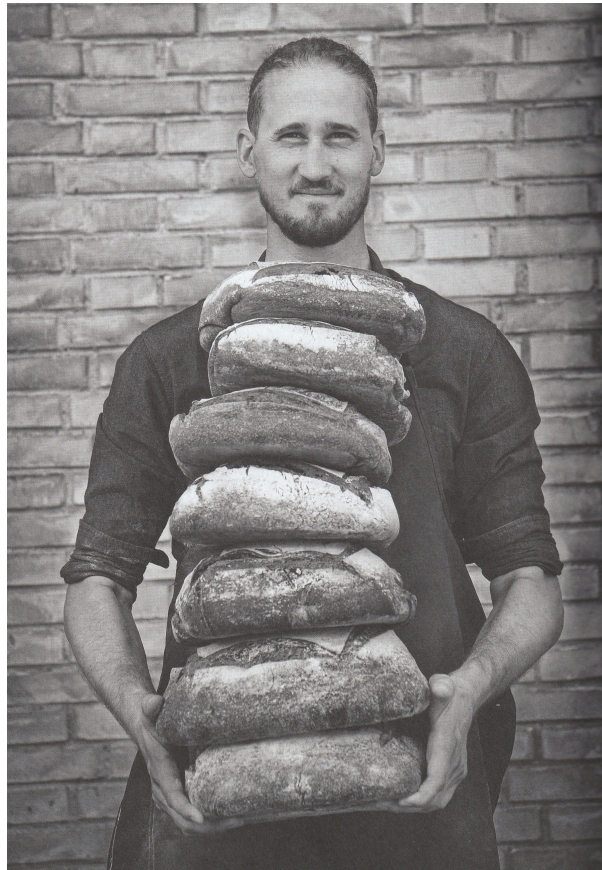


Abbildung 27.1: Martin Mayer mit einem Stapel Brot (Quelle: Sasa Noël und Heike Grein, Brothandwerk, Aarau und München, 2021, Seite 50.)



Abbildung 27.2: Dole Queue Great Depression (Quelle: <https://view.genially.com/609aac10d34c960d5992809a/interactive-content-great-depression-breakout>, besucht am 22.01.15).

28 Queues in Python

Queues sind Datenstrukturen, welche Daten speichern und grundsätzlich in der Reihenfolge, in der sie abgespeichert worden sind, wieder zurückgeben (First In - First Out, FIFO).

Eine Queue funktioniert damit wie eine Warteschlange an einer Kasse: Wenn eine neue Person ankommt, stellt sie sich hinten an. Wenn sie die Kasse erreicht (nachdem alle, die vorher angestanden sind, bedient worden sind), verlässt sie die Schlange wieder.

Eine Queue ist eine derart fundamentale Datenstruktur, dass Python sie als [Library](#) zur Verfügung stellt.

Um zu verstehen, wie eine Queue funktioniert, geht es im folgenden darum, eine eigene Klasse Queue in Python zu implementieren.

Zu Beginn ist zu überlegen, welche Eigenschaften, die Queue aufweisen muss. Sie muss Daten abspeichern und diese in der gleichen Reihenfolge wieder ausgeben können. Wir brauchen also eine Struktur für die Daten und eine Struktur, welche die Reihenfolge der Speicherung festhält. Die Struktur, welche die Reihenfolge festhält, muss ausserdem in der Lage sein, neue Daten abzuspeichern und bereits abgespeicherte Daten wieder zurückzugeben. Diese Anforderungen können mit Hilfe bereits programmierter Klassen umgesetzt werden. Um die Daten abzuspeichern, können wir Nodes verwenden und für die Struktur zum Erhalt der Reihenfolge die Linked List.

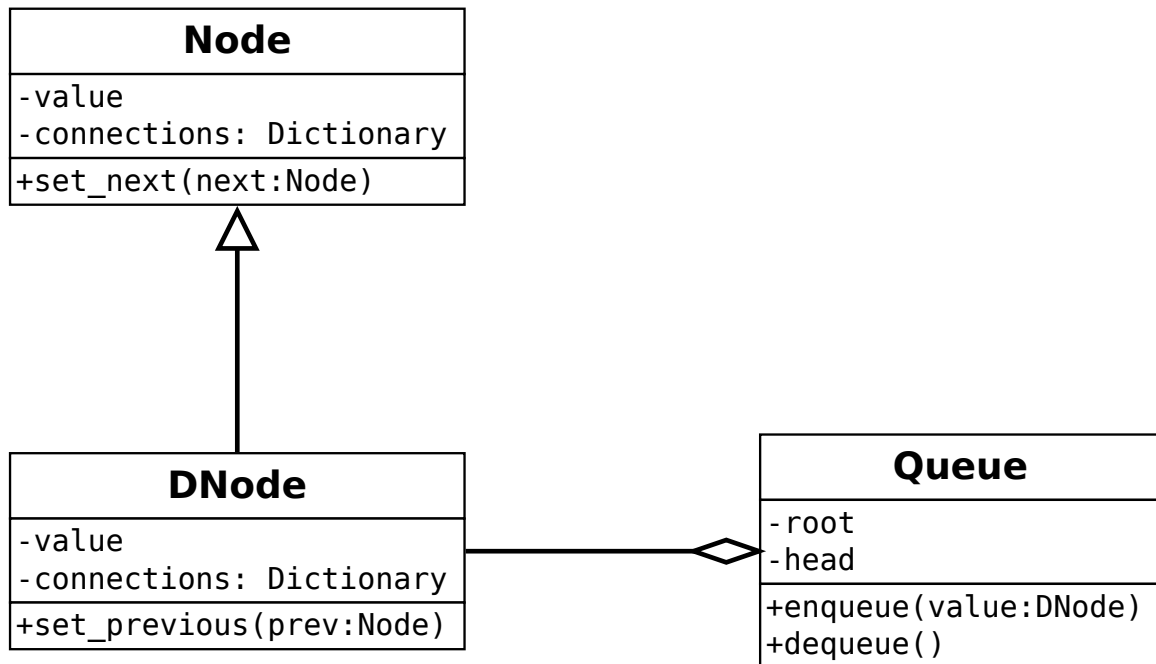
Linked List

Die Linked List ist eine Datenstruktur, die aus einer Anzahl von Nodes besteht. Jeder Node hat einen Bezug auf den nächsten Node in der Liste. Damit ist es möglich, die Nodes in einer bestimmten Reihenfolge abzuspeichern.

Im Node selber gibt es ein Datenfeld `key` für die Bezeichnung des Nodes, eines `value` für die Daten, die gespeichert werden sollen sowie ein solches für den Verweis auf den nächsten Node. Die Datenstruktur Linked List stellt in erster Linie den Verweis auf den zuletzt eingefügten Node zur Verfügung.

In der aktuellen Implementation der Linked List gibt es nur einen Positionsbezug auf das letzte eingefügte Element (`self.root`). Damit die Linked List als Queue verwendet werden kann, muss auch ein Bezug auf das erste eingefügte Element angelegt werden (`self.head`). Für das Erste überhaupt in die Datenstruktur eingefügte Element ist dies kein Problem. Wenn aber weitere Elemente eingefügt oder entfernt werden, dann muss in den einzelnen Nodes nicht nur ein Bezug auf das folgende Element (`self.connections['next']`) sondern auch einer auf das Vorangehende Element (`self.connections['previous']`). Entsprechend müssen die beiden Klassen Node und Linked List angepasst werden.

Das kann umgesetzt werden, indem basierend auf den bereits existierenden Klassen abgeleitete Klassen implementiert werden. Als UML-Klassendiagramm sieht das folgendermassen aus:

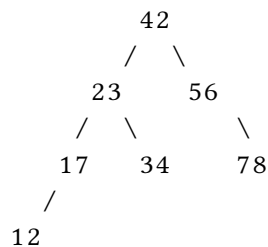


Für die Umsetzung der obigen Ausführungen stehen hier zwei Module ([linked_list.py](#) und [nodes.py](#)) zur Verfügung.

29 Binary Search Tree

Ein binärer Suchbaum (Binary Search Tree, BST) ist eine Datenstruktur, die es ermöglicht, Daten in einer hierarchischen Struktur zu speichern. Jeder Knoten im Baum hat maximal zwei Kinder, wobei das linke Kind kleiner und das rechte Kind grösser als der Knoten selbst ist. Dies ermöglicht das effiziente Suchen, Einfügen und Löschen von Elementen.

Sollen beispielsweise die Zahlen 42, 23, 17, 34, 56, 78 und 12 der Reihe nach in einen binären Suchbaum eingefügt werden, geschieht dies wie folgt:



Um einen Knoten zu löschen, gibt es drei Fälle zu beachten: 1. Der Knoten ist ein Blatt (keine Kinder): Der Knoten kann einfach entfernt werden. 2. Der Knoten hat ein Kind: Das Kind ersetzt den Knoten. 3. Der Knoten hat zwei Kinder: Der Knoten wird durch den kleinsten Knoten im rechten Teilbaum ersetzt.

In den folgenden Abschnitten findet sich eine Mögliche Implementierung eines binären Suchbaums in Python.

29.1 Klasse BSTNode

```
1 class BSTNode:
2     def __init__(self, key, value=None):
3         self.key = key
4         self.value = value
5         self.parent = None
6         self.left = None
7         self.right = None
8
9     def __str__(self):
10        key = str(self.key)
```

```

11     parent = 'None' if self.parent is None else
    ↪ str(self.parent.key)
12     left = 'None' if self.left is None else str(self.left.key)
13     right = 'None' if self.right is None else str(self.right.key)
14     s = (
15         f'\tParent = {parent}\n'
16         f'\tKey = {key}\n'
17         f'Left = {left}\tRight = {right}'
18     )
19     return s

```

29.2 Klasse BST

```

1 class BST:
2     def __init__(self, key=None, value=None):
3         if key is None:
4             self.root = None
5         else:
6             node = BSTNode(key, value)
7             self.root = node
8
9     def insert(self, key, value=None, root=None):
10        node = BSTNode(key, value)
11        if self.root is None:
12            self.root = node
13            return
14
15        if root is None:
16            root = self.root
17
18        if key < root.key and root.left is None:
19            root.left = node
20            node.parent = root
21            return
22
23        if key < root.key:
24            root = root.left
25            self.insert(key, value, root)
26
27
28        if key > root.key and root.right is None:
29            root.right = node
30            node.parent = root

```

```

31         return
32
33     if key > root.key:
34         root = root.right
35         self.insert(key, value, root)
36
37     def min(self, bst=None):
38         if bst is None:
39             minimum = self.root
40         else:
41             minimum = bst.root
42
43         while minimum.left is not None:
44             minimum = minimum.left
45
46         return minimum
47
48     def max(self, bst=None):
49         if bst is None:
50             maximum = self.root
51         else:
52             maximum = bst.root
53
54         while maximum.right is not None:
55             maximum = maximum.right
56
57         return maximum
58
59     def search(self, key, node=None):
60         # If initial call or we've hit None in recursion
61         if node is None:
62             if self.root is None: # Empty tree
63                 return -1
64             node = self.root
65
66         # Found the key
67         if key == node.key:
68             return node
69
70         # Key doesn't exist in this path
71         if key < node.key:
72             if node.left is None:
73                 return -1
74             return self.search(key, node.left)
75         else: # key > node.key
76             if node.right is None:

```

```

77         return -1
78         return self.search(key, node.right)
79
80     def delete(self, key):
81         # Find the node to delete
82         node = self.search(key)
83
84         # If node not found, return
85         if node == -1:
86             return
87
88         self._delete_node(node)
89
90     def _delete_node(self, node):
91         # Case 1: Node has no children (leaf node)
92         if node.left is None and node.right is None:
93             if node == self.root:
94                 self.root = None
95             else:
96                 if node.parent.left == node:
97                     node.parent.left = None
98                 else:
99                     node.parent.right = None
100
101         # Case 2: Node has only one child
102         elif node.left is None: # Has only right child
103             if node == self.root:
104                 self.root = node.right
105                 node.right.parent = None
106             else:
107                 if node.parent.left == node:
108                     node.parent.left = node.right
109                 else:
110                     node.parent.right = node.right
111                 node.right.parent = node.parent
112
113         elif node.right is None: # Has only left child
114             if node == self.root:
115                 self.root = node.left
116                 node.left.parent = None
117             else:
118                 if node.parent.left == node:
119                     node.parent.left = node.left
120                 else:
121                     node.parent.right = node.left
122                 node.left.parent = node.parent

```

```

123
124     # Case 3: Node has two children
125     else:
126         # Find successor (smallest node in right subtree)
127         successor = None
128         current = node.right
129
130         while current.left is not None:
131             current = current.left
132
133         successor = current
134
135         # Copy successor's key and value to the node
136         node.key = successor.key
137         node.value = successor.value
138
139         # Delete the successor (which has at most one right child)
140         self._delete_node(successor)
141
142     def iterate(self, node=None, result=None):
143         # Initialize result list on first call
144         if result is None:
145             result = []
146
147         # Use root if no starting node provided
148         if node is None:
149             if self.root is None: # Empty tree
150                 return result
151             node = self.root
152
153         # In-order traversal: left -> current -> right
154         if node.left is not None:
155             self.iterate(node.left, result)
156
157         result.append(node)
158
159         if node.right is not None:
160             self.iterate(node.right, result)
161
162         return result

```

30 Das Binärsystem

Wir sind es gewohnt uns im Dezimalsystem zu bewegen. Das bedeutet, dass wir Zahlen in der Basis 10 darstellen. Wir verwenden dazu 10 verschiedene Ziffern, nämlich die Ziffern 0 bis 9.

Um Zahlen darzustellen, welche grösser sind als 9 darzustellen, verwenden wir die Zehnerpotenzen. Das heisst, dass die Ziffern von rechts nach links gelesen, die Zehnerpotenzen von 0 an aufsteigend sind. Die Zahl 123 entspricht also der Rechnung $1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0$.

Das Binärsystem funktioniert nach dem gleichen Prinzip, nur dass wir nur die Ziffern 0 und 1 verwenden. Das bedeutet, dass wir Zahlen in der Basis 2 darstellen. Sobald eine Zahl grösser als 1 dargestellt werden soll, schreiben wir die Zahl als Summe von Zweierpotenzen. Die Zahl 101 entspricht also der Rechnung $1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$.

30.1 Umrechnung von Dezimalzahlen in Binärzahlen

Um eine Dezimalzahl in eine Binärzahl umzurechnen, gibt es eine systematische Methode, die auf wiederholter Division durch 2 basiert:

1. Teile die Dezimalzahl durch 2 und notiere den Rest (0 oder 1).
2. Teile den Quotienten (Ergebnis der Division) erneut durch 2 und notiere wieder den Rest.
3. Fahre damit fort, bis der Quotient 0 ist.
4. Die Binärzahl wird nun gebildet, indem man die Reste von unten nach oben (vom letzten zum ersten) liest.

30.1.1 Beispiel: Umrechnung von 42 ins Binärsystem

Lassen Sie uns die Dezimalzahl 42 in eine Binärzahl umrechnen:

Division	Rechnung	Quotient	Rest
$42 \div 2$	$42 = 21 \cdot 2 + 0$	21	0
$21 \div 2$	$21 = 10 \cdot 2 + 1$	10	1
$10 \div 2$	$10 = 5 \cdot 2 + 0$	5	0
$5 \div 2$	$5 = 2 \cdot 2 + 1$	2	1
$2 \div 2$	$2 = 1 \cdot 2 + 0$	1	0
$1 \div 2$	$1 = 0 \cdot 2 + 1$	0	1

Nun lesen wir die Reste von unten nach oben: 101010

Also ist 42 im Dezimalsystem gleich 101010 im Binärsystem.

30.1.2 Überprüfung:

$$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 0 + 8 + 0 + 2 + 0 = 42$$

Eine alternative Methode zur Umrechnung ist die Verwendung der Zweierpotenzen. Man zerlegt die Dezimalzahl in eine Summe von Zweierpotenzen und schreibt dann eine 1 an den Stellen der verwendeten Potenzen und eine 0 an allen anderen Stellen.

30.2 Umrechnung von Binärzahlen in Dezimalzahlen

Die Umrechnung von Binärzahlen in Dezimalzahlen ist vergleichsweise einfach und basiert auf der Berechnung der Stellenwerte im Binärsystem.

30.2.1 Methode:

1. Identifiziere jede Stelle in der Binärzahl und ihre Position (von rechts nach links beginnend bei 0).
2. Multipliziere jede Binärziffer (0 oder 1) mit dem Wert von 2 hoch der Stellenposition.
3. Addiere alle resultierenden Werte zusammen.

30.2.2 Beispiel: Umrechnung von 10110 ins Dezimalsystem

Lassen Sie uns die Binärzahl 10110 in eine Dezimalzahl umrechnen:

Position (von rechts)	4	3	2	1	0
Binäre Ziffer	1	0	1	1	0
Berechnung	$1 \cdot 2^4$	$0 \cdot 2^3$	$1 \cdot 2^2$	$1 \cdot 2^1$	$0 \cdot 2^0$
Dezimalwert	16	0	4	2	0

Nun addieren wir alle Dezimalwerte: $16 + 0 + 4 + 2 + 0 = 22$

Also ist 10110_2 im Binärsystem gleich 22 im Dezimalsystem.

30.2.3 Formel:

Für eine Binärzahl mit n Stellen ($b_0, b_1, \dots, b_{n-1}$) gilt:

$$\text{Dezimalzahl} = b_0 \cdot 2^0 + b_1 \cdot 2^1 + b_2 \cdot 2^2 + \dots + b_{n-1} \cdot 2^{n-1}$$

wobei b_0 die erste Stelle von rechts ist.

30.2.4 Praktischer Trick:

Bei der Umrechnung kann man auch eine Tabelle mit den Stellenwerten erstellen:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Man schreibt die Binärzahl unter die Tabelle und addiert nur die Werte, an deren Position eine 1 steht.

30.3 Binärcodierung von Strings

In der Informatik werden Textzeichen (Strings) durch binäre Codes repräsentiert. Bei Unicode-Zeichen wird eine fortschrittliche Codierung verwendet, um sowohl einfache als auch komplexe Schriftzeichen aus allen Sprachen der Welt darstellen zu können.

30.3.1 Unicode: Ein universeller Zeichencode

Unicode ist ein internationaler Standard zur Darstellung von Textzeichen aller Schriftsysteme. Jedes Zeichen erhält eine eindeutige Nummer (Codepoint), z.B. hat der Buchstabe "A" den Codepoint U+0041 (dezimal: 65).

30.3.2 UTF-8: Die häufigste Unicode-Codierung

UTF-8 ist die am weitesten verbreitete Codierungsform für Unicode. Sie hat folgende Eigenschaften:

1. **Variable Länge:** Ein Zeichen wird mit 1 bis 4 Bytes codiert
2. **Abwärtskompatibilität:** ASCII-Zeichen (0-127) werden mit nur 1 Byte dargestellt
3. **Selbstsynchronisierend:** Der Anfang eines Zeichens ist eindeutig erkennbar

30.3.3 UTF-8 Codierungsschema:

Anzahl Bytes	Binärmuster	Codepoint-Bereich
1	0xxxxxxx	U+0000 bis U+007F
2	110xxxxx 10xxxxxx	U+0080 bis U+07FF
3	1110xxxx 10xxxxxx 10xxxxxx	U+0800 bis U+FFFF
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	U+10000 bis U+10FFFF

Die mit x markierten Stellen werden mit den Bits des Unicode-Codepoints gefüllt.

30.3.4 Beispiel: Codierung deutscher Umlaute

Nehmen wir als Beispiel den Buchstaben “ü” (U+00FC):

1. Der Codepoint von “ü” ist U+00FC (dezimal: 252)
2. Als Binärzahl: 11111100
3. Da der Wert > 127 ist, benötigen wir 2 Bytes
4. Nach dem 2-Byte-Schema: 110xxxxx 10xxxxxx
5. Wir füllen den Unicode-Wert ein: 110(00000) 10(111100) → 11000011 10111100
6. Somit wird “ü” als die zwei Bytes 11000011 10111100 codiert

30.3.5 Weiteres Beispiel: Das Euro-Zeichen €

1. Der Codepoint von “€” ist U+20AC (dezimal: 8364)
2. Als Binärzahl: 10000010101100
3. Da der Wert > 2047 ist, benötigen wir 3 Bytes
4. Nach dem 3-Byte-Schema: 1110xxxx 10xxxxxx 10xxxxxx
5. Wir füllen den Unicode-Wert ein: 1110(0010) 10(000010) 10(101100) → 11100010 10000010 10101100
6. Somit wird “€” als die drei Bytes 11100010 10000010 10101100 codiert

30.3.6 Beispiel: ‘Informatik ist interessant.’

Lassen Sie uns den Satz “Informatik ist interessant.” in die binäre UTF-8-Kodierung umwandeln:

Zeichen	Unicode-Codepoint	Dezimal	Binär	UTF-8 Kodierung
I	U+0049	73	1001001	01001001
n	U+006E	110	1101110	01101110
f	U+0066	102	1100110	01100110
o	U+006F	111	1101111	01101111
r	U+0072	114	1110010	01110010
m	U+006D	109	1101101	01101101
a	U+0061	97	1100001	01100001
t	U+0074	116	1110100	01110100
i	U+0069	105	1101001	01101001
k	U+006B	107	1101011	01101011
ü	U+0020	32	100000	00100000
i	U+0069	105	1101001	01101001
s	U+0073	115	1110011	01110011
t	U+0074	116	1110100	01110100
ü	U+0020	32	100000	00100000
i	U+0069	105	1101001	01101001
n	U+006E	110	1101110	01101110
t	U+0074	116	1110100	01110100

Zeichen	Unicode-Codepoint	Dezimal	Binär	UTF-8 Kodierung
e	U+0065	101	1100101	01100101
r	U+0072	114	1110010	01110010
e	U+0065	101	1100101	01100101
s	U+0073	115	1110011	01110011
s	U+0073	115	1110011	01110011
a	U+0061	97	1100001	01100001
n	U+006E	110	1101110	01101110
t	U+0074	116	1110100	01110100
.	U+002E	46	101110	00101110

Da alle Zeichen in diesem Beispiel im ASCII-Bereich liegen (0-127), wird jedes Zeichen mit genau einem Byte codiert. Der vollständige Text benötigt also 27 Bytes im UTF-8 Format.

Die komplette binäre Darstellung des Textes ist:

```
01001001 01101110 01100110 01101111 01110010 01101101 01100001 01110100
01101001 01101011 00100000 01101001 01110011 01110100 00100000 01101001
01101110 01110100 01100101 01110010 01100101 01110011 01110011 01100001
01101110 01110100 00101110
```

In hexadezimaler Schreibweise:

```
49 6E 66 6F 72 6D 61 74 69 6B 20 69 73 74 20 69 6E 74 65 72 65 73 73 61 6E 74 2E
```

Dieser String kann direkt in einem Computer gespeichert und verarbeitet werden. In Textdateien, bei der Übertragung im Internet oder in Datenbanken sind Strings immer in solchen binären Formaten gespeichert.

30.3.7 Vorteile der Unicode-Codierung:

1. **Universalität:** Alle Schriftsysteme und Sonderzeichen können dargestellt werden
2. **Effizienz:** Häufig verwendete Zeichen benötigen weniger Speicherplatz
3. **Kompatibilität:** Rückwärtskompatibel mit ASCII, was die Integration erleichtert

Die Umwandlung zwischen Textzeichen und ihrer binären Repräsentation wird in Computersystemen automatisch durch Codierungs- und Decodierungsprozesse abgewickelt, sodass Benutzer sich in der Regel nicht mit den Details befassen müssen.

31 Python Funktionen zum Umrechnen zwischen den Zahlensystemen

Python stellt Funktionen zur Verfügung, mit deren Hilfe Zahlen zwischen den verschiedenen Zahlensystemen umgerechnet werden können. Trotzdem sollen in diesem Notebook eigene Funktionen implementiert werden, mit deren Hilfe Zahlen zwischen dem Dezimal- und Binärsystem umgerechnet werden können.

Ausserdem sollen Funktionen implementiert werden, mit welchen Strings in ihre binäre Repräsentation umgewandelt werden können und umgekehrt.

31.1 Umrechnung von Binär- in Dezimalzahlen

```
1 def bin2dec(binary: str) -> int:
2     # TODO: implementieren Sie hier die Funktion
3     pass
4
```

31.2 Umrechnung von Dezimal- in Binärzahlen

```
1 def dec2bin(decimal: int) -> str:
2     # TODO: implementieren Sie hier die Funktion
3     pass
```

31.3 Binäre Repräsentation von Strings

Für die Darstellung eines Strings in binärer Form wird auf die dezimale Repräsentation des Unicode-Zeichens zurückgegriffen. Dazu stellt Python die Funktion `ord()` zur Verfügung. Diese Funktion gibt die dezimale Repräsentation eines Unicode-Zeichens zurück.

```
1 def str2bin(s: str) -> str:
2     # TODO: implementieren Sie hier die Funktion
3     pass
```

31.4 Umwandlung der binären Repräsentation in einen String

Um aus der dezimalen Repräsentation eines Unicode-Zeichens wieder den entsprechenden String zu erhalten, stellt Python die Funktion `chr()` zur Verfügung.

```
1 def bin2str(binary: str) -> str:
2     # TODO: implementieren Sie hier die Funktion
3     pass
```

32 Python Funktionen zum Umrechnen zwischen den Zahlensystemen

Python stellt Funktionen zur Verfügung, mit deren Hilfe Zahlen zwischen den verschiedenen Zahlensystemen umgerechnet werden können. Trotzdem sollen in diesem Notebook eigene Funktionen implementiert werden, mit deren Hilfe Zahlen zwischen dem Dezimal- und Binärsystem umgerechnet werden können.

Ausserdem sollen Funktionen implementiert werden, mit welchen Strings in ihre binäre Repräsentation umgewandelt werden können und umgekehrt.

32.1 Umrechnung von Binär- in Dezimalzahlen

```
1 def bin2dec(binary: str) -> int:
2     decimal = 0
3     length = len(binary)
4
5     for i in range(1, length + 1):
6         decimal += int(binary[-i]) * 2 ** (i-1)
7
8     return decimal
9
```

32.2 Umrechnung von Dezimal- in Binärzahlen

```
1 def dec2bin(decimal: int) -> str:
2     binary = ''
3     while decimal > 0:
4         binary = str(decimal % 2) + binary
5         decimal = decimal // 2
6
7     return binary
```

32.3 Binäre Repräsentation von Strings

Für die Darstellung eines Strings in binärer Form wird auf die dezimale Repräsentation des Unicode-Zeichens zurückgegriffen. Dazu stellt Python die Funktion `ord()` zur Verfügung. Diese Funktion gibt die dezimale Repräsentation eines Unicode-Zeichens zurück.

```
1 def str2bin(s: str) -> str:
2     length_string = len(s)
3     binary = ''
4     for i in range(length_string):
5         c = s[i]
6         u10 = ord(c)
7         b = dec2bin(u10)
8         length_binary = len(b)
9         if i % 5 == 0 and i != 0:
10             binary += '\n'
11         b = '0' * (8 - length_binary) + b
12         binary += b + ' '
13     return binary.strip()
```

32.4 Umwandlung der binären Repräsentation in einen String

Um aus der dezimalen Repräsentation eines Unicode-Zeichens wieder den entsprechenden String zu erhalten, stellt Python die Funktion `chr()` zur Verfügung.

```
1 def bin2str(binary: str) -> str:
2     binary = binary.replace(' ', '')
3     binary = binary.replace('\n', '')
4     length = len(binary)
5     s = ''
6     for i in range(0, length, 8):
7         b = binary[i:i+8]
8         u10 = bin2dec(b)
9         c = chr(u10)
10        s += c
11    return s
```

33 Base64-Codierung

Die Base64-Codierung ist ein Verfahren zur Umwandlung von Binärdaten in eine Zeichenkette, die nur aus lesbaren ASCII-Zeichen besteht. Diese Codierung wird häufig verwendet, um binäre Daten über textbasierte Systeme zu übertragen, die möglicherweise nicht mit Binärdaten umgehen können (z.B. E-Mails). Eine weitere Anwendung ist die Einbettung von Bildern in HTML- oder Markdown-Dokumente.

Untenstehende Octocat-Grafik ist ein Beispiel für eine Base64-codierte Grafik.



Abbildung 33.1: Base64 codierte Octocat

Sofern die Website in Chrome geöffnet ist, kann die Grafik mit einem Rechtsklick “untersucht” werden. Die “Untersuchung” zeigt den Quellcode der Seite an. Dort kann die Base64-codierte Grafik gefunden werden. Diese beginnt mit `data:image/png;base64`, und setzt sich mit einer langen Zeichenkette fort. Diese Zeichenkette ist die Base64-Codierung der Grafik.

33.1 Prinzip der Base64-Codierung

Base64 wandelt 3 Bytes (24 Bits) Binärdaten in 4 druckbare ASCII-Zeichen um:

1. Die Binärdaten werden in 6-Bit-Blöcke aufgeteilt ($2^6 = 64$ mögliche Werte)
2. Jeder 6-Bit-Block wird in ein druckbares ASCII-Zeichen umgewandelt
3. Bei unvollständigen Blocks am Ende werden Füllzeichen (=) hinzugefügt

33.2 Base64-Zeichensatz

Der Base64-Zeichensatz besteht aus 64 Zeichen: - Die Großbuchstaben A-Z (26 Zeichen) - Die Kleinbuchstaben a-z (26 Zeichen) - Die Ziffern 0-9 (10 Zeichen) - Zwei zusätzliche Zeichen, meist '+' und '/' (2 Zeichen)

Diese 64 Zeichen repräsentieren die Werte 0-63 und können mit 6 Bits dargestellt werden.

33.3 Codierungsprozess

1. Die Binärdaten werden in Gruppen von 3 Bytes (24 Bits) aufgeteilt
2. Diese 24 Bits werden in vier 6-Bit-Blöcke umgewandelt
3. Jeder 6-Bit-Wert wird als Index für den Base64-Zeichensatz verwendet

Wenn die Anzahl der zu codierenden Bytes nicht durch 3 teilbar ist: - Bei einem übrig bleibenden Byte: Auffüllen mit vier Nullbits, Codierung ergibt zwei Zeichen und zwei '='-Zeichen - Bei zwei übrig bleibenden Bytes: Auffüllen mit zwei Nullbits, Codierung ergibt drei Zeichen und ein '='-Zeichen

33.4 Beispiele:

33.4.1 Beispiel 1: Codierung von "KBW"

Buchstabe	K							B							W													
Dezimal	75							66							87													
Binär	0	1	0	0	1	0	1	1	0	1	0	0	0	0	1	0	0	1	0	1	0	1	1	1				
Dezimal	18							52							9							23						
Base64	S							0							J							X						

Abbildung 33.2: Base64 Beispiel

33.4.2 Beispiel 2: Codierung von "Hallo"

Betrachten wir das Wort "Hallo":

1. ASCII-Werte: H=72, a=97, l=108, l=108, o=111
2. Binär: 01001000 01100001 01101100 01101100 01101111
3. In 6-Bit-Gruppen aufteilen: 010010 000110 000101 101100 011011 000110 1111
4. Da die letzte Gruppe nur 4 Bits hat, wird sie mit zwei Nullen aufgefüllt: 010010 000110 000101 101100 011011 000110 111100
5. Die 6-Bit-Werte sind: 18, 6, 5, 44, 27, 6, 60
6. Diese werden in Base64-Zeichen umgewandelt: S G F s b G 8 =

Das Ergebnis "SGFsbG8=" ist die Base64-Codierung von "Hallo".

33.5 Anwendungen der Base64-Codierung

- E-Mail-Anhänge (MIME)
- Datenkodierung in URLs
- Einbettung von Bildern in HTML/CSS (Data-URLs)
- Übertragung binärer Daten in JSON
- Speicherung von Binärdaten in XML

Base64 vergrößert die Datenmenge um etwa 33% (da 3 Bytes in 4 Zeichen umgewandelt werden), bietet aber den Vorteil, dass die codierten Daten in jedem Textformat sicher übertragen werden können.

34 Lernziele für die Prüfung vom 26. März 2025

Ich erwarte, dass Sie in der Lage sind,

- die folgenden Datenstrukturen zu beschreiben und ihre Implementation in Python grundsätzlich zu erklären:
 - Linked List
 - Stack
 - Queue sowie
 - Binary Search Tree;
- Zahlen vom Dezimalsystem ins Binärsystem umzurechnen und umgekehrt sowie
- zu erklären, wozu die Base64-Codierung verwendet wird und wie sie funktioniert.

And der Prüfung dürfen Sie eine handschriftliche Zusammenfassung im Umfang von einer Seite A4 verwenden. Das einzige, was nicht auf der Zusammenfassung stehen darf, sind die für das Arbeiten mit Jupyter Notebooks notwendigen Einrichtungsschritte.

Teil X

Künstliche Intelligenz

35 The Social Dilemma

The Social Dilemma ist ein Film von Netflix.

<https://www.netflix.com/title/81254224>

36 Was ist eine Künstliche Intelligenz?

Sie alle kennen heute **generative K.I.**-Tools. Wir werden zuerst K.I. anschauen, die keinen Inhalt generieren, sondern Entscheidungen fällen. Dies ist zu Beginn einfacher nachzuvollziehen, was ein Modell ist.

Nehmen Sie Kopfhörer nach vorne und absolvieren Sie selbstständig diesen Kurs: <https://studio.code.org/courses/oceans/units/1/lessons/1/levels/1>

37 Maschinelles Lernen: Punkt- und Linienklassifikatoren

Besuchen Sie die Seite <https://ricardoheinzmann.com/ml-class/level-selection>.

Spielen Sie in folgender Reihenfolge:

1. [Linien-Klassifizieren mit 2D-Daten](#)
2. [Punkt-Klassifizieren mit 2D-Daten](#)
3. [Punkt-Klassifizieren mit 3D-Daten](#)

38 Maschinelles Lernen: Gewinnstrategie

Spielen Sie das Spiel und entdecken Sie den Lernalgorithmus.

Bedienung:

- Klicken Sie auf einen blauen Spielstein, nachher auf das gefärbte Feld eins weiter unten.

<https://aieducation.computatrum.ch/>

Nach der “Übung 1: Lernalgorithmus in Aktion - Einfach” dürfen Sie selber entscheiden, ob Sie die selbe Übung mit einem *Mittel* schweren Spiel machen möchten, oder zur Übung 2 gehen.

Teil XI

Anleitungen

39 Installation von Python

Hier ist eine einfache Schritt-für-Schritt-Anleitung zur Installation von Python:

39.1 Python herunterladen und installieren

1. Öffnen Sie Ihren Webbrowser und gehen Sie zur offiziellen Python-Website: www.python.org
2. Klicken Sie auf der Startseite auf den Button “Downloads”
3. Suchen Sie die neueste Python-Version für Windows (Juli 24 Python 3.12.x)
4. Klicken Sie auf den Download-Button für die 64-bit-Version (falls nicht automatisch ausgewählt)
5. Nachdem der Download abgeschlossen ist, öffnen Sie die heruntergeladene Datei (sie endet auf .exe)
6. Im Installationsfenster setzen Sie ein Häkchen bei “Add Python x.x to PATH” (**sehr wichtig!**)
7. Klicken Sie auf “Install Now” und warten Sie, bis die Installation abgeschlossen ist
8. Klicken Sie am Ende auf “Close”

39.2 Installation überprüfen

1. Öffnen Sie die Windows-Eingabeaufforderung (cmd/Terminal):
 - Drücken Sie die Windows-Taste + R
 - Geben Sie “cmd” ein und drücken Sie Enter
2. In der Eingabeaufforderung/Terminal (dem neu aufgegangenen Fenster) tippen Sie:

```
1 python --version
```

und drücken Sie Enter

3. Wenn die Installation erfolgreich war, sehen Sie die installierte Python-Version (stimmt mit dem Überein, was auf dem Download-Button gestanden hat)

39.3 Erste Schritte mit Python

1. In der Eingabeaufforderung tippen Sie `python` und drücken Enter
2. Sie sehen nun den Python-Prompt (`>>>`). Der Prompt ist eine Eingabeaufforderung. Hier können Sie direkt Python-Code eingeben, z.B.:

```
1 print("Hallo, Welt!")
```

3. Drücken Sie Enter, um den Code auszuführen
4. Um die Python-Umgebung zu verlassen, geben Sie `exit()` ein und drücken Enter

Glückwunsch! Sie haben Python erfolgreich installiert und Ihr erstes Programm ausgeführt.

Mit dieser Anleitung sollten Sie Python problemlos installieren und erste Schritte in der Programmierung machen können. Viel Erfolg beim Lernen!

Dieser Text wurde mit Hilfe von Perplexity.ai erstellt.

40 Anleitung zum Arbeiten mit Python Virtual Environments

Eine Python Virtual Environment (venv) ist eine geschützte Umgebung, in der die Installation zusätzlicher Module zu keinen Konflikten mit dem Gesamtsystem führen. Aus diesem Grund ist es sinnvoll, für jedes neue Python Projekt eine solche venv anzulegen.

Hier eine Checkliste für das Arbeiten in einer venv:

Als Vorfrage ist zu prüfen, ob bereits eine venv angelegt ist.

40.1 Arbeiten mit einer bereits bestehenden venv

1. Aktivieren der venv:

```
1 C:\Pfad\zum\aktuellen\Ordner>venv\Scripts\activate
```

Dass die venv aktiviert worden ist, erkennt man daran, dass das Terminal folgenden Prompt aufweist:

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>
```

2. Jupyter Server starten:

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>jupyter notebook
```

Das Terminal, in welchem der Jupyter Server gestartet wurde, darf **nicht** geschlossen werden.

3. Arbeiten am Projekt

4. Jupyter Server deaktivieren.

Der Jupyter Server kann im Terminal, in welchem er gestartet wurde, mit der Tastenkombination [Ctrl] + [C] angehalten werden.

5. venv deaktivieren

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>deactivate
```

40.2 Einrichten einer neuen venv

1. Projektordner erstellen

2. venv anlegen:

```
1 C:\Pfad\zum\aktuellen\Ordner>python -m venv venv
```

3. Aktivieren der venv:

```
1 C:\Pfad\zum\aktuellen\Ordner>venv\Scripts\activate
```

Dass die venv aktiviert worden ist, erkennt man daran, dass das Terminal folgenden Prompt aufweist:

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>
```

4. Installieren der erforderlichen Pakete:

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>python -m pip install jupyter  
↪ (und allfällige andere Pakete)
```

5. Jupyter Server starten:

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>jupyter notebook
```

Das Terminal, in welchem der Jupyter Server gestartet wurde, darf **nicht** geschlossen werden.

6. Arbeiten am Projekt

7. Jupyter Server deaktivieren.

Der Jupyter Server kann im Terminal, in welchem er gestartet wurde, mit der Tastenkombination [Ctrl] + [C] angehalten werden.

8. venv deaktivieren

```
1 (venv) C:\Pfad\zum\aktuellen\Ordner>deactivate
```

41 Jupyter Notebook

In einem Jupyter Notebook kann Text mit ausführbarem Programmcode kombiniert werden. Wir werden Jupyter Notebooks für die Anwendungsübungen im Unterricht verwenden.

41.1 Installationsanleitung

Hier ist eine Schritt-für-Schritt-Anleitung zur Installation von Jupyter Notebooks:

1. Öffnen Sie die Windows-Eingabeaufforderung (cmd).
2. Wechseln Sie mit

```
1 cd \Pfad\zum\Arbeitsverzeichnis
```

in Ihr Arbeitsverzeichnis.

3. Starten Sie eine Python Virtual Environment

```
1 python -m venv venv
2
3 venv\Scripts\activate
```

4. Geben Sie folgenden Befehl ein und drücken Sie Enter:

```
1 python -m pip install jupyter
```

Die Installation braucht etwas Zeit. Haben Sie Geduld mit Ihrem Computer.

Falls pip aus irgendeinem Grund nicht installiert wurde oder nicht funktioniert, können Sie es manuell installieren, indem Sie den folgenden Befehl in der Eingabeaufforderung ausführen:

```
1 python -m ensurepip --upgrade
```

Dieser Befehl stellt sicher, dass pip installiert und auf dem neuesten Stand ist.

41.2 Jupyter Notebook starten

1. Öffnen Sie erneut die Eingabeaufforderung/das Terminal.
2. Geben Sie ein:

```
jupyter notebook
```

3. Drücken Sie Enter. Ihr Standardbrowser öffnet sich automatisch mit der Jupyter Notebook-Oberfläche.

41.3 Ihr erstes Notebook erstellen

1. Klicken Sie in der Jupyter-Oberfläche auf “New” und wählen Sie “Notebook”.
2. Wählen Sie im Pop-up Fenster “Python 3 (ipykernel)” und bestätigen Sie mit select.
3. Ein neues Notebook öffnet sich in einem neuen Tab. Der Tab trägt den Namen “Untitled”.
4. Geben Sie in die erste Zelle etwas Python-Code ein, z.B.:

```
1 print("Hallo Jupyter!")
```

5. Drücken Sie Shift+Enter, um die Zelle auszuführen.

41.4 Tipps zur Verwendung

- Verwenden Sie “Code”-Zellen für Python-Code und “Markdown”-Zellen für Text.
- Speichern Sie Ihr Notebook regelmäßig mit dem Disketten-Symbol oder Strg+S.
- Schließen Sie das Browserfenster und geben Sie in der Eingabeaufforderung/im Terminal Strg+C ein, um Jupyter zu beenden.

Glückwunsch! Sie haben nun Jupyter Notebooks installiert und können damit arbeiten. Erkunden Sie weitere Funktionen und Möglichkeiten, während Sie sich mit dem Tool vertraut machen.

41.5 Bearbeiten von Jupyter Notebooks in VS Code

1. Jupyter-Erweiterung installieren:
 - Suchen Sie im Erweiterungen-Bereich nach “Jupyter”
 - Installieren Sie die Erweiterung “Jupyter” von Microsoft
2. Neues Jupyter Notebook erstellen:
 - Drücken Sie Strg+Shift+P, um die Befehlspalette zu öffnen

- Geben Sie “Jupyter: Create New Jupyter Notebook” ein und wählen Sie diesen Befehl aus
- Ein neues Notebook wird geöffnet

3. Mit dem Notebook arbeiten:

- Um eine neue Zelle hinzuzufügen, klicken Sie auf den “+ Code” Button oben im Notebook
- Geben Sie Ihren Python-Code in die Zelle ein
- Um den Code auszuführen, klicken Sie auf den Play-Button links neben der Zelle oder drücken Sie Shift+Enter
- Die Ausgabe erscheint direkt unter der Zelle

4. Notebook speichern:

- Gehen Sie auf Datei > Speichern unter
- Wählen Sie einen Speicherort und geben Sie einen Namen mit der Endung .ipynb ein

5. Bestehendes Notebook öffnen:

- Gehen Sie auf Datei > Öffnen
- Navigieren Sie zu Ihrem .ipynb-File und öffnen Sie es

Tipps: - Sie können zwischen Code- und Markdown-Zellen wechseln, indem Sie auf den Zellentyp oben im Notebook klicken - Nutzen Sie die Befehlspalette (Strg+Shift+P), um weitere Jupyter-bezogene Befehle zu finden

Dieser Text wurde mit Hilfe von Perplexity.ai erstellt.

42 Plain Text Writing

In diesem Abschnitt geht es um einen Aspekt der digitalen Nachhaltigkeit. Digitale Nachhaltigkeit wird hier so verstanden, dass digitale Produkte unabhängig von konkreten Herstellern dargestellt werden können.

Um zu erklären, wo das Problem liegt, muss zuerst zwischen den zwei Dateitypen Textdateien und Binärdateien unterscheiden werden.

42.1 Text- und Binärdateien

Textdateien speichern ihren Inhalt als reinen Text. Formatierung in solchen Dateien erfolgen durch Steuerzeichen (vgl. z.B. ASCII-Tabelle, dort insbesondere die Beispiele der Zeichen 9 für einen horizontalen Tabulatoren oder Zeichen 10 für den Zeilenumbruch) und durch Konventionen (vgl. z.B. die Formatierungen in Markdown oder LaTeX). Entsprechend können sie in beliebigen Texteditoren gelesen werden.

Im Gegensatz dazu sind Binärdateien für ihre Erstellung, Bearbeitung und Darstellung grundsätzlich an Programme gebunden, welche ihren Standard lesen können. Ihre Inhalte werden als eine Folge von Bytes gespeichert. Ein Beispiel für eine Binärdatei ist ein in MS Word erstelltes Dokument.

Diese Unterscheidung zeigt, dass Textdateien grundsätzlich auf jedem funktionierenden Computer mit einem Texteditor angezeigt werden können. Im Gegensatz dazu erfordern Binärdateien in der Regel spezielle Programme, die das jeweilige Dateiformat interpretieren können, was zu einer gewissen Abhängigkeit von entsprechender Software führen kann.

Weiterführende Überlegungen zum Verfassen von Texten in Plain Text können den hier angeführten Texten entnommen werden.¹

42.2 Plain Text Formate für die Textredaktion

Die einfachste Form eines Textfiles ist eine `.txt` Datei. Dabei handelt es sich in der Regel um reinen Text ohne Formatierungen. Das Bedürfnis nach formatierten Textdarstellungen hat zu verschiedenen Dateiformaten geführt. Dazu gehören zum Beispiel HTML-Dateien (`.html` oder `.htm`: **H**yper **T**ext **M**arkup **L**anguage). Dieses Format wurde ursprünglich unter anderem auch dazu geschaffen, Text formatiert am Bildschirm darzustellen. Ein anderes Beispiel ist LaTeX. Auch LaTeX ist eine sogenannte Auszeichnungssprache (Markup Language). Allerdings ist das Zielmedium von

¹Kieran Healy, *The Plain Person's Guide to Plain Text Social Science* (2020); Blair Fix, «Why and How I Write Scientific Documents in Plain Text», in *Economics from the Top Down*, 2020.

LaTeX weniger der Bildschirm sondern eher der Druck, heute vor allem das Portable Document Format (PDF), welches sich zum Quasi-Standard entwickelt hat.

Allerdings ist der Umfang der Formatierungsmöglichkeiten sowohl in HTML wie auch in LaTeX so umfangreich, dass die Hürde für deren Verwendung relativ hoch ist.

Vor diesem Hintergrund ist eine eher minimalistische Auszeichnungssprache, Markdown (.md) geschaffen worden. Diese stellt alle wesentlichen Formatierungsmöglichkeiten, wie zum Beispiel hierarchische Überschriften oder das setzen von kursivem Text, direkt zur Verfügung. Dort wo die Formatierungsmöglichkeiten den Bedürfnissen nicht genügen, kann in vielen Fällen auf HTML oder LaTeX Auszeichnungen zurückgegriffen werden.

Eine Übersicht über die Formatierungsbefehle in Markdown findet sich auf der Website [Markdown Cheat Sheet](#). Wichtige Formatierungen in LaTeX (mathematische Formeln) werden auf der Website [Wikibooks LaTeX/Mathematics](#) erklärt.

43 Konvertierung von Markdown in PDF

Markdown hat den klaren Vorzug, in jedem Texteditor gelesen werden zu können. Leider ist reines Markdown nicht sehr ästhetisch. Zudem müssen die meisten Texte zu Händen des Empfängers in ein PDF konvertiert werden.

43.1 Vorbereitung der Konvertierung

Für diese Konvertierung kann das Programm Pandoc verwendet werden. Pandoc kann Markdown in unzählige Formate Konvertieren. Die genaue Vorgehensweise wird im Folgenden Schritt für Schritt erläutert.

1. Pandoc installieren

Das Installationsprogramm für Pandoc kann auf der Website von [Pandoc heruntergeladen](#) werden. Dazu ist dem Link "Download the latest Installer" zu folgen. Für Windows ist die Datei mit der Endung `.msi` herunterzuladen.

Das heruntergeladene Installationsprogramm kann anschliessend mit einem Doppelklick für die Installation ausgeführt werden.

2. LaTeX installieren

Damit Pandoc ein Markdown Dokument in ein PDF Konvertieren kann, braucht es auf dem Rechner eine Installation von LaTeX. LaTeX gibt es für Windows in verschiedenen Varianten. Die einfachste Installation erfolgt in der Variante von MiKTeX. Das entsprechende Installationspaket kann von der Website von [MiKTeX](#) heruntergeladen werden.

Das heruntergeladene Installationsprogramm kann anschliessend mit einem Doppelklick für die Installation ausgeführt werden.

3. Pandoc verwenden

Pandoc ist ein Programm für die Kommandozeile. Das heisst, es gibt keine grafische Oberfläche für die Verwendung von Pandoc. Am einfachsten kann Pandoc verwendet werden, wenn im Ordner, in dem sich das zu konvertierende File befindet, ein Terminal geöffnet wird.

Die Konvertierung erfolgt mit folgendem Befehl:

```
1 pandoc -o output.pdf input.md
```

Output.pdf und input.md muss an die eigenen Dateinamen angepasst werden. Alternativ können auch andere Dateiformate, wie .docx oder .html als Zielformate gewählt werden. Für eine abschliessende Liste aller möglichen Zielformate sei auf die Website von Pandoc verwiesen.

43.2 Spezifische Formatierung der Ausgabe

Die oben dargestellte Version führt zu einem sauber strukturierten PDF. Allerdings fehlen wesentliche Dokumentteile wie Deckblatt oder Inhaltsverzeichnis.

Für diese weiterführenden Formatierungen werden dem Dokument die erforderlichen Informationen entweder in einem speziellen Header vorangestellt oder in einem separaten Dokument dem Befehl für die Konvertierung übergeben. Hier soll die zweite Variante dargestellt werden.

Das Dokument mit den Informationen für die Formatierung ist ein sogenanntes YAML-Dokument .yaml. YAML steht dabei für Yet Another Markup Language. Das Dokument kann einen beliebigen Namen haben. Hier im Beispiel wird es format.yaml genannt. Das folgende Listing zeigt eine Formatierung mit dem Namen des Authors, dem Titel und weiteren Angaben. Was die einzelnen Einträge bedeuten, ist weitestgehend selbsterklärend.

```
1 from: markdown           # Ausgangsformat
2 to: pdf                  # Zielformat
3 standalone: true        # Erzwingt eigenständiges Dokument
4 pdf-engine: xelatex     # Wählt eine spezifische pdf-engine
5 output-file: output.pdf # Name des Ausgabedokuments
6 metadata:               # Dokumentspezifische Angaben
7   title: "Anleitung"    # Titel
8   author: "Jacques Mock Schindler" # Verfasser
9   lang: de-CH           # Sprache des Dokuments
10  fontsize: 11pt        # Schriftgrösse
11  geometry: left=2.5cm, right=3cm # Seitenränder
12  date: "9. Januar 2026" # Datum
13  toc: true             # Erstellt ein Inhaltsverzeichnis
14  number-sections: true # Nummeriert die Titel
```

Die Einrückung ist für die Syntax im YAML Dokument wichtig. Sie beträgt zwei Leerzeichen.

Wenn die Datei format.yaml das Format des PDF steuern soll, lautet der Befehl für die Konvertierung des Markdown Dokuments in ein PDF

```
1 pandoc --defaults=format.yaml input.md
```

Falls der Name des neu erstellten PDF Dokuments bei jedem Befehlsaufruf individuell festgelegt werden soll, kann die Zeile output-file: output.pdf in der YAML Datei auch weggelassen werden. Dann lautet der Befehl für die Konvertierung

```
1 pandoc --defaults=format.yaml -o output.pdf input.md
```

Falls für komplexe Formatierungen auf externe LaTeX Pakete abgestellt werden muss, werden diese über ein separates `.tex` Dokument geladen. Auch dieses Dokument kann grundsätzlich beliebig benannt werden. Für das Beispiel hier wird die Datei als `header.tex` bezeichnet. Der Inhalt dieser Datei ist folgendermassen zu formatieren:

```
1 \usepackage{caption}
```

Der Befehl für die Konvertierung lautet jetzt

```
1 pandoc --defaults=format.yaml -H header.tex input.md
```

Die Website [CTAN Comprehensive TeX Archive Network](#) gibt Auskunft darüber, was für LaTeX Pakete zur Verfügung stehen.

44 Referenzen in Markdown

Damit in Texten, die im Markdown Format erstellt werden, Literaturzitate eingefügt werden können, braucht es neben dem Markdown Dokument mit dem eigentlichen Text, drei weitere Dateien:

- eine Datei mit allen Formatierungsinformationen (.yaml),
- eine Datei mit den bibliographischen Angaben (.bib) sowie
- eine Datei mit den Formatierungsangaben für die Literaturdarstellung (.cs1).

Diese Dateien müssen sich alle im gleichen Verzeichnis befinden. Falls dies nicht der Fall ist, müssen sie in der Konfigurationsdatei (.yaml) mit dem Pfad angegeben werden.

Von diesen drei Dateien muss lediglich die Datei mit den Formatierungsinformationen manuell erstellt werden. Die Datei mit den bibliographischen Angaben wird von Zotero erstellt und die Datei für die Darstellung der bibliographischen Angaben im Text kann aus dem [Zotero Style Repository](#) heruntergeladen werden.

44.1 Textformat (.yaml)

Die Darstellung des Textes sowie dessen Ausgabeformat kann über die Datei `format.yaml` gesteuert werden. Der Name der Datei ist frei wählbar. Hier wurde `format.yaml` gewählt, weil die Datei genau das macht: das Format der Ausgabe steuern. Im folgenden Listing findet sich ein Beispiel mit den Angaben für die Verwendung einer `.bib` und `.cs1` Datei.

```
1 # Input und Output
2 input-files:
3   - mein-text.md # es können auch mehrere Dateien angegeben werden
4   # - zweiter_teil.md
5   # - dritter_teil.md
6 output-file: ausgabe.pdf
7
8 # Grundeinstellungen
9 from: markdown
10 to: pdf
11 # lualatex als pdf-engine hilft bei der Darstellung
12 # von Umlauten und anderen Sonderzeichen
13 pdf-engine: lualatex # oder xelatex, pdflatex
14
15 # Aktiviert citeproc - damit wird die Bibliographie eingebunden
```

```

16 citeproc: true
17
18 # Inhaltsverzeichnis
19 table-of-contents: true
20
21 # Variablen und Metadaten
22 variables:
23   lof: true
24   lot: true
25   number-sections: true
26   header-includes:
27     - \setcounter{secnumdepth}{3}
28     - \setcounter{tocdepth}{3}
29
30 metadata:
31   title: "Titel des Dokuments"
32   author: "Ihr Name"
33   lang: de-CH
34   date: "Heute"
35   bibliography: bibliography.bib
36   csl: chicago-notes-bibliography-access-dates.csl
37   link-citations: true

```

Hier die Erklärungen zu den einzelnen Einträgen in dieser Konfigurationsdatei:

- `input-files` Hier werden alle Dateien erfasst, welche in das finale Ausgabedokument einfließen sollen. Die einzelnen Dateien werden wie in der Vorlage angedeutet untereinander als Aufzählung aufgelistet.
- `output-file` Mit diesem Schlüssel wird der Pfad zur Ausgabedatei mit deren Namen erfasst.
- `from` ist der Schlüssel für das Format der `input-files`.
- `to` ist der Schlüssel für das Format des `output-file`.
- `pdf-engine` legt fest, mit welchem Programm das Markdown Dokument in eine PDF Datei umgewandelt wird. Aktuell ist `lualatex` die modernste Variante. `lualatex` unterstützt Schweizerische Umlaute und Akzente sehr gut. Aus diesem Grund wurde hier `lualatex` ausgewählt.
- `citeproc` ist das Programm, dass für die Übernahme der bibliographischen Angaben verantwortlich ist. Mit dem Wert `true` wird dessen Verwendung aktiviert.
- `table-of-contents` aktiviert, wenn der Wert `true` ist, die Erstellung eines Inhaltsverzeichnisses.
- `variables` stellt die Details für die Darstellung des Ausgabedokuments zur Verfügung.
- `lof` erstellt ein Abbildungsverzeichnis.
- `lot` erstellt ein Tabellenverzeichnis.
- `number-sections` nummeriert die Titel ihrer Hierarchiestufe gemäss.
- `header-includes` ermöglicht es, LaTeX Formatierungsbefehle zu verwenden.

- - `\setcounter{secnumdepth}{3}` ist eine LaTeX Formatierung, die dafür sorgt, dass Titel nur über drei Hierarchieebenen hinweg nummeriert werden.
- - `\setcounter{tocdepth}{3}` ist eine LaTeX Formatierung, welche nur Titel bis zur dritten Hierarchieebene ins Inhaltsverzeichnis aufgenommen werden.
- `metadata` speichert die inhaltlichen Eckwerte für das Dokument.
- `title` ist jener Titel, der auf dem Deckblatt abgedruckt wird.
- `author` ist der Name des Autors.
- `lang` ist die Variabel zur Speicherung der Sprache des Ausgabedokuments. Indem sie hier auf `de-CH` gesetzt wird, werden die typographischen Besonderheiten der Deutschschweiz berücksichtigt.
- `date` ist das Datum. Für Dokumente, welche ein PDF als Ausgabeformat haben, kann auch der Befehl `9. Juni 2026` verwendet werden. Dieser liest bei der Konvertierung von Markdown zu PDF das aktuelle Systemdatum aus.
- `bibliography` gibt den Pfad zur Datei mit den bibliographischen Angaben an.
- `cs1` gibt den Pfad zur Datei mit dem gewählten Zitierstil an.

Weitere Konfigurationsmöglichkeiten für die Konvertierung in ein PDF finden sich [Pandoc User Guide](#).

44.2 Bibliographische Angaben

Das Dokument mit den bibliographischen Angaben kann Zotero automatisch erstellen. Trotzdem wird hier ein Beispiel eines Eintrags in diesem Dokument erklärt. Das Dokument für das Beispiel heisst `bibliography.bib`.

```

1 @book{healy2020,
2   title = {The {{Plain Person}}'s {{Guide}} to {{Plain Text Social
   ↪ Science}}},
3   author = {Healy, Kieran},
4   year = 2020,
5   }

```

Der Eintrag bezieht sich auf ein Buch (`@book`). Unmittelbar auf diesen Identifikator folgt nach der geschweiften Klammer der Zitierschlüssel (`healy2020`). Dieser wird von Zotero grundsätzlich automatisch vergeben, kann aber auch manuell festgelegt werden. Anschliessend folgen Zeile für Zeile die notwendigen bibliographischen Angaben als Schlüssel - Werte Paare.

44.3 Zitierstil

Der Zitierstil wird in einem Dokument mit der Endung `.cs1` definiert. Die Abkürzung `cs1` steht für Citation Style Language und ist eine Form von [xml](#). Grundsätzlich ist es möglich, eigene `cs1` Definitionen zu verfassen. Das [Zotero Style Repository](#) stellt jedoch die gängigsten Zitierstile zur

Verfügung. Dort kann der gewünschte Stil heruntergeladen und im Projektverzeichnis gespeichert werden.

44.4 Einfügen von Referenzen in Markdown

Um Referenzen aus dem Bibliographie Dokument in den Markdown Text zu übernehmen, verwendet man den Zitierschlüssel. Eingefügt wird er mit `[@zitierschlüssel]`. Soll die Referenz mit einer Seitenzahl versehen werden, wird der Eintrag folgendermassen ergänzt: `[@zitierschlüssel, 25]`. Sollen in der gleichen Referenz mehrere Belege angeführt werden, sieht die Syntax folgendermassen aus: `[@zitierschlüssel1, 25; @zitierschlüssel2, 33]`.

Die Zitierschlüssel braucht man nicht auswendig zu wissen. Die Zotero Einträge zeigen sie als obersten Eintrag zu jedem erfassten Werk an.

44.5 Erstellen des PDF mit Referenzen

Sofern ein entsprechendes Konfigurationsdokument (`.yaml`) erstellt worden ist, kann das PDF mit dem Aufruf

```
1 pandoc -d format.yaml
```

erstellt werden. Damit der Befehl funktioniert, muss er im Verzeichnis, in welchem die Datei `format.yaml` liegt, ausgeführt werden.

45 Literaturverwaltung mit Zotero

Zotero bewirbt sich selber als persönlicher Forschungsassistent. Im gymnasialen Umfeld ist Zotero in erster Linie ein Hilfsmittel Zitate in Texten korrekt zu formatieren. Darüber hinaus kann Zotero auch als E-Reader mit Zusatzfunktionen zur Verwaltung von Notizen verwendet werden.

45.1 Zotero installieren und einrichten

Um die Funktionen von Zotero im vollen Umfang zu nutzen, ist es sinnvoll ein Zotero Account zu erstellen. Der Account kann auf der Website von [Zotero](#) erstellt werden. Wenn der Account nicht ausschliesslich privat genutzt werden soll, ist es sinnvoll die Schul- bzw. Geschäftsemail zu verwenden. Im Nutzerprofil können auch mehrere Adressen hinterlegt werden. Das ermöglicht es, die eigene Datenbank zu behalten, wenn man eine Institution verlässt.

Nachdem optional ein persönlicher Account erstellt worden ist, kann der [Zotero Desktop Client](#) heruntergeladen und installiert werden. Auf der gleichen Seite findet sich auch die Möglichkeit den Zotero Connector als Browser Plugin herunterzuladen. Dieses Plugin ist für die Arbeit mit Zotero nicht zwingend erforderlich, vereinfacht aber die Erfassung von Quellen ungemein.

Wenn Zotero installiert ist, kann es über den Menüpunkt Bearbeiten > Einstellungen, Registerkarte Sync mit dem eigenen Zotero Account verbunden werden.

Für die Verwendung von Zotero in einem Markdown – Pandoc-Workflow ist die Installation des [BetterBibTeX](#) Plugins in Zotero unbedingt erforderlich. Das Plugin kann direkt aus seinem [GitHub Repository](#) heruntergeladen werden. Das heruntergeladene .xpi Dokument kann über das Menü Werkzeuge > Plugins, Zahnradsymbol, Install Plugin from File, installiert werden. Nach der Installation kann die .xpi Datei gelöscht werden.

45.2 Einträge in Zotero erstellen und organisieren

Wie Einträge in Zotero erfasst werden, wird auf der Website von Zotero ausführlich beschrieben. An dieser Stelle darf daher auf die dortigen [Erklärungen](#) verwiesen werden. Das gleiche gilt für die [Organisation](#) der eigenen Bibliothek.

Damit die Einträge aus Zotero in den Texten gelesen werden können, müssen sie mit dem Zitierschlüssel (citation key) adressiert werden. Die genaue Syntax dazu wird in Kapitel [44.4](#) erklärt. Zotero erstellt für jeden Eintrag einen Standardschlüssel. Dieser ist jedoch oft schwer zu merken. Es empfiehlt sich daher, den Zitierschlüssel selber festzulegen. Dazu ist im Feld “Extra” der Metadaten des Eintrags das “Key – Value” Paar “citation key: eigener_schlüssel” einzutragen. Gespeichert wird

der so festgelegte Zitierschlüssel in dem das Feld “Extra” wieder verlassen wird. Der einfachste Zitierschlüssel ist wohl `authorYYYY` wobei `YYYY` für die vierstellige Jahreszahl steht.

45.3 Erstellen einer Bibliographie Datei für den Markdown – Pandoc-Workflow

Die für das jeweilige Projekt erforderlichen Einträge in Zotero werden sinnvollerweise in einer Sammlung zusammengefasst. Diese Sammlung kann dann in eine `.bib` Datei exportiert werden. Dazu ist das Kontextmenü der Sammlung mit Rechtsklick zu öffnen. Anschliessend ist der Menüpunkt ‘Sammlung Exportieren’ auszuwählen. In der sich öffnenden Dialogbox ist Better BibLaTeX als Format auszuwählen. Sinnvollerweise werden die beiden Optionen ‘Halte aktuell’ und ‘Hintergrund-Export’ ausgewählt. Die erste Option sorgt dafür, dass Änderungen in Zotero-Einträgen automatisch in die `.bib` Datei geschrieben werden, die zweite verhindert, dass der Export einfriert.

Standardmässig erhält die so erstellte `.bib` Datei den Namen der Sammlung. Der Name kann während des Erstellens noch angepasst werden. Falls der Name nachträglich angepasst wird, ist nicht sichergestellt, dass die automatische Aktualisierung weiterhin funktioniert. Falls man dem Automatismus nicht vertraut, kann die `.bib` Datei nach jeder Änderung manuell neu aus Zotero exportiert werden.

Teil XII

Alte Programme

46 Programm im Frühlingssemester 2025

Datum	Thema
19.02.2025	Queues Implementieren in Python
26.02.2025	BST Implementieren in Python
05.03.2025	BST Implementieren in Python
12.03.2025	Binärsystem
19.03.2025	Base64 Codierung
26.03.2025	Test (Datenstrukturen & Datencodierung)
02.04.2025	Datenbanken
09.04.2025	Datenbanken
16.04.2025	Datenbanken
07.05.2025	Datenbanken
14.05.2025	Datenbanken: Übungen
21.05.2025	Test (Datenbanken)
28.05.2025	Datenvisualisierung mit Python
04.06.2025	Datenvisualisierung mit Python
11.06.2025	Datenvisualisierung mit Python
18.06.2025	Datenvisualisierung mit Python
25.06.2025	Datenvisualisierung mit Python
02.07.2025	Datenvisualisierung mit Python
09.07.2025	Datenvisualisierung mit Python